Polymorphic Symmetric Multiple Dispatch with Variance (Extended with Formalization)

GYUNGHEE PARK, Oracle Labs and KAIST
JAEMIN HONG, KAIST
GUY L. STEELE JR., Oracle Labs
SUKYOUNG RYU, KAIST

Many object-oriented languages provide *method overloading*, which allows multiple method declarations with the same name. For a given method invocation, in order to choose what method declaration to invoke, *multiple dispatch* considers the run-time types of the arguments. While multiple dispatch can support binary methods (such as mathematical operators) intuitively and consistently, it is difficult to guarantee that calls will be neither ambiguous nor undefined at run time, especially in the presence of expressive language features such as *multiple inheritance* and *parametric polymorphism*. Previous efforts have formalized languages that include such features by using *overloading rules* that guarantee a unique and type-sound resolution of each overloaded method call; in many cases, such rules resolve ambiguity by treating the arguments asymmetrically. Here we present the first formal specification of a strongly typed object-oriented language with *symmetric* multiple dispatch, multiple inheritance, and parametric polymorphism with *variance*. We define both a static (type-checking) semantics and a dynamic (dispatching) semantics and prove the type soundness of the language, thus demonstrating that our novel dynamic dispatch algorithm is consistent with the static semantics. Details of our dynamic dispatch algorithm address certain technical challenges that arise from structural asymmetries inherent in object-oriented languages (e.g., classes typically declare ancestors explicitly but not descendants).

1 INTRODUCTION

In object-oriented languages, *method overloading* and its unique resolution are important features. Method overloading allows multiple method declarations of the same name but different parameter types. Thus, it enables to define different behaviors of + or `equal` for different parameter types, for example. For a given overloaded method invocation of `equal`, a resolution mechanism should select a single declaration to call among the overloaded declarations of `equal` using the argument types of the method invocation.

When resolving an overloaded method invocation, *dynamic dispatch* considers the run-time types of the arguments of the method invocation. When a dispatch mechanism considers the run-time type of only a single argument, usually the receiver of a method call, as in languages like C++ and Java, it is *single dispatch*. By contrast, *multiple dispatch* considers the run-time types of more than one argument as in languages like Common Lisp, Fortress, and Julia. Especially, *symmetric multiple dispatch* considers multiple arguments equally, which supports mathematical operators like binary methods intuitively and uniformly [Clifton et al. 2000].

While method overloading and its resolution by dynamic dispatch give a consistency in naming methods that logically perform very similar tasks, they impose two restrictions to guarantee a unique and type-sound resolution of each overloaded method invocation [Castagna et al. 1992, 1995]. First, the dispatch mechanism should be able to find the *most specific applicable* method declaration for any overloaded method invocation. A method declaration is applicable to given arguments if its parameter types are supertypes of the types of the arguments. One method declaration is more specific than another if the *domain type* of the former is a subtype of that of the latter. Hereafter,

2018. 2475-1421/2018/1-ART1 $15.00
https://doi.org/

we consider that every method declaration has a single parameter possibly a tuple of two or more components, and call the parameter type of a method declaration its domain type. Second, for a given overloaded method invocation, its static type should be a supertype of the return type of a dynamically chosen method declaration by the dispatch mechanism. Otherwise, a method invocation result may be used in a type-unsound manner.

Moreover, expressive language features often add various possibilities to make overloaded method invocations ambiguous. As we discuss below, such features make it difficult to design a language that guarantees no ambiguous nor undefined calls at run time.

One such a feature is *multiple inheritance*, which allows one class to extend multiple classes. While multiple inheritance supports modular and reusable code, it easily introduces ambiguous method calls. Because one parent class may not be more specific than another parent class, dynamically selecting the most specific applicable method declaration may be unclear. For example, consider that each of two classes A and B has a method declaration m with the same parameter type `Number`, class C extends both A and B, and c is an instance of C. Then, evaluation of $c.m(0)$ results in an ambiguous call because neither A nor B is more specific than each other.

Another feature is *parametric polymorphism*, or *generics*. By introducing type parameters in class and method declarations, parametric polymorphism enables programmers to manage types in a fine-grained way. For example, the `List` class may be instantiated as either `List⟦String⟧` or `List⟦Integer⟧`. However, selecting the most specific applicable declaration among polymorphic method declarations at run time is not trivial. Moreover, many languages with parametric polymorphism support automatic inference for the type arguments of polymorphic method invocations; at run time, dynamic dispatch must not only find the most specific applicable declaration, but also infer appropriate type arguments for that declaration.

Finally, *variance* defines additional subtype relations for polymorphic types [Igarashi and Viroli 2006]. Without variance, instances of the same polymorphic type are in a subtype relation only when their type arguments are the same type. Variance supports subtype relationships between different instances of the same polymorphic type by allowing the programmer to label each type parameter of a class *covariant*, *contravariant*, or *invariant*. If a parametric type T has one covariant type parameter, then it preserves the subtype relation of its type argument; hence, when a type P is a subtype of another type Q, which we write $P <: Q$, it implies $T⟦P⟧ <: T⟦Q⟧$. If the type parameter of T is contravariant, then T reverses the subtype relationship of its type arguments; hence, $P <: Q$ implies $T⟦Q⟧ <: T⟦P⟧$. If the type parameter of T is invariant, then $T⟦P⟧ <: T⟦Q⟧$ only if $P \equiv Q$, where $P \equiv Q$ denotes that P and Q are equivalent, that is, P is a subtype of Q and vice versa. Variance annotations allow the programmer to define more expressive polymorphic methods. For example, consider the following excerpt from the `List` class source code in Scala [EPFL 2017]:

```
sealed abstract class List[+A] extends Seq[A] {
    ...
    def :::[B >: A](prefix: List[B]): List[B] = ···
    ···
}
```

The `List` class takes a single covariant (denoted by `+`) type parameter `A`, and the `:::` method concatenates two lists, its receiver and argument. If `A` were invariant, the element type `B` of the argument list should be a supertype of the element type `A` of the receiver list. However, since `A` is covariant, the method can take a list of any element type `C` that is less than or equal to `B`, and return a list whose element type is `B`, which is a common supertype of the element types of its receiver and argument: `A<:B` and `C<:B`. As one might expect, variance also increases the difficulty of overloading resolution.

Therefore, supporting symmetric multiple dispatch in the presence of various expressive language features has been a long-standing effort. Previous efforts have formalized a variety of core calculi that capture some of the aforementioned features in a piecemeal rather than holistic fashion. While they are widely different in terms of the expressivity, completeness, decidability, and correctness of the formalization of the target languages and their type systems, they all define a set of *overloading rules* that overloaded method declarations should satisfy to make dynamic dispatch select the most specific applicable method declaration with a type-safe return type.

For a simple object-oriented language with multiple inheritance, Allen et al. [2008, 2007] and Kim and Ryu [2011] presented three overloading rules: Exclusion Rule, Subtype Rule, and Meet Rule. Every pair of method declarations with the same name must satisfy *one* of these overloading rules. The authors proved that a set of overloaded method declarations satisfying the overloading rules does not produce any ambiguous method invocation at run time.

To understand the interplay between symmetric multiple dispatch and parametric polymorphism, Allen et al. [2011] focused on a type system without any specific target language and informally presented three other overloading rules: a No Duplicates Rule, a revised Meet Rule, and a Return Type Rule, where every pair of method declarations with the same name must satisfy *all three* overloading rules. The authors showed that interpreting a polymorphic method declaration as a set of infinitely many monomorphic declarations is not appropriate for overloading resolution. Instead, they considered a polymorphic method declaration as a single declaration whose domain type is existentially quantified over its type parameters as proposed by Bourdoncle and Merz [1997].

In this paper, we present a formal core calculus *FGFV* (Featherweight Generic Fortress with Variance) for an object-oriented language with symmetric multiple dispatch, multiple inheritance, and parametric polymorphism with variance. For conciseness, FGFV omits functional methods [Allen et al. 2007] and declarations of closed types [Allen et al. 2011], but these could be added back to FGFV in an obvious way. Unlike the prior work, we formally present overloading rules for FGFV, define its dynamic semantics with a novel dynamic dispatch algorithm, and prove type soundness. As we discuss in Section 5.4, defining a provable system for an object-oriented language with asymmetric features requires careful tradeoffs between static expressivity and dynamic efficiency.

In the rest of the paper, we first describe the previous efforts in more detail with concrete code examples (Section 2). After presenting the FGFV calculus that represents an object-oriented language with symmetric multiple dispatch, multiple inheritance, parametric polymorphism, and variance (Section 3), we formally define the overloading rules for FGFV (Section 4). Then, we define both static and dynamic overloading resolution with a novel dynamic dispatch algorithm, and prove the type soundness of FGFV (Section 5). Finally, we discuss related work (Section 6) and conclude (Section 7).

2 MOTIVATION

In this section, we first discuss existing research achievements in the pursuit of formally defining object-oriented languages with symmetric multiple dispatch, multiple inheritance, and parametric polymorphism with variance. Then, we present how FGFV completes the long-standing endeavor by defining a core calculus with static overloading rules and a dynamic dispatch algorithm that are proven type-sound.

2.1 Overloading Rules for Multiple Inheritance

As we discussed in Section 1, both symmetric multiple dispatch and multiple inheritance may cause ambiguous method calls that cannot be resolved at run time. Let us assume that class C is the only subtype of both classes A and B, and they are all distinct throughout this section. Consider the

following overloaded method declarations of m_1:

$$m_1(x\colon B, y\colon C)\colon B = \cdots \qquad\qquad m_1(x\colon C, y\colon B)\colon B = \cdots$$

which have the same return type B but different domain types (B, C) and (C, B). For a method invocation $m_1(c, c)$ where the run-time type of c is C, it is ambiguous which declaration to invoke due to symmetric multiple dispatch: both method declarations are applicable to (c, c) since both (B, C) and (C, B) are supertypes of (C, C), and neither declaration is more specific than the other. Also, consider the following overloaded declarations of m_2:

$$m_2(x\colon A)\colon \texttt{Boolean} = \cdots \qquad\qquad m_2(x\colon B)\colon \texttt{Boolean} = \cdots$$

which have domain types A and B, respectively. For a method invocation $m_2(c)$, it is ambiguous which declaration to invoke due to multiple inheritance: both declarations are applicable to c and neither declaration is more specific than the other.

In order to guarantee that no such ambiguous method calls are possible at run time, researchers have proposed several calculi with restrictions on overloaded method declarations. CF [Allen et al. 2007], Core Fortress, is a core calculus of the Fortress programming language [Allen et al. 2008], which contains components, top-level functions, functional methods, and declarations of exclusive types in addition to multiple inheritance. However, it does not specify the language semantics and informally provides rules for valid overloaded method declarations. FFMM [Kim and Ryu 2011], Featherweight Fortress with Multiple Dispatch and Multiple Inheritance, simplified CF to focus only on multiple inheritance, and it has a complete formalization of the language semantics including overloading rules and its type soundness proof in Coq. Since the overloading rules of CF and FFMM are the same, we will refer to these two languages collectively as *Featherweight Fortress (FF)*.

The overloading rules of FF require every pair in a set of overloaded method declarations to satisfy *one* of the following rules:

Exclusion Rule: Their domain types are *exclusive* (their intersection is `Bottom`, where the `Bottom` type is a subtype of every type and no value belongs to `Bottom`).

Subtype Rule: The domain type of one declaration is a strict subtype of the domain type of the other, and the return type of the former is a subtype of the return type of the latter.

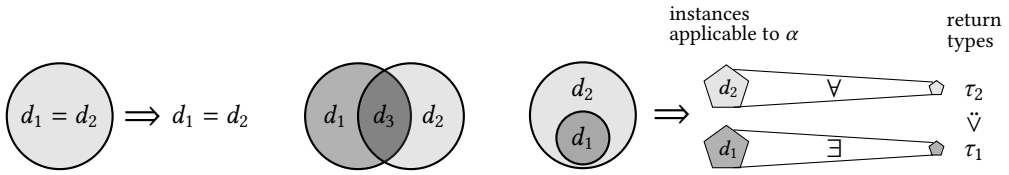
Meet Rule: The domain types are not exclusive nor in a subtype relation, and the overloaded set contains a third disambiguating method declaration which we call the *meet* of the pair. The meet of two method declarations is more specific than each of the declarations and less specific than any other method declaration that is more specific than both declarations.

Let us revisit the previous examples, with an additional method declaration each for m_1 and m_2:

$$\begin{array}{lll} d_1^1 : m_1(x\colon B, y\colon C)\colon B = \cdots & d_1^2 : m_1(x\colon C, y\colon B)\colon C = \cdots & d_1^3 : m_1(x\colon C, y\colon C)\colon C = \cdots \\ d_2^1 : m_2(x\colon A)\colon \texttt{Boolean} = \cdots & d_2^2 : m_2(x\colon B)\colon \texttt{Boolean} = \cdots & d_2^3 : m_2(x\colon C)\colon \texttt{Boolean} = \cdots \end{array}$$

Note that, for both m_1 and m_2, the third method declaration is the meet of the first two declarations. Each set of overloaded method declarations is valid since every pair satisfies one of the overloading rules (for $i = 1$ and 2, d_i^1 and d_i^2 satisfy the Meet Rule; d_i^1 and d_i^3 satisfy the Return Type Rule, as do d_i^2 and d_i^3).

The overloading rules guarantee two important properties: (1) no ambiguous method calls can occur at run time, and (2) types are preserved even when a method call is dispatched to a different method declaration at run time from a method declaration chosen at compile time. In FF, Exclusion Rule is for the trivial cases where no ambiguous calls can occur, Meet Rule ensures the existence of disambiguating method declarations, and Subtype Rule serves a key role for type preservation.



(a) No Duplicates Rule (b) Meet Rule (c) Return Type Rule

Fig. 1. Overloading rules for multiple inheritance and generics where circles denote applicable sets of generic method declarations and pentagons denote instantiated method declarations

2.2 Overloading Rules for Multiple Inheritance and Generics

Similarly for multiple inheritance, generics may cause ambiguous method calls that cannot be resolved at run time. Consider the following overloaded declarations of *sort*:

$$sort[\![P <: A]\!](x{:}\,\texttt{List}^{\texttt{I}}[\![P]\!]){:}\,\texttt{SortedList}^{\texttt{I}}[\![P]\!] = \cdots$$
$$sort[\![P <: B]\!](x{:}\,\texttt{List}^{\texttt{I}}[\![P]\!]){:}\,\texttt{SortedList}^{\texttt{I}}[\![P]\!] = \cdots$$

Their domain types are respectively "$\texttt{List}^{\texttt{I}}[\![P]\!]$ where P is a subtype of A" and "$\texttt{List}^{\texttt{I}}[\![P]\!]$ where P is a subtype of B"; for a method invocation $sort(l)$ where the run-time type of l is $\texttt{List}^{\texttt{I}}[\![C]\!]$, both declarations are applicable to l and neither declaration is more specific than the other.

In order to support generics as well for symmetric multiple dispatch, Allen et al. [2011] proposed new overloading rules for given generic declarations of types and methods. Even though they did not present a specific language with generics, we refer to the underlying language as *FGF*, Featherweight Generic Fortress. To support generics, the authors re-defined the terms *applicability* and *specificity* for generic method declarations and their overloading rules. A generic method declaration d is applicable to a type α if there exists an instance D of d that is applicable to α, that is, the domain type of D is a supertype of α. For example, consider the following generic method declaration:

$$m[\![P <: A]\!](x{:}\,P){:}\,P = \cdots$$

which has one type parameter P with an upper bound A. It is applicable to `Bottom`, C, and A, because they are subtypes of A. In general, a generic method declaration d_1 is more specific than d_2 if the applicable set of d_1 is a subset of that of d_2, where the *applicable set* of a method declaration is a set of types to which the declaration is applicable. Note that a monomorphic method declaration is a generic method declaration without any type parameter.

The overloading rules of FGF require every pair in a set of overloaded method declarations to satisfy *all* of the following rules as illustrated in Figure 1:

No Duplicates Rule: Distinct method declarations must have different applicable sets (there must be some argument type α for which one method is applicable and the other is not).

Meet Rule: Let $\mathcal{S}$ be the intersection of the applicable sets of the two method declarations; if $\mathcal{S}$ is not empty, then the overloaded set must contain a method declaration (not necessarily distinct from the two given method declarations) whose applicable set is precisely $\mathcal{S}$.

Return Type Rule: If one method declaration d_1 is more specific than d_2, for any non-`Bottom` type α in the applicable set of d_1, the return type of an instance of d_2 that is applicable to α must be a supertype of the return type of some instance of d_1 that is applicable to α.

Let us revisit the above ambiguous *sort* example with an additional method declaration:

$$\begin{array}{ll} d_1: & \mathit{sort}[\![P <: A]\!](x: \texttt{List}^{\texttt{I}}[\![P]\!]): \texttt{SortedList}^{\texttt{I}}[\![P]\!] = \cdots \\ d_2: & \mathit{sort}[\![P <: B]\!](x: \texttt{List}^{\texttt{I}}[\![P]\!]): \texttt{SortedList}^{\texttt{I}}[\![P]\!] = \cdots \\ d_3: & \mathit{sort}[\![P <: C]\!](x: \texttt{List}^{\texttt{I}}[\![P]\!]): \texttt{SortedList}^{\texttt{I}}[\![P]\!] = \cdots \end{array}$$

This set of overloaded method declarations is valid. For all pairs, the No Duplicates Rule is trivially satisfied. In the case of d_1 and d_2, the Meet Rule is satisfied (d_3 is their meet), and the Return Type Rule is vacuously satisfied since neither is more specific than the other. For d_1 and d_3, the Meet Rule is satisfied with d_3 itself as the meet; to see that the Return Type Rule is satisfied, note that $\texttt{List}^{\texttt{I}}$ is an invariant type constructor since FGF does not support variance. For any $\texttt{List}^{\texttt{I}}[\![X]\!]$ in the applicable set of d_3, there is exactly one instance of d_1 (with X substituted for P) applicable to it:

$$\mathit{sort}(x: \texttt{List}^{\texttt{I}}[\![X]\!]): \texttt{SortedList}^{\texttt{I}}[\![X]\!] = \cdots.$$

Since the return type of this instance equals the return type of the corresponding instance of d_3:

$$\mathit{sort}(x: \texttt{List}^{\texttt{I}}[\![X]\!]): \texttt{SortedList}^{\texttt{I}}[\![X]\!] = \cdots,$$

d_1 and d_3 satisfy the Return Type Rule. Similarly, d_2 and d_3 also satisfy all of the overloading rules.

Again, the overloading rules ensure that there is no ambiguous method calls at run time, and that types are preserved. In FGF, No Duplicates Rule rules out the trivial ambiguous calls with duplicate declarations. Meet Rule plays an important role for ensuring the existence of a disambiguating method declaration, and Return Type Rule serves a key role for type preservation.

2.3 Overloading Rules for Multiple Inheritance and Generics with Variance

Variance, as well as symmetric multiple dispatch, multiple inheritance, and generics, may cause ambiguous method calls that cannot be resolved at run time. We revisit the ambiguous *sort* example in Section 2.2. This time, we define the same method in a simpler way using covariant definitions of $\texttt{List}^{\texttt{C}}$ and $\texttt{SortedList}^{\texttt{C}}$. Consider the following overloaded method declarations of *sort*:

$$\begin{array}{ll} d_1: & \mathit{sort}(x: \texttt{List}^{\texttt{C}}[\![A]\!]): \texttt{SortedList}^{\texttt{C}}[\![A]\!] = \cdots \\ d_2: & \mathit{sort}(x: \texttt{List}^{\texttt{C}}[\![B]\!]): \texttt{SortedList}^{\texttt{C}}[\![B]\!] = \cdots \end{array}$$

Similarly to the example in Section 2.2, for a method invocation $\mathit{sort}(l)$ where the run-time type of l is $\texttt{List}^{\texttt{C}}[\![C]\!]$, it is ambiguous which declaration to invoke: both d_1 and d_2 are applicable to l and neither is more specific than the other. In order to ensure that there is no ambiguous method calls for *sort* at run time, we need to add a disambiguating method declaration d_3:

$$d_3: \quad \mathit{sort}(x: \texttt{List}^{\texttt{C}}[\![C]\!]): \texttt{SortedList}^{\texttt{C}}[\![C]\!] = \cdots$$

with a more specific return type than those of d_1 and d_2 for type preservation.

Variance also adds complications in type-sound dynamic dispatch. Let us now assume that $\texttt{List}^{\texttt{C}}$ is covariant in its type argument but $\texttt{SortedList}^{\texttt{I}}$ is invariant. Then, the set of three method declarations above replacing $\texttt{SortedList}^{\texttt{C}}$ with $\texttt{SortedList}^{\texttt{I}}$ would be invalid, because the return type $\texttt{SortedList}^{\texttt{I}}[\![A]\!]$ of d_1 would not be a supertype of the return type $\texttt{SortedList}^{\texttt{I}}[\![C]\!]$ of d_3.

Supporting variance in a type-sound manner requires keeping track of the static types of method invocations at run time. Let us again assume that $\texttt{List}^{\texttt{C}}$ is covariant but $\texttt{SortedList}^{\texttt{I}}$ is invariant, and consider the following overloaded set (note that d_3 now has a type parameter P):

$$\begin{array}{ll} d_1: & \mathit{sort}(x: \texttt{List}^{\texttt{C}}[\![A]\!]): \texttt{SortedList}^{\texttt{I}}[\![A]\!] = \cdots \\ d_2: & \mathit{sort}(x: \texttt{List}^{\texttt{C}}[\![B]\!]): \texttt{SortedList}^{\texttt{I}}[\![B]\!] = \cdots \\ d_3: & \mathit{sort}[\![P]\!](x: \texttt{List}^{\texttt{C}}[\![C]\!]): \texttt{SortedList}^{\texttt{I}}[\![P]\!] = \cdots \end{array}$$

as well as the following generic method declaration of *merge*:

$$\mathit{merge}[\![P]\!](x: \texttt{SortedList}^{\texttt{I}}[\![P]\!], y: \texttt{SortedList}^{\texttt{I}}[\![P]\!]): \texttt{SortedList}^{\texttt{I}}[\![P]\!] = \cdots$$

For a method invocation $merge(sort(l_1), sort(l_2))$ where the static types of l_1 and l_2 are both $\texttt{List}^{\texttt{C}}[\![A]\!]$ and their run-time types are $\texttt{List}^{\texttt{C}}[\![A]\!]$ and $\texttt{List}^{\texttt{C}}[\![C]\!]$, respectively, at compile time, both method invocations $sort(l_1)$ and $sort(l_2)$ choose d_1 and their static types are both $\texttt{SortedList}^{\texttt{I}}[\![A]\!]$. Thus, the above *merge* call statically invokes the following instance of the generic *merge* declaration:

$$merge(x\colon \texttt{SortedList}^{\texttt{I}}[\![A]\!], y\colon \texttt{SortedList}^{\texttt{I}}[\![A]\!])\colon \texttt{SortedList}^{\texttt{I}}[\![A]\!]$$

that is applicable to $(\texttt{SortedList}^{\texttt{I}}[\![A]\!], \texttt{SortedList}^{\texttt{I}}[\![A]\!])$. However, at run time, while $sort(l_1)$ chooses d_1 and its return type is $\texttt{SortedList}^{\texttt{I}}[\![A]\!]$, $sort(l_2)$ chooses d_3 and its return type depends on what actual type is inferred at run time for the type parameter P. If the dynamic type inference algorithm instantiates P as A, $sort(l_2)$ has type $\texttt{SortedList}^{\texttt{I}}[\![A]\!]$, which is type sound. But it could equally plausibly instantiate P as some other type T, and then no dynamic instance of the generic *merge* declaration would be applicable to $(\texttt{SortedList}^{\texttt{I}}[\![A]\!], \texttt{SortedList}^{\texttt{I}}[\![T]\!])$. Hence, the *merge* call would fail, despite the fact that the program successfully passed static type checking. In order to help run-time type inference algorithms make type-sound choices of type arguments, the language semantics needs to make the static types of method invocations available at run time.

In this paper, we present FGFV, which sheds light on the long-standing endeavor to support strong types, symmetric multiple dispatch, multiple inheritance, and parametric polymorphism with variance. We formalize the static overloading rules for FGFV using existential types and universal types quantified over method type parameters, and design a new dynamic dispatch algorithm that is proven type-sound to guarantee unambiguous method calls and type preservation at run time.

3 LANGUAGE

In this section, we define the abstract syntax of FGFV and its well-formedness requirements.

3.1 Notation

Following, for example, Vytiniotis et al. [2013] and Coughlin and Chang [2014], we use an overline above mathematical material to indicate an appropriately punctuated sequence of zero or more repetitions of the material, with a subscript added to each metavariable in the material. Thus $\{\overline{\alpha}\}$ is equal to $\{\alpha_1, \alpha_2, \ldots, \alpha_n\}$. Sometimes we write $\bullet$ as an explicit indication of an empty sequence. Overlines may be nested, in which case they are expanded outermost first, so "$[\![\overline{\{\overline{\zeta}\} <: Q <: \{\overline{\xi}\}}]\!]$" expands first to "$[\![\{\overline{\zeta_1}\} <: Q_1 <: \{\overline{\xi_1}\}, \{\overline{\zeta_2}\} <: Q_2 <: \{\overline{\xi_2}\}, \ldots, \{\overline{\zeta_n}\} <: Q_n <: \{\overline{\xi_n}\}]\!]$" and then to

$$\begin{array}{l}[\![\ \{\zeta_{1\,1}, \zeta_{1\,2}, \ldots, \zeta_{1\,k_1}\} <: Q_1 <: \{\xi_{1\,1}, \xi_{1\,2}, \ldots, \xi_{1\,m_1}\},\\ \quad \{\zeta_{2\,1}, \zeta_{2\,2}, \ldots, \zeta_{2\,k_2}\} <: Q_2 <: \{\xi_{2\,1}, \xi_{2\,2}, \ldots, \xi_{2\,m_2}\},\\ \quad \ldots,\\ \quad \{\zeta_{n1}, \zeta_{n2}, \ldots, \zeta_{nk_n}\} <: Q_n <: \{\xi_{n1}, \xi_{n2}, \ldots, \xi_{nm_n}\}\ ]\!]\end{array}$$

(conceptually all on one line, broken here for reasons of space). The copies of each inner overline expand independently, so the sequences into which they expand may have different lengths; thus "$[\![\overline{\{\overline{\zeta}\} <: Q <: \{\overline{\xi}\}}]\!]$" could expand into, e.g., "$[\![\{\zeta_{1\,1}, \zeta_{1\,2}\} <: Q_1 <: \{\xi_{1\,1}\}, \{\} <: Q_2 <: \{\xi_{2\,1}, \xi_{2\,2}, \xi_{2\,3}\}]\!]$".

Following a suggestion of Steele [2017, 0:59:37], we use an underline below submaterial within overlined material to prevent an overline from attaching subscripts to the metavariables in the submaterial; the effect is to require the underlined material to be the same in each copy. Thus "$\overline{\underline{\Delta} \vdash \psi\ \text{ok}}$" represents the sequence "$\Delta \vdash \psi_1\ \text{ok} \quad \Delta \vdash \psi_2\ \text{ok} \quad \cdots \quad \Delta \vdash \psi_n\ \text{ok}$" for any $n \geq 0$.

Program	Π	::=	$\overline{\psi}, e$
Class declaration	ψ	::=	$\texttt{trait}\ T[\![\overline{V\ \beta}]\!] <: \{\overline{t}\}\ \overline{\mu}\ \texttt{end} \mid \texttt{object}\ O[\![\overline{\beta}]\!](\overline{x{:}\tau}) <: \{\overline{t}\}\ \overline{\mu}\ \texttt{end}$
Method definition	μ	::=	$m[\![\overline{\kappa}]\!](\overline{x{:}\tau}){:}\ \tau = e$
Class type parameter binding	β	::=	$P <: \{\overline{\tau}\}$
Method type parameter binding	κ	::=	$\{\overline{\tau}\} <: P <: \{\overline{\tau}\}$
Variance mark	V	::=	$+ \mid - \mid =$
Expression	e	::=	$z \mid ((\overline{x{:}\tau}){:}\ \tau \Rightarrow e) \mid e@(\overline{e}) \mid O[\![\overline{\tau}]\!](\overline{e}) \mid e.m(\overline{e})$
Bindable variable	z	::=	$x \mid \texttt{self}$
Type	τ	::=	$P \mid c \mid (\overline{\tau}) \mid (\tau \to \tau) \mid \texttt{Any}$
Constructed type	c	::=	$t \mid O[\![\overline{\tau}]\!]$
Trait type	t	::=	$T[\![\overline{\tau}]\!]$

Fig. 2. Abstract syntax of FGFV

Similarly $\overline{\overline{\Delta' \vdash \xi\ \text{ok}}}$ expands to "$\overline{\Delta' \vdash \xi_1\ \text{ok}} \quad \overline{\Delta' \vdash \xi_2\ \text{ok}} \quad \cdots \quad \overline{\Delta' \vdash \xi_n\ \text{ok}}$" and then to

$$\begin{array}{llll} \Delta' \vdash \xi_{1\,1}\ \text{ok} & \Delta' \vdash \xi_{1\,2}\ \text{ok} & \cdots & \Delta' \vdash \xi_{1\,m_1}\ \text{ok} \\ \Delta' \vdash \xi_{2\,1}\ \text{ok} & \Delta' \vdash \xi_{2\,2}\ \text{ok} & \cdots & \Delta' \vdash \xi_{2\,m_2}\ \text{ok} \\ \cdots & & & \\ \Delta' \vdash \xi_{n1}\ \text{ok} & \Delta' \vdash \xi_{n2}\ \text{ok} & \cdots & \Delta' \vdash \xi_{n m_n}\ \text{ok} \end{array}$$

The overline notation may explicitly bind index variables by using a range expression. In such cases, the index variables may appear explicitly in the covered material, and subscripts are not automatically attached to metavariables. For example, $\left(\overline{m(e_i)}^{\,3 \le i \le 6}\right)$ means $(m(e_3), m(e_4), m(e_5), m(e_6))$.

Metavariables are expanded *after* overlines. Multiple occurrences of a metavariable with the same subscript(s) must have the same expansion, but occurrences of a metavariable with different subscript(s) may be expanded independently. Thus "$\overline{[\![\{\overline{\zeta}\} <: Q <: \{\overline{\xi}\}]\!]}$" might ultimately expand into, e.g., "$[\![\{\texttt{A},\texttt{B}\} <: \texttt{U} <: \{\texttt{List}[\![\texttt{Boolean}]\!]\}, \{\} <: \texttt{V} <: \{\texttt{Any},\texttt{C},\texttt{Number}\}]\!]$".

We use horizontal lines as overlines and underlines, but also in the usual Gentzen notation for inference rules, with premises above the line and the conclusion below. To help reduce confusion: (1) we use a slightly heavier and wider line for inference rules; (2) the line for inference rules has more space above and below it; (3) we do not use the inference rule notation for axioms (rules that have no premises); and (4) we do not nest the inference rule notation to present derivation trees.

3.2 Syntax

Figure 2 shows the syntax of the FGFV calculus. Metavariables T and U range over trait names; O ranges over object names; x ranges over variable and field names; z is either x or the keyword `self`; m ranges over method names; P, Q, S range over type parameter names; and τ, ω, ζ, ξ range over types.

A program consists of a sequence of class declarations and a top-level expression. A class is either a trait or an object; traits are like Java interfaces that do not have any fields, and objects are like Java final classes that cannot be extended. A trait may have type parameters with variance annotations and upper bounds, extend any number of other traits, and have method definitions. An object is similar to a trait except that (1) its type parameters do not have variance annotations because they are always invariant, and (2) it may have fields, which are initialized when an object instance is created. When a class does not extend any trait, we omit the empty extends clause $<: \{\}$. When a class has no type parameters, we omit the empty brackets; we write T for $T[\![\,]\!]$. A method may have type parameters as well, but they have both lower and upper bounds. We omit clauses when no bounds are specified; for example, we write $T[\![+\ P]\!]$ for $T[\![+\ P <: \{\}]\!]$ and $m[\![Q, \{Q\} <: S]\!]$

for $m[\![\{\} <: Q <: \{\}, \{Q\} <: S <: \{\}]\!]$. By default, each type parameter has an upper bound `Any` and a lower bound `Bottom`, which we describe in the next subsection.

The bounds of a type parameter place restrictions on valid instantiations of the type parameter; a valid type instantiation must be a subtype of all its upper bounds and, at the same time, a supertype of all its lower bounds, if any. We write + for covariance, − for contravariance, and = for invariance. As discussed in Section 2.3, variance annotations of type parameters define subtype relations between different instances of a generic trait. For example, consider the following trait declarations:

$$\texttt{trait}\ U_1\ __\ \texttt{end}$$
$$\texttt{trait}\ U_2 <: \{U_1\}\ __\ \texttt{end}$$
$$\texttt{trait}\ T[\![+\,P, -\,Q, =\,S]\!]\ __\ \texttt{end}$$

where "__" denotes omitted material. Because U_2 is a subtype of U_1, the variance annotations of the type parameters of T define subtype relations such as $T[\![U_2, U_1, U_1]\!] <: T[\![U_1, U_2, U_1]\!]$.

Expressions and types are conventional. One can refer to a field or bound variable simply by mentioning it; the keyword `self` refers to the target object for the currently executing method. Functions are first-class values; $((\overline{x{:}\,\tau}){:}\,\tau \Rightarrow e)$ is a (typed) function constructor, and we write $e@(\overline{e})$ for function calls. An object creation $O[\![\overline{\tau}]\!](\overline{e})$ makes an instance of O by instantiating its type parameters with $\overline{\tau}$ and initializing its fields with the values of $\overline{e}$. While tuples are not first-class values in FGFV (they would be trivial to add, but are not needed for our purposes), we use tuple types $(\overline{\tau})$ to denote the domain types of functions and methods, which may have multiple parameters. The type `Any` is a supertype of every type (and `Bottom`, used below, is a subtype of every type but is intentionally not expressible in the surface syntax).

3.3 Well-formedness

We define well-formedness of a program and its declarations as shown in Figure 3. A type-declaration environment Δ is a set of class declarations and type parameter bindings, and a variable-type environment Γ is a sequence of bindable variables and their types. Checking well-formedness may introduce *internal types*:

$$\text{Internal type}\quad \alpha ::= P \mid C \mid (\overline{\alpha}) \mid (\alpha \to \alpha) \mid \texttt{Any} \mid \texttt{Bottom} \mid (\alpha \sqcup \alpha) \mid (\alpha \sqcap \alpha)$$

which contain `Bottom`, union types, and intersection types. Type `Bottom` is a subtype of every type; hence, `Any` and `Bottom` are the top and bottom, respectively, of the type lattice. Metavariables $\alpha, \gamma, \rho, \chi, \eta$ range over internal types. We introduce metavariables K, C, J, and M, analogous to κ, c, t, and μ, respectively, where types τ in the latter are replaced by internal types α in the former.

Rule [T-Program] states that a program is well-formed if its class declarations have distinct names and are well-formed, and its top-level expression is well-formed. The type of a program is the type of its top-level expression; the metavariable g and h represents a *ground type*, which is a type that contains no type parameters. We describe well-formedness and type checking of expressions in Section 5.1.

Rules [D-Trait] and [D-Object] check well-formedness of class declarations. For the declaration of a class C, they verify that the type parameter names of the declaration are distinct, that the types used in the declaration are well-formed, and that the declared methods are well-formed. In addition, as we explain later in this subsection, they make sure that (1) the *proper ancestors* of C, which are all the traits C extends, by explicitly extending and transitively inheriting as well, are well-formed, and (2) the *visible methods* of C, which are the methods either declared by C or inherited from the traits C extends, obey the overloading rules. The only differences are that rule [D-Trait] additionally guarantees that a trait does not extend itself, and rule [D-Object] guarantees that the names of the fields are distinct.

Well-formed programs: $\boxed{\vdash \Pi : g}$

$$\frac{\Pi = \overline{\psi}, e \qquad \mathit{distinct}(\overline{\mathit{name}(\psi)}) \qquad \Delta = \{\overline{\psi}\} \qquad \overline{\underline{\Delta} \vdash \psi\ \mathsf{ok}} \qquad \Delta; \bullet \vdash e : (_, g)}{\vdash \Pi : g}\ [\textsc{T-Program}]$$

Well-formed trait and object declarations: $\boxed{\Delta \vdash \psi\ \mathsf{ok}}$

$$\frac{\begin{array}{c}\Delta' = \Delta \cup \left\{\overline{\{\} <: P <: \{\overline{\xi}\}}\right\} \qquad \mathit{distinct}(\overline{P}) \qquad \overline{\underline{\underline{\Delta'} \vdash \xi\ \mathsf{ok}}} \qquad \overline{\underline{\Delta'} \vdash t\ \mathsf{ok}} \\ \{\overline{J}\} = \mathit{properAncestors}(\Delta', T[\![\overline{P}]\!]) \qquad \overline{\Delta'; \overline{J} \vdash J_i \text{ and } J_j \text{ ancestors ok}}^{1 \le i < j \le \#(\overline{J})} \\ \overline{\underline{T} \neq \mathit{name}(J)} \qquad \overline{\underline{\Delta'; \mathsf{self}: T[\![\overline{P}]\!]; \overline{V\ P}} \vdash \mu\ \mathsf{ok}} \qquad \{\overline{d}\} = \mathit{allVisible}(\Delta', T[\![\overline{P}]\!]) \\ \overline{\Delta' \vdash d_i \text{ not duplicate of } d_j}^{1 \le i < j \le \#(\overline{d})} \qquad \overline{\Delta' \vdash d_i \text{ meet } d_j \text{ wrt } \{\overline{d}\}\ \mathsf{ok}}^{1 \le i < j \le \#(\overline{d})} \\ \overline{\Delta' \vdash d_i \text{ return type wrt } d_j\ \mathsf{ok}}^{1 \le i < j \le \#(\overline{d})} \qquad \overline{\Delta' \vdash d_j \text{ return type wrt } d_i\ \mathsf{ok}}^{1 \le i < j \le \#(\overline{d})}\end{array}}{\Delta \vdash \mathtt{trait}\ T[\![\overline{V\ P <: \{\overline{\xi}\}}]\!] <: \left\{\overline{t}\right\}\ \overline{\mu}\ \mathsf{end\ ok}}\ [\textsc{D-Trait}]$$

$$\frac{\begin{array}{c}\Delta' = \Delta \cup \left\{\overline{\{\} <: P <: \{\overline{\xi}\}}\right\} \qquad \mathit{distinct}(\overline{P}) \qquad \overline{\underline{\underline{\Delta'} \vdash \xi\ \mathsf{ok}}} \qquad \overline{\underline{\Delta'} \vdash t\ \mathsf{ok}} \\ \{\overline{J}\} = \mathit{properAncestors}(\Delta', O[\![\overline{P}]\!]) \qquad \overline{\Delta'; \overline{J} \vdash J_i \text{ and } J_j \text{ ancestors ok}}^{1 \le i < j \le \#(\overline{J})} \\ \overline{\underline{\Delta'; \mathsf{self}: O[\![\overline{P}]\!], \overline{x{:}\tau}; \overline{= P}} \vdash \mu\ \mathsf{ok}} \qquad \{\overline{d}\} = \mathit{allVisible}(\Delta', O[\![\overline{P}]\!]) \\ \overline{\Delta' \vdash d_i \text{ not duplicate of } d_j}^{1 \le i < j \le \#(\overline{d})} \qquad \overline{\Delta' \vdash d_i \text{ meet } d_j \text{ wrt } \{\overline{d}\}\ \mathsf{ok}}^{1 \le i < j \le \#(\overline{d})} \\ \overline{\Delta' \vdash d_i \text{ return type wrt } d_j\ \mathsf{ok}}^{1 \le i < j \le \#(\overline{d})} \qquad \overline{\Delta' \vdash d_j \text{ return type wrt } d_i\ \mathsf{ok}}^{1 \le i < j \le \#(\overline{d})} \\ \overline{\underline{\Delta'} \vdash \tau\ \mathsf{ok}} \qquad \mathit{distinct}(\overline{x})\end{array}}{\Delta \vdash \mathtt{object}\ O[\![\overline{P <: \{\overline{\xi}\}}]\!](\overline{x{:}\tau}) <: \left\{\overline{t}\right\}\ \overline{\mu}\ \mathsf{end\ ok}}\ [\textsc{D-Object}]$$

Well-formed method declarations: $\boxed{\Delta; \Gamma; \overline{V\ P} \vdash \mu\ \mathsf{ok}}$

$$\frac{\begin{array}{c}\Delta \vdash [\![\overline{\{\overline{\zeta}\} <: Q <: \{\overline{\xi}\}}]\!]\ \mathsf{ok} \qquad \Delta' = \Delta \cup \left\{\overline{\{\overline{\zeta}\} <: Q <: \{\overline{\xi}\}}\right\} \qquad \overline{\underline{\underline{\Delta'} \vdash \zeta\ \mathsf{ok}}} \qquad \overline{\underline{\underline{\Delta'} \vdash \xi\ \mathsf{ok}}} \\ \mathit{distinct}(\overline{x}) \qquad \overline{\underline{\Delta'} \vdash \tau\ \mathsf{ok}} \qquad \Delta' \vdash \omega\ \mathsf{ok} \qquad \Delta'; \Gamma, \overline{x{:}\tau} \vdash e : (_, \rho) \qquad \Delta' \vdash \rho <: \omega \\ \mathit{distinct}(\overline{P}, \overline{Q}) \qquad \overline{(V = =) \vee (\forall \mathcal{T}.\ (((\underline{\overline{\tau}) \to \omega}) \neq \mathcal{T}\ \boxed{P}) \vee (\underline{\Delta} \vdash \mathcal{T}\ \mathsf{variance}\ V))}\end{array}}{\Delta; \Gamma; \overline{V\ P} \vdash m[\![\overline{\{\overline{\zeta}\} <: Q <: \{\overline{\xi}\}}]\!](\overline{x{:}\tau}){:}\,\omega = e\ \mathsf{ok}}\ [\textsc{D-Method}]$$

Fig. 3. Well-formed programs, class declarations, and method declarations

Well-formedness of types is conventional as shown in Figure 4. The types `Any` and `Bottom` are trivially well-formed. A type parameter P is well-formed under a type-declaration environment Δ, if Δ contains a binding of P. A constructed type $T[\![\overline{\alpha}]\!]$ or $O[\![\overline{\alpha}]\!]$ is well-formed, if the trait T or the object O is declared in the given type-declaration environment and all the type arguments $\overline{\alpha}$ are well-formed and satisfy the declared bounds of their corresponding type parameters. Finally, the remaining structural types such as tuple types, arrow types, union types, and intersection types are well-formed, if all their component types are.

Well-formed types: $\boxed{\Delta \vdash \alpha \text{ ok}}$

$$\frac{}{\Delta \vdash \texttt{Any ok}} \text{ [W-Any]} \qquad \frac{}{\Delta \vdash \texttt{Bottom ok}} \text{ [W-Bottom]}$$

$$\frac{_ <: P <: _ \in \Delta}{\Delta \vdash P \text{ ok}} \text{ [W-Param]} \qquad \frac{\overline{\underline{\Delta} \vdash \alpha \text{ ok}}}{\Delta \vdash (\overline{\alpha}) \text{ ok}} \text{ [W-Tuple]}$$

$$\frac{\Delta \vdash \alpha \text{ ok} \qquad \Delta \vdash \rho \text{ ok}}{\Delta \vdash (\alpha \to \rho) \text{ ok}} \text{ [W-Arrow]} \qquad \frac{\Delta \vdash \alpha \text{ ok} \qquad \Delta \vdash \gamma \text{ ok}}{\Delta \vdash (\alpha \sqcup \gamma) \text{ ok}} \text{ [W-Union]}$$

$$\frac{\Delta \vdash \alpha \text{ ok} \qquad \Delta \vdash \gamma \text{ ok}}{\Delta \vdash (\alpha \sqcap \gamma) \text{ ok}} \text{ [W-Intersection]}$$

$$\frac{\texttt{trait } T[\![\overline{V\ P <: \{\overline{\xi}\}}]\!] __ \texttt{ end} \in \Delta \qquad \overline{\underline{\Delta} \vdash \alpha \text{ ok}} \qquad \overline{\underline{\Delta} \vdash \alpha <: \underline{[\overline{\alpha/P}]} \bigsqcap \{\overline{\xi}\}}}{\Delta \vdash T[\![\overline{\alpha}]\!] \text{ ok}} \text{ [W-Trait]}$$

$$\frac{\texttt{object } O[\![\overline{P <: \{\overline{\xi}\}}]\!] __ \texttt{ end} \in \Delta \qquad \overline{\underline{\Delta} \vdash \alpha \text{ ok}} \qquad \overline{\underline{\Delta} \vdash \alpha <: \underline{[\overline{\alpha/P}]} \bigsqcap \{\overline{\xi}\}}}{\Delta \vdash O[\![\overline{\alpha}]\!] \text{ ok}} \text{ [W-Object]}$$

Fig. 4. Well-formed types

Well-formed ancestors: $\boxed{\Delta; \overline{J} \vdash J \text{ and } J \text{ ancestors ok}}$

$$\frac{T \neq T'}{\Delta; \overline{J} \vdash T[\![\overline{\alpha}]\!] \text{ and } T'[\![\overline{\eta}]\!] \text{ ancestors ok}} \text{ [Anc-Diff-Trait]}$$

$$\frac{T[\![\overline{\gamma}]\!] \in \{\overline{J}\} \qquad \Delta \vdash T[\![\overline{\gamma}]\!] <: T[\![\overline{\alpha}]\!] \qquad \Delta \vdash T[\![\overline{\gamma}]\!] <: T[\![\overline{\eta}]\!]}{\Delta; \overline{J} \vdash T[\![\overline{\alpha}]\!] \text{ and } T[\![\overline{\eta}]\!] \text{ ancestors ok}} \text{ [Anc-Same-Trait]}$$

Well-formed type parameter bindings: $\boxed{\Delta \vdash [\![\overline{K}]\!] \text{ ok}}$

$$\frac{}{\Delta \vdash [\![\,]\!] \text{ ok}} \text{ [Binding-Empty]}$$

$$\frac{FV(\overline{\chi'}) \subseteq \mathit{parameters}(\Delta) \cup \{\overline{P}\} \qquad FV(\overline{\eta'}) \subseteq \mathit{parameters}(\Delta) \qquad \Delta \vdash \bigsqcup \{\overline{\chi'}\} <: \bigsqcap \{\overline{\eta'}\} \qquad \Delta \vdash [\![\overline{\{\overline{\chi}\} <: P <: \{\overline{\eta}\}}]\!] \text{ ok}}{\Delta \vdash [\![\overline{\{\overline{\chi}\} <: P <: \{\overline{\eta}\}}, \{\overline{\chi'}\} <: P' <: \{\overline{\eta'}\}]\!] \text{ ok}} \text{ [Binding-Step]}$$

Fig. 5. Well-formed ancestors and type parameter bindings

The proper ancestors $\{\overline{J}\}$ of a class C are well-formed, if every pair in $\{\overline{J}\}$ satisfies one of the well-formed ancestors rules shown in Figure 5: $\overline{\Delta'; \overline{J} \vdash J_i \text{ and } J_j \text{ ancestors ok}}^{1 \le i < j \le \#(\overline{J})}$. The well-formed ancestors rules ensure that if a class C extends two instances of a trait T, $T[\![\overline{\alpha}]\!]$ and $T[\![\overline{\rho}]\!]$, then C also extends some $T[\![\overline{\gamma}]\!]$ (which may be $T[\![\overline{\alpha}]\!]$, $T[\![\overline{\rho}]\!]$, or a third instance) that extends both $T[\![\overline{\alpha}]\!]$ and $T[\![\overline{\rho}]\!]$. Therefore, because the rules guarantee that there always exists a single minimal ancestor when a class extends multiple instances of a trait, no ambiguous method calls are possible due to multiple instances of a trait as discussed in Section 5.2.

The visible methods $\{\overline{d}\} = \{\overline{(C, M)}\}$ of a class obey the overloading rules if every pair in $\{\overline{d}\}$ satisfies the overloading rules described in Section 4.1. The set $\{\overline{(C, M)}\}$ contains pairs of visible method declarations $\overline{M}$ and their defining classes $\overline{C}$.

Variance of type context: $\boxed{\Delta \vdash \mathcal{T} \text{ variance } V}$

$$\frac{}{\Delta \vdash \square \text{ variance } +} \text{ [Tc-Empty]}$$

$$\frac{\texttt{trait}\ T[\![\overline{V\ _ <: _}]\!]\ __\ \text{end} \in \Delta \quad i = \#(\overline{\tau}) + 1 \quad \#(\overline{V}) = \#(\overline{\tau}) + \#(\overline{\omega}) + 1 \quad \Delta \vdash \mathcal{T} \text{ variance } V' \quad V = V_i \times V'}{\Delta \vdash T[\![\overline{\tau}, \mathcal{T}, \overline{\omega}]\!] \text{ variance } V} \text{ [Tc-Trait]}$$

$$\frac{}{\Delta \vdash O[\![\overline{\tau}, \mathcal{T}, \overline{\omega}]\!] \text{ variance } =} \text{ [Tc-Obj]}$$

$$\frac{\Delta \vdash \mathcal{T} \text{ variance } V}{\Delta \vdash (\overline{\tau}, \mathcal{T}, \overline{\omega}) \text{ variance } V} \text{ [Tc-Tuple]}$$

$$\frac{\Delta \vdash \mathcal{T} \text{ variance } V' \quad V = \mathit{flip}(V')}{\Delta \vdash (\mathcal{T} \to \tau) \text{ variance } V} \text{ [Tc-Arrow-Param]}$$

$$\frac{\Delta \vdash \mathcal{T} \text{ variance } V}{\Delta \vdash (\tau \to \mathcal{T}) \text{ variance } V} \text{ [Tc-Arrow-Return]}$$

Flip of variance: $\boxed{\mathit{flip}(V) = V}$

$$\mathit{flip}(V) = \begin{cases} - & \text{if } V = + \\ + & \text{if } V = - \\ = & \text{if } V = = \end{cases}$$

Composition of variance: $\boxed{V \times V = V}$

$$V_1 \times V_2 = \begin{cases} = & \text{if } V_1 = = \vee V_2 = = \\ + & \text{if } V_1 \neq = \wedge V_2 \neq = \wedge V_1 = V_2 \\ - & \text{if } V_1 \neq = \wedge V_2 \neq = \wedge V_1 \neq V_2 \end{cases}$$

Fig. 6. Variance of type context

Rule [D-Method] checks well-formedness of method declarations. For the method declaration μ, it verifies that the parameter names of the declaration are distinct; that the class type parameters and the method type parameters of the declaration are collectively distinct; that types used in the declaration are well-formed; that the body expression is well-formed; and that the type of the body expression is a subtype of the declared return type. In addition, as we discuss below, it checks that method type parameter bindings are well-formed and that any references to trait type parameters are correctly used according to their variance annotations.

A sequence of method type parameter bindings is well-formed, if (1) every reference to a method type parameter falls within its scope (which extends to the right of its declaration), (2) no reference to a method type parameter appears in any of the upper bounds, and (3) for each type parameter, the union of its lower bounds is a subtype of the intersection of its upper bounds. As we discuss in Section 5.4, restriction (1) makes dynamic dispatch more efficient and restriction (2) makes it consistent. Figure 5 shows the rules for well-formed type parameter bindings.

Trait type parameters are correctly used according to their variance annotations, if covariant and contravariant type parameters appear only at covariant and contravariant positions, respectively. In order to express this restriction, we introduce a *type context* $\mathcal{T}$, which defines positions where a type can appear within another type. Similarly to the conventional evaluation contexts [Wright and Felleisen 1994], a type context $\mathcal{T}$ is a type with one of its components replaced by a hole, denoted by "$\square$". The type $\mathcal{T}\boxed{\tau}$ results from placing a type τ in the hole of $\mathcal{T}$:

$$\text{Type context} \quad \mathcal{T} ::= \square \mid T[\![\overline{\tau}, \mathcal{T}, \overline{\tau}]\!] \mid O[\![\overline{\tau}, \mathcal{T}, \overline{\tau}]\!] \mid (\overline{\tau}, \mathcal{T}, \overline{\tau}) \mid (\mathcal{T} \to \tau) \mid (\tau \to \mathcal{T})$$

Thus, if $\tau = ((P, T[\![Q]\!]) \to \mathsf{Any})$, then $\tau = \mathcal{T}\boxed{P} = \mathcal{T}'\boxed{Q}$ where $\mathcal{T} = ((\square, T[\![Q]\!]) \to \mathsf{Any})$ and $\mathcal{T}' = ((P, T[\![\square]\!]) \to \mathsf{Any})$.

Using type contexts, the last premise of rule [D-Method] ensures that the class type parameters of a method appear in correct positions. Considering the arrow type representing the method's type $\tau' = ((\overline{\tau}) \to \omega)$, an invariant type parameter can appear anywhere in τ'. However, when a covariant or contravariant type parameter P appears in $\tau' = \mathcal{T}\boxed{P}$, the *variance of the type context* of P should match the variance V of P: $\Delta \vdash \mathcal{T} \text{ variance } V$. As shown in Figure 6, the variance of a type context is defined inductively: (1) the hole is covariant; (2) components of tuple types, return types of arrow types, and covariant type parameters of traits preserve the variance of their

No Duplicates Rule: $\boxed{\Delta \vdash d \text{ not duplicate of } d}$

$$\frac{name(d) \neq name(d')}{\Delta \vdash d \text{ not duplicate of } d'} \text{ [No-Dup-Triv]}$$

$$\frac{\neg(\Delta \vdash dom(d) \sqsubseteq dom(d'))}{\Delta \vdash d \text{ not duplicate of } d'} \text{ [No-Dup-Not-Less]}$$

$$\frac{\neg(\Delta \vdash dom(d') \sqsubseteq dom(d))}{\Delta \vdash d \text{ not duplicate of } d'} \text{ [No-Dup-Not-Gtr]}$$

Meet Rule: $\boxed{\Delta \vdash d \text{ meet } d \text{ wrt } \{\overline{d}\} \text{ ok}}$

$$\frac{name(d) \neq name(d')}{\Delta \vdash d \text{ meet } d' \text{ wrt } _ \text{ ok}} \text{ [Meet-Triv]}$$

$$\frac{\Delta \vdash (dom(d) \sqcap dom(d')) \sqsubseteq \exists[\![\,]\!]\texttt{Bottom}}{\Delta \vdash d \text{ meet } d' \text{ wrt } _ \text{ ok}} \text{ [Meet-Excl]}$$

$$\frac{\Delta \vdash dom(d) \sqsubseteq dom(d')}{\Delta \vdash d \text{ meet } d' \text{ wrt } _ \text{ ok}} \text{ [Meet-Less]}$$

$$\frac{\Delta \vdash dom(d') \sqsubseteq dom(d)}{\Delta \vdash d \text{ meet } d' \text{ wrt } _ \text{ ok}} \text{ [Meet-Gtr]}$$

$$\frac{d''' \in \{\overline{d''}\} \quad name(d) = name(d') = name(d''') \quad \Delta \vdash dom(d''') \equiv (dom(d) \sqcap dom(d'))}{\Delta \vdash d \text{ meet } d' \text{ wrt } \{\overline{d''}\} \text{ ok}} \text{ [Meet-Third]}$$

Return Type Rule: $\boxed{\Delta \vdash d \text{ return type wrt } d \text{ ok}}$

$$\frac{name(d) \neq name(d')}{\Delta \vdash d \text{ return type wrt } d' \text{ ok}} \text{ [Return-Triv]}$$

$$\frac{\neg(\Delta \vdash dom(d) \sqsubseteq dom(d'))}{\Delta \vdash d \text{ return type wrt } d' \text{ ok}} \text{ [Return-Not-Less]}$$

$$\frac{\begin{array}{c} arrow(d) = \forall[\![\overline{\kappa}]\!](\alpha \to \rho) \qquad \overline{\kappa = \{\overline{\chi}\} <: P <: \{\overline{\eta}\}} \qquad arrow(d') = \forall[\![\overline{\kappa'}]\!](\alpha' \to \rho') \\ \overline{\kappa' = \{\overline{\chi'}\} <: Q <: \{\overline{\eta'}\}} \qquad distinct(\overline{P}, \overline{Q}) \qquad \Delta \vdash dom(d) \sqsubseteq dom(d') \\ \Delta \vdash \forall[\![\overline{\kappa}]\!](\alpha \to \rho) \sqsubseteq \forall[\![\overline{\kappa}, \overline{\kappa'}]\!]((\alpha \sqcap \alpha') \to \rho') \end{array}}{\Delta \vdash d \text{ return type wrt } d' \text{ ok}} \text{ [Return-Test]}$$

Fig. 7. Overloading rules for FGFV

enclosing contexts; (3) parameter types of arrow types and contravariant type parameters of traits flip the variance of their enclosing contexts; and (4) type parameters of objects and invariant type parameters of traits are invariant. For example, if $\tau = ((P, T[\![Q]\!]) \to \texttt{Any})$ is a method's type where P and Q are class type parameters and T has a contravariant type parameter, then P is required to be contravariant or invariant and Q is required to be covariant or invariant.

4 OVERLOADING RULES

Now, we formalize the overloading rules informally discussed in Section 2.2 for FGFV.

4.1 Formalization of Overloading Rules

The overloading rules require that every pair in a set of overloaded method declarations satisfies all three rules defined in Figure 7. To determine whether one method declaration is more specific than the other, while the informal rules in Section 2.2 used the subset relation of their applicable sets, which is not practically checkable, we use the subtype relation of their domain types as in Allen et al. [2011]. Note that even though FGFV additionally supports variance annotations of trait type parameters, the overloading rules for FGF remain the same because covariant or contravariant types affect only the well-formedness of method declarations rather than the validity of overloading.

If two method declarations have different names, they vacuously satisfy all the rules ([No-Dup-Triv], [Meet-Triv], and [Return-Triv]). The No Duplicates Rule also checks whether the domain types of two declarations are different; if the domain type of one declaration is not a subtype of the

Well-formed substitution: $\boxed{\Delta \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{\chi} \rangle \text{ and } \langle \overline{\eta} \rangle}$

$$\frac{\sigma = [\overline{\alpha/P}] \qquad \overline{\underline{\Delta} \vdash \underline{\sigma}\chi <: \alpha} \qquad \overline{\underline{\Delta} \vdash \alpha <: \underline{\sigma}\eta}}{\Delta \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{\chi} \rangle \text{ and } \langle \overline{\eta} \rangle}$$

Fig. 8. Well-formed substitution

domain type of the other, they satisfy the rule ([No-Dup-Not-Less] and [No-Dup-Not-Gtr]). The Meet Rule is satisfied if two domain types exclude each other ([Meet-Excl]), they are in a subtype relation ([Meet-Less] and [Meet-Gtr]), or the domain type of a third declaration of the same name is the intersection type of the domain types of two declarations ([Meet-Third]). The Return Type Rule is satisfied if two domain types are not in a subtype relation ([Return-Not-Less]), or if two arrow types derived from the two declarations are in a subtype relation ([Return-Test]).

4.2 Quantified Types

As proposed by Bourdoncle and Merz [1997], we represent domain types and arrow types of generic method declarations as existential types Ξ and universal types Υ, respectively, collectively called *quantified types*. For presentation brevity, we omit C from $d = (C, M)$ where a method declaration $M = m[\![\overline{K}]\!](\overline{x{:}\,\alpha}){:}\,\rho = e$ is declared in a class C. The domain type of d is $dom(d) = \exists[\![\overline{K}]\!](\overline{\alpha})$ and its arrow type is $arrow(d) = \forall[\![\overline{K}]\!]((\overline{\alpha}) \to \rho)$. We compare domain types to decide whether one generic method declaration is more specific than the other; an existential type $\exists[\![\overline{K}]\!]\alpha$ is a subtype of another existential type $\exists[\![\overline{K'}]\!]\alpha'$ if there exists a well-formed substitution σ for the type parameters bound in $\overline{K'}$ that makes $\sigma\alpha'$ a supertype of α. A substitution is well-formed if it satisfies the bounds of type parameters as shown in Figure 8. In contrast, we compare arrow types to see whether the return type of a more specific generic method declaration can be a subtype of the return type of a less specific one; a universal type $\forall[\![\overline{K}]\!]\alpha$ is a subtype of the other universal type $\forall[\![\overline{K'}]\!]\alpha'$ if there exists a well-formed substitution σ for the type parameters bound in $\overline{K}$ that makes $\sigma\alpha$ a subtype of α'. Figure 9 formally defines these "inner" subtyping rules for quantified types. (We assume the usual conventions about using α-conversion to rename bound variables when necessary to satisfy premises that require variable names to be distinct.) Finally, the intersection type of two existential types $\exists[\![\overline{K}]\!]\alpha$ and $\exists[\![\overline{K'}]\!]\alpha'$ used in [Meet-Excl] and [Meet-Third] is $\exists[\![\overline{K}, \overline{K'}]\!](\alpha \sqcap \alpha')$, because whenever both $\sigma\alpha$ and $\sigma'\alpha'$ are supertypes of a type τ for some σ and σ', $(\sigma \circ \sigma')(\alpha \sqcap \alpha')$ is a supertype of τ as well.

While the subtype relation of domain types is a sound approximation of the subset relation of applicable sets, it is not complete; when the applicable set of one declaration is a subset of that of the other, their domain types may not be in a subtype relation. Consider the following:

$$m[\![P]\!](x : \mathsf{ArrayList}[\![P]\!])\ _ \qquad m[\![Q <: T]\!](y : \mathsf{List}[\![Q]\!])\ _ \qquad m[\![R <: T]\!](z : \mathsf{ArrayList}[\![R]\!])\ _$$

for some trait T where ArrayList extends List and both are invariant, and call them d_1, d_2, and d_3 in order. Even though they are a valid overloading because the applicable set of d_3 is the intersection of those of d_1 and d_2, the intersection of the two domain types $\Xi_\sqcap = \exists[\![P, Q <: T]\!](\mathsf{ArrayList}[\![P]\!] \sqcap \mathsf{List}[\![Q]\!])$ is not a subtype of the domain type of d_3, $\Xi_3 = \exists[\![R <: T]\!]\mathsf{ArrayList}[\![R]\!]$. However, because ArrayList extends List and both are invariant, any subtypes of theirs are exclusive unless they are both $\mathsf{ArrayList}[\![S]\!]$ for some type S that is a subtype of T. Thus, $\Xi_\sqcap$ denotes the same applicable set as $\exists[\![S <: T]\!]\mathsf{ArrayList}[\![S]\!]$, which is a subtype of Ξ_3 using $[R/S]$.

Existential inner subtyping: $\boxed{\Delta \vdash \Xi \lesssim \Xi \text{ using } \sigma}$

$$\frac{\Delta' = \Delta \cup \{\overline{\{\overline{\chi}\} <: P <: \{\overline{\eta}\}}\} \qquad \{\overline{P}\} \cap FV(\overline{\overline{\chi'}}, \overline{\overline{\eta'}}, \alpha') = \emptyset \qquad \sigma = \left[\overline{\gamma/Q}\right] \qquad \overline{\gamma \neq \texttt{Bottom}}}{\Delta' \vdash \sigma \text{ on } \langle \overline{Q} \rangle \text{ obeys } \langle \overline{\bigsqcup\{\overline{\chi'}\}} \rangle \text{ and } \langle \overline{\bigsqcap\{\overline{\eta'}\}} \rangle \qquad \Delta' \vdash \alpha <: \sigma\alpha'}{\Delta \vdash \exists[\![\overline{\{\overline{\chi}\} <: P <: \{\overline{\eta}\}}]\!]\alpha \lesssim \exists[\![\overline{\{\overline{\chi'}\} <: Q <: \{\overline{\eta'}\}}]\!]\alpha' \text{ using } \sigma} \text{ [E-Sub]}$$

Universal inner subtyping: $\boxed{\Delta \vdash \Upsilon \lesssim \Upsilon \text{ using } \sigma}$

$$\frac{\Delta' = \Delta \cup \{\overline{\{\overline{\chi'}\} <: Q <: \{\overline{\eta'}\}}\} \qquad \{\overline{Q}\} \cap FV(\overline{\overline{\chi}}, \overline{\overline{\eta}}, \alpha) = \emptyset \qquad \sigma = \left[\overline{\gamma/P}\right] \qquad \overline{\gamma \neq \texttt{Bottom}}}{\Delta' \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{\bigsqcup\{\overline{\chi}\}} \rangle \text{ and } \langle \overline{\bigsqcap\{\overline{\eta}\}} \rangle \qquad \Delta' \vdash \sigma\alpha <: \alpha'}{\Delta \vdash \forall[\![\overline{\{\overline{\chi}\} <: P <: \{\overline{\eta}\}}]\!]\alpha \lesssim \forall[\![\overline{\{\overline{\chi'}\} <: Q <: \{\overline{\eta'}\}}]\!]\alpha' \text{ using } \sigma} \text{ [U-Sub]}$$

Intersection of existential types: $\boxed{\Xi \sqcap \Xi}$

$$\frac{distinct(\overline{P}, \overline{P'}) \qquad \{\overline{P'}\} \cap FV(\overline{\overline{\chi}}, \overline{\overline{\eta}}, \alpha) = \{\overline{P}\} \cap FV(\overline{\overline{\chi'}}, \overline{\overline{\eta'}}, \alpha') = \emptyset}{\left(\exists[\![\overline{\{\overline{\chi}\} <: P <: \{\overline{\eta}\}}]\!]\alpha\right) \sqcap \left(\exists[\![\overline{\{\overline{\chi'}\} <: P' <: \{\overline{\eta'}\}}]\!]\alpha'\right) = \exists[\![\overline{\{\overline{\chi}\} <: P <: \{\overline{\eta}\}}, \overline{\{\overline{\chi'}\} <: P' <: \{\overline{\eta'}\}}]\!](\alpha \sqcap \alpha')}$$

Existential reduction: $\boxed{\Delta \vdash \Xi \xrightarrow{\equiv} \Xi \text{ using } \sigma}$

$$\frac{\Delta \vdash \alpha \not\equiv \texttt{Bottom} \Rightarrow C \qquad toConstraint(\{\overline{K}\}) = C' \qquad \Delta \vdash unify(C \wedge C') = (\sigma, C'') \qquad toBounds(C'') = \{\overline{K'}\}}{\Delta \vdash \exists[\![\overline{K}]\!]\alpha \xrightarrow{\equiv} \exists[\![\overline{K'}]\!]\sigma\alpha \text{ using } \sigma} \qquad \frac{\text{otherwise}}{\Delta \vdash \exists[\![\overline{K}]\!]\alpha \xrightarrow{\equiv} \exists[\![\overline{K}]\!]\alpha \text{ using } []}$$

Universal reduction: $\boxed{\Delta \vdash \Upsilon \xrightarrow{\equiv} \Upsilon \text{ using } \sigma}$

$$\frac{\Delta \vdash \exists[\![\overline{K}]\!]\alpha \xrightarrow{\equiv} \exists[\![\overline{K'}]\!]\alpha' \text{ using } \sigma}{\Delta \vdash \forall[\![\overline{K}]\!](\alpha \to \omega) \xrightarrow{\equiv} \forall[\![\overline{K'}]\!](\alpha' \to \sigma\omega) \text{ using } \sigma}$$

Existential subtyping: $\boxed{\Delta \vdash \Xi \sqsubseteq \Xi \text{ using } \sigma \qquad \Delta \vdash \Xi \sqsubseteq \Xi}$

$$\frac{\Delta \vdash \Xi \xrightarrow{\equiv} \Xi'' \text{ using } _ \qquad \Delta \vdash \Xi'' \lesssim \Xi' \text{ using } \sigma}{\Delta \vdash \Xi \sqsubseteq \Xi' \text{ using } \sigma} \qquad \frac{\Delta \vdash \Xi \sqsubseteq \Xi' \text{ using } _}{\Delta \vdash \Xi \sqsubseteq \Xi'}$$

Universal subtyping: $\boxed{\Delta \vdash \Upsilon \sqsubseteq \Upsilon \text{ using } \sigma \qquad \Delta \vdash \Upsilon \sqsubseteq \Upsilon}$

$$\frac{\Delta \vdash \Upsilon' \xrightarrow{\equiv} \Upsilon'' \text{ using } _ \qquad \Delta \vdash \Upsilon \lesssim \Upsilon'' \text{ using } \sigma}{\Delta \vdash \Upsilon \sqsubseteq \Upsilon' \text{ using } \sigma} \qquad \frac{\Delta \vdash \Upsilon \sqsubseteq \Upsilon' \text{ using } _}{\Delta \vdash \Upsilon \sqsubseteq \Upsilon'}$$

Constraint solving: $\boxed{\Delta \vdash unify(C) = (\sigma, C)}$

Conversion of type environment to constraints: $\boxed{toConstraint(\Delta) = C}$

Conversion of constraints to type environment: $\boxed{toBounds(C) = \Delta}$

Fig. 9. Subtype relations of quantified types

Therefore, in order to reduce the gap between the subset relation of applicable sets and the subtype relation of domain types, we extend the subtype relation of existential types using *existential reduction*. As defined in Figure 9, existential reduction infers a simplified existential type from a

Constraint grammar

$$
\begin{array}{rcl}
C & ::= & \mathsf{false} \mid \mathsf{true} \\
& \mid & T <: T \mid T \lozenge T \\
& \mid & T \not<: T \mid T \not\lozenge T \\
& \mid & C \wedge C \mid C \vee C
\end{array}
$$

Constraint negation: $\boxed{\neg C = C}$

$$
\begin{array}{llll}
\neg(\mathsf{false}) = \mathsf{true} & & \neg(\mathsf{true}) = \mathsf{false} \\
\neg(T <: T') = T \not<: T' & & \neg(T \lozenge T') = T \not\lozenge T' \\
\neg(T \not<: T') = T <: T' & & \neg(T \not\lozenge T') = T \lozenge T' \\
\neg(C \wedge C) = \neg(C) \vee \neg(C) & & \neg(C \vee C) = \neg(C) \wedge \neg(C)
\end{array}
$$

Conditional non-subtyping: $\boxed{\Delta \vdash \alpha \not<: \alpha \Leftarrow C}$

$$\frac{\Delta \vdash \alpha <: \eta \Leftarrow C}{\Delta \vdash \alpha \not<: \eta \Leftarrow \neg C}$$

Conditional equivalence: $\boxed{\begin{array}{c}\Delta \vdash \alpha \equiv \alpha \Leftarrow C \\ \Delta \vdash \alpha \not\equiv \alpha \Leftarrow C\end{array}}$

$$\frac{\Delta \vdash \alpha <: \eta \Leftarrow C \qquad \Delta \vdash \eta <: \alpha \Leftarrow C'}{\Delta \vdash \alpha \equiv \eta \Leftarrow C \wedge C'}$$

$$\frac{\Delta \vdash \alpha \equiv \eta \Leftarrow C}{\Delta \vdash \alpha \not\equiv \eta \Leftarrow \neg C}$$

Conditional variance test:

$$\frac{\boxed{\Delta \vdash \alpha \text{ and } \alpha \text{ compare } V \Leftarrow C} \quad \Delta \vdash \alpha <: \eta \Leftarrow C}{\Delta \vdash \alpha \text{ and } \eta \text{ compare } + \Leftarrow C}$$

$$\frac{\Delta \vdash \eta <: \alpha \Leftarrow C}{\Delta \vdash \alpha \text{ and } \eta \text{ compare } - \Leftarrow C}$$

$$\frac{\Delta \vdash \alpha \equiv \eta \Leftarrow C}{\Delta \vdash \alpha \text{ and } \eta \text{ compare } = \Leftarrow C}$$

Converse non-equivalence: $\boxed{\Delta \vdash \alpha \not\equiv \alpha \Rightarrow C}$

$$\frac{\Delta \vdash \alpha \equiv \eta \Leftarrow C}{\Delta \vdash \alpha \not\equiv \eta \Rightarrow \neg C}$$

Fig. 10. Constraints. The $\wedge$, $\vee$, and $\neg$ operators form a Boolean algebra.

given existential type $\exists[\![\overline{K}]\!]\alpha$. As defined in Figure 10, it first collects constraints C that make α different from `Bottom`: $\Delta \vdash \alpha \not\equiv \texttt{Bottom} \Rightarrow C$. It then tries to find types that instantiate the type parameters bound in $\overline{K}$ or simply collects their bounds by solving C and the constraints on the type parameters; it makes a substitution σ for the type parameters that have types to instantiate with and constraints C'' for the type parameters that have only bounds. Using the substitution σ and type parameter bindings $\overline{K'}$ generated from C'', it produces a simplified existential type $\exists[\![\overline{K'}]\!]\sigma\alpha$. Finally, in the extended subtype relation, one existential type is a subtype of the other if a reduced existential type of the former is a subtype of the latter by the inner subtyping rule as shown in Figure 9.

Collecting constraints C that make a type α different from another type α' requires various relations between types such as subtype, non-subtype, equivalence, non-equivalence, exclusion, and non-exclusion relations. A constraint is inductively defined with boolean literals, true and false, and simple type relations like subtype, non-subtype, exclusion, and non-exclusion relations as base cases, and conjunction and disjunction of constraints as inductive steps. We also defined the various type relations inductively on the structure of types. In addition, we defined all the type relations in a way that supports complete and disjoint case analysis for types. The entire type relations are formally defined in Appendix A.

Arrow types are compared only in rule [Return-Test], to see whether the return type of a more specific generic method declaration can be a subtype of the return type of a less specific one. As the informal rule in Section 2.2 specifies, Return Type Rule checks that for two declarations d_1 and d_2 where d_1 is more specific than d_2, and for any non-`Bottom` type γ that d_1 is applicable to, the return type of an instance of d_2 that is applicable to γ is a supertype of the return type of some instance of d_1 that is applicable to γ. When the arrow types of d_1 and d_2 are $\forall[\![\overline{\kappa}]\!](\alpha \to \rho)$ and $\forall[\![\overline{\kappa'}]\!](\alpha' \to \rho')$, respectively, [Return-Test] checks this by checking whether the arrow type of d_1 is a subtype of $\forall[\![\overline{\kappa}, \overline{\kappa'}]\!]((\alpha \sqcap \alpha') \to \rho')$. Since d_1 is more specific than d_2 and is applicable to γ, there exist substitutions σ_1 and σ_2 that make d_1 and d_2 applicable to γ, respectively, thus both

$\sigma_1\alpha$ and $\sigma_2\alpha'$ are supertypes of γ, therefore so is $(\sigma_1 \circ \sigma_2)(\alpha \sqcap \alpha')$. Thus, if the arrow type of d_1 is a subtype of $\forall[\![\,\overline{\kappa}, \overline{\kappa'}\,]\!]((\alpha \sqcap \alpha') \to \rho')$ using some substitution σ, it makes $\sigma\alpha$ a supertype of $(\alpha \sqcap \alpha')$, which makes $(\sigma_1 \circ \sigma_2 \circ \sigma)\alpha$ a supertype of γ, and $\sigma\rho$ a subtype of ρ'. Hence, $\sigma_1 \circ \sigma_2 \circ \sigma$ is the one that makes the instance of d_1 that is applicable to γ and has the return type that is a subtype of the instance of d_2. In summary, [Return-Test] ensures that for any instance of d_2 instantiated with σ_2 that is applicable to γ, there exists an instance of d_1 instantiated with $\sigma_1 \circ \sigma_2 \circ \sigma$ that is applicable to γ and has a return type that is a subtype of the return type of the instance of d_2.

In the same manner as for existential types, we extend the subtype relation of universal types using *universal reduction* as defined in Figure 9. When [Return-Test] checks whether one universal type is a subtype of the other, the domain type of the latter is an intersection type. Thus, in order to utilize the extended expressivity by existential reduction, universal reduction takes a universally quantified arrow type $\forall[\![\,\overline{K}\,]\!](\alpha \to \omega)$, simplifies its domain type to $\exists[\![\,\overline{K'}\,]\!]\alpha'$ using σ via existential reduction, and produces a simplified universal type $\forall[\![\,\overline{K'}\,]\!](\alpha' \to \sigma\omega)$. In the extended subtype relation, one universal type is a subtype of the other if the former is a subtype of a reduced universal type of the latter by the inner subtyping rule as shown in Figure 9.

4.3 Unique Existence of the Most Specific Declaration

For any method invocation, there always exists a unique most specific method declaration in its set of applicable method declarations.

Theorem 4.1. *Let* $\{\overline{d}\} = \mathit{visible}(\Delta, (C, m))$ *for* $\Delta \vdash C$ ok.
If $\mathit{applicable}(\Delta, \{\overline{d}\}, \alpha) \neq \emptyset$ *for* $\Delta \vdash \alpha$ ok *and* $\alpha \neq$ `Bottom`,
then $\left|\mathit{mostSpecific}(\Delta, \mathit{applicable}(\Delta, \{\overline{d}\}, \alpha))\right| = 1$.

Proof. See Appendix D. □

5 OVERLOADING RESOLUTION

Method overloading is resolved both at compile time for type checking and at run time for evaluation, and these *static and dynamic overloading resolutions* should be consistent. For a method invocation expression, static overloading resolution chooses an instance of the most specific method declaration that is applicable to the static types of the method arguments; dynamic overloading resolution then must select an instance of the most specific method declaration that is applicable to the dynamic types of the arguments and whose return type is a subtype of the static type of the expression.

5.1 Static Overloading Resolution

As discussed in Section 2.3, in order to make type-sound choices, the static types of method invocation expressions must be available at run time. Thus, checking well-formedness of an expression e produces its type-annotated version ϵ and its type α as shown in Figure 11. A *type-annotated internal expression* is just like an expression except that all the method invocation expressions in it are annotated with their static types:

Type-annotated internal expression $\quad \epsilon ::= z \mid ((\overline{x{:}\,\alpha}){:}\,\alpha \Rightarrow \epsilon) \mid \epsilon@(\overline{\epsilon}) \mid O[\![\,\overline{\alpha}\,]\!](\overline{\epsilon}) \mid \epsilon.m(\overline{\epsilon}){:}\,\alpha$

We focus on rule [T-Method] for method invocation, since the rules for the other kinds of expressions are conventional. For a method invocation $e.m(\overline{e'})$, after checking well-formedness of expressions e and $\overline{e'}$, the rule performs static overloading resolution via rule [MSAV] to find an instance of the most specific applicable visible method declaration, and annotates the method invocation with the return type of the chosen instance. Rule [MSAV] first selects the most specific applicable visible method declaration d' with a set of substitutions $\{\overline{\sigma'}\}$ using the static types

Well-formed expressions: $\boxed{\Delta;\Gamma \vdash e : (\epsilon, \alpha)}$

$$\frac{\Gamma = __, z\colon\alpha, \overline{z'\colon_} \qquad \overline{z \neq z'}}{\Delta;\Gamma \vdash z : (z, \alpha)} \quad \text{[T-Variable]}$$

$$\frac{distinct(\overline{x}) \qquad \overline{\Delta \vdash \tau\ \mathsf{ok}} \qquad \Delta \vdash \omega\ \mathsf{ok} \qquad \Delta;\Gamma, \overline{x\colon\tau} \vdash e : (\epsilon, \rho) \qquad \Delta \vdash \rho <\colon \omega}{\Delta;\Gamma \vdash ((\overline{x\colon\tau})\colon\omega \Rightarrow e) : (((\overline{x\colon\tau})\colon\omega \Rightarrow \epsilon), ((\overline{\tau}) \to \omega))} \quad \text{[T-Closure]}$$

$$\frac{\Delta;\Gamma \vdash e : (\epsilon, ((\overline{\alpha}) \to \rho)) \qquad \overline{\Delta;\Gamma \vdash e' : (\epsilon', \chi)} \qquad \overline{\Delta \vdash \chi <\colon \alpha}}{\Delta;\Gamma \vdash e@(\overline{e'}) : (\epsilon@(\overline{\epsilon'}), \rho)} \quad \text{[T-Apply]}$$

$$\frac{\begin{array}{c}\mathtt{object}\ O[\![\,\overline{P <\colon _}\,]\!](\overline{x\colon\omega})\ __\ \mathsf{end} \in \Delta \\ \Delta \vdash O[\![\,\overline{\tau}\,]\!]\ \mathsf{ok} \qquad \overline{\Delta;\Gamma \vdash e : (\epsilon, \eta)} \qquad \overline{\Delta \vdash \eta <\colon \left[\overline{\tau/P}\right]\omega}\end{array}}{\Delta;\Gamma \vdash O[\![\,\overline{\tau}\,]\!](\overline{e}) : (O[\![\,\overline{\tau}\,]\!](\overline{\epsilon}), O[\![\,\overline{\tau}\,]\!])} \quad \text{[T-Object]}$$

$$\frac{\Delta;\Gamma \vdash e : (\epsilon, C) \qquad \overline{\Delta;\Gamma \vdash e' : (\epsilon', \eta)} \qquad msav(\Delta, (C, m), (C, \overline{\eta})) = m[\![\,]\!](_)\colon\rho = _}{\Delta;\Gamma \vdash e.m(\overline{e'}) : (\epsilon.m(\overline{\epsilon'})\colon\rho, \rho)} \quad \text{[T-Method]}$$

Most specific applicable visible method: $\boxed{msav(\Delta, (C, m), \alpha) = D}$

$$\frac{\begin{array}{cc}mostSpecific(\Delta, applicable(\Delta, visible(\Delta, (C, m)), \alpha)) = \{\overline{(d, \{\overline{\sigma}\})}\} & \#\left(\overline{(d, \{\overline{\sigma}\})}\right) = 1 \\ (d', \{\overline{\sigma'}\}) \in \{\overline{(d, \{\overline{\sigma}\})}\} \qquad \overline{instantiate(\underline{d'}, \sigma') = D'} & D \in \{\overline{D'}\}\end{array}}{msav(\Delta, (C, m), \alpha) = D} \quad \text{[MSAV]}$$

Miscellaneous judgments:

$$\boxed{\begin{array}{cc}mostSpecific(\Delta, \{\overline{(d, \{\overline{\sigma}\})}\}) = \{\overline{(d, \{\overline{\sigma}\})}\} & moreSpecific(\Delta)(d, d') \\ applicable(\Delta, \{\overline{d}\}, \alpha) = \{\overline{(d, \{\overline{\sigma}\})}\} & visible(\Delta, (C, m)) = \{\overline{d}\} \quad instantiate(d, \sigma) = D\end{array}}$$

$$mostSpecific(\Delta, \{\overline{(d, \{\overline{\sigma}\})}\}) = \left\{ (d', \{\overline{\sigma'}\}) \;\middle|\; (d', \{\overline{\sigma'}\}) \in \{\overline{(d, \{\overline{\sigma}\})}\}, \bigwedge_{(d'', _) \in \{\overline{(d, \{\overline{\sigma}\})}\}} moreSpecific(\Delta)(d', d'') \right\} \quad \text{[Most-Specific]}$$

$$moreSpecific(\Delta)(d, d') = \Delta \vdash dom(d) \sqsubseteq dom(d') \quad \text{[More-Specific]}$$

$$applicable(\Delta, \{\overline{d}\}, \alpha) = \left\{ (d', \{\overline{\sigma}\}) \;\middle|\; d' \in \{\overline{d}\}, \{\overline{\sigma}\} = \left\{ \sigma' \;\middle|\; \Delta \vdash \exists[\![\,]\!]\alpha \sqsubseteq dom(d') \text{ using } \sigma' \right\} \neq \emptyset \right\} \quad \text{[Applicable]}$$

$$visible(\Delta, (C, m)) = \{\, d \mid d \in allVisible(\Delta, C), name(d) = m \,\} \quad \text{[Visible]}$$

$$instantiate\big((C, m[\![\,__\,]\!](\overline{x\colon\alpha})\colon\rho = e), \sigma\big) = \big(C, m[\![\,]\!](\overline{x\colon\sigma\alpha})\colon\sigma\rho = \sigma e\big) \quad \text{[Instantiate-Method]}$$

Fig. 11. Well-formed expressions

of the method arguments α. The unique existence of the most specific applicable visible method

Class-declaration environment	Ψ	$::= \{\overline{\psi}\}$
Ground type	g	$::= T[\![\overline{g}]\!] \mid O[\![\overline{g}]\!] \mid (\overline{g}) \mid (g \to g) \mid \mathsf{Any} \mid (g \sqcup g) \mid (g \sqcap g)$
Value	v	$::= O[\![\overline{g}]\!](\overline{v}) \mid ((\overline{x{:}g}){:}g \Rightarrow \epsilon)$
Ilk	k	$::= O[\![\overline{g}]\!] \mid (\overline{g}) \mid (g \to g)$
Evaluation context	E	$::= \square \mid E@(\overline{\epsilon}) \mid v@(\overline{v}, E, \overline{\epsilon}) \mid O[\![\overline{g}]\!](\overline{v}, E, \overline{\epsilon}) \mid E.m(\overline{\epsilon}){:}g \mid v.m(\overline{v}, E, \overline{\epsilon}){:}g$
Redex	R	$::= v@(\overline{v}) \mid v.m(\overline{v}){:}g$

Dynamic semantics: $\boxed{\Psi \vdash E\boxed{R} \longrightarrow E\boxed{\epsilon}}$

$$\frac{v'' = O[\![\overline{g}]\!](\overline{v}) \qquad \mathtt{object}\ O[\![_]\!](\overline{x{:}_})\ _ \ \mathsf{end} \in \Psi \qquad \langle \overline{d} \rangle = \mathit{topologicalSort}(\Psi, \mathit{visible}(\Psi, (O[\![\overline{g}]\!], m))) \qquad (_, m[\![\ ]\!](\overline{x'{:}_}){:}_ = \epsilon) = \mathit{firstDispatchable}(\Psi, \langle \overline{d} \rangle, (\mathit{ilk}(v''), \overline{\mathit{ilk}(v')}), g')}{\Psi \vdash E\boxed{v''.m(\overline{v'}){:}g'} \longrightarrow E\boxed{[v''/\mathtt{self}, \overline{v/x}, \overline{v'/x'}]\epsilon}} \text{ [R-Method]}$$

First dispatchable method instance: $\boxed{\mathit{firstDispatchable}(\Psi, \langle \overline{d} \rangle, k, g) = D}$

$$\frac{\langle \overline{d} \rangle = \langle \overline{d'}, d'', __ \rangle \qquad \overline{\neg(\underline{\Psi} \vdash \underline{k} \text{ and } \underline{g} \text{ dispatch } d' \text{ using } _)} \qquad \Psi \vdash k \text{ and } g \text{ dispatch } d'' \text{ using } \sigma \qquad D = \mathit{instantiate}(d'', \sigma)}{\mathit{firstDispatchable}(\Psi, \langle \overline{d} \rangle, k, g) = D} \text{ [First-Dispatchable]}$$

Sorting of method declarations: $\boxed{\mathit{topologicalSort}(\Delta, \{\overline{d}\}) = \langle \overline{d} \rangle}$

$$\frac{\{\overline{d}\} = \{\overline{d'}\} \qquad \overline{\neg(\mathit{moreSpecific}(\Delta)(d'_j, d'_i))}^{1 \le i < j \le \#(\overline{d'})}}{\mathit{topologicalSort}(\Delta, \{\overline{d}\}) = \langle \overline{d'} \rangle} \text{ [Topological-Sort]}$$

Dispatch semipredicate: $\boxed{\Psi \vdash k \text{ and } g \text{ dispatch } d \text{ using } \sigma}$

$$\frac{\mathit{arrow}(d) = \forall[\![\overline{K}]\!](\alpha \to \rho) \qquad \Psi \vdash k \text{ match } [\![\overline{K}]\!]\ \alpha \text{ + bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle \qquad \Psi \vdash g \text{ match } [\![\overline{K}]\!]\ \rho \text{ - bounds } \left\langle \overline{\langle \overline{l'} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u'} \rangle} \right\rangle \qquad \Psi \vdash \text{solve } [\![\overline{K}]\!] \text{ bounds } \bigsqcup\nolimits^{\times}_{\overline{K}} \left\langle \left\langle \overline{\langle \overline{l} \rangle} \right\rangle, \left\langle \overline{\langle \overline{l'} \rangle} \right\rangle \right\rangle \text{ and } \bigsqcap\nolimits^{\times}_{\overline{K}} \left\langle \left\langle \overline{\langle \overline{u} \rangle} \right\rangle, \left\langle \overline{\langle \overline{u'} \rangle} \right\rangle \right\rangle \text{ using } \sigma}{\Psi \vdash k \text{ and } g \text{ dispatch } d \text{ using } \sigma} \text{ [Dispatch]}$$

Fig. 12. Dynamic semantics of method invocation expressions

declaration is guaranteed by Theorem 4.1. Then, the rule selects one instance D out of instantiations of d' with $\{\overline{\sigma'}\}$.

5.2 Dynamic Overloading Resolution

When evaluating a method invocation expression, dynamic overloading resolution selects a method declaration to call by using the dynamic types of the arguments and the static type of the expression. We describe the dynamic overloading resolution in a top-down manner from the dynamic semantics rule for method invocation (Section 5.2.1) to the details of its helper judgments (Sections 5.2.2 and 5.2.3), present a concrete example (Section 5.2.4), and prove its correctness (Section 5.2.5).

5.2.1 Dynamic Semantics.

Dynamic overloading resolution uses additional metavariables as in Figure 12. A class-declaration environment Ψ is a set of class declarations. A ground type g is a dynamically available type, which is an internal type that does not contain type parameters nor `Bottom`; our type system requires that

both static and dynamic type inference algorithms instantiate type parameters with non-`Bottom` types, and that all the type parameters are instantiated with ground types at run time. We call the run-time type of a value the *ilk* of the value; the ilk of an object instance $O[\![\overline{g}]\!](\overline{v})$ is $O[\![\overline{g}]\!]$, and the ilk of a function $((\overline{x{:}\,g}){:}\,g' \Rightarrow \epsilon)$ is $((\overline{g}) \to g')$. Similarly to domain types, we write $(\overline{g})$ to denote ilks of multiple function parameters. Evaluation contexts E and redexes R are conventional.

For a method invocation, rule [R-Method] seeks the most specific method declaration with a *dispatchable instance* that is applicable to the ilk of the arguments and whose return type is a subtype of the annotated static return type. We say a method declaration is *dispatchable* to the ilk of arguments and the static return type if it has a dispatchable instance. For the sake of efficiency, rule [R-Method] first topologically sorts all the visible method declarations in a most-to-least specific order. (This order is statically determinable, so the sorting can be done at compile time.) The rule then checks the *dispatch semipredicate* for each method declaration in the sorted list until it finds the first method declaration d'' dispatchable to the given ilk of the arguments k and the static return type g with a dispatchable instance $D = \mathit{instantiate}(d'', \sigma)$.

The dispatch semipredicate defined in rule [Dispatch] checks if a given method declaration d is dispatchable to a given ilk k and a static return type g. The dispatch semipredicate is satisfied using substitution σ if d has a dispatchable instance (obtained by deleting the method type parameters of d and applying σ to its argument types, return type, and body). Metavariable l and a range over lower type bounds, and u and b range over upper type bounds. We write $\langle \overline{l} \rangle$ to denote ordered lists. The dispatch semipredicate consists of three steps: (1) computing type bounds $\left\langle \overline{\langle \overline{l} \rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u} \rangle} \right\rangle$ that will make the domain type of the method declaration d be a supertype of the ilk k, so that d will be applicable to k, (2) computing type bounds $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle$ that make the static return type g a supertype of the return type of d to make sure that the dynamic dispatch preserves type, and (3) solving those type bounds to get a valid substitution σ. (If any of these steps fails, the semipredicate is false: the declaration is not applicable.) The computed type bounds are either ground types or `Bottom`. The dispatch semipredicate computes type bounds by type matching as described in the next section.

5.2.2 *Computing Bounds by Type Matching.*

The match rules in Figure 13 compute lower and upper type bounds $\left\langle \overline{\langle \overline{l} \rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u} \rangle} \right\rangle$ for method type parameters in $\overline{K}$ by matching a ground type g against an internal type α according to a variance mark V. When V is respectively +, −, or =, the rules compute the type bounds that make g respectively a subtype, a supertype, or an equivalent type of α. For example, as in [M-Param-Co], if a ground type g is matched against a method type parameter P_i according to +, g becomes the lower bound of P_i so that g is a subtype of P_i as long as P_i satisfies its type bounds $g <: P_i <: \mathsf{Any}$.

The match rules compute lower and upper type bounds in the form of two lists of lists $\left\langle \overline{\langle \overline{l} \rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u} \rangle} \right\rangle$ where $\#(\overline{\langle \overline{l} \rangle}) = \#(\overline{\langle \overline{u} \rangle})$ and $\overline{\#(\overline{K}) = \#(\overline{l_i}) = \#(\overline{u_i})}^{\,i}$. Matching may produce multiple solutions; each inner list is one possible solution. Multiple solutions come from cases with union types and intersection types as in rule [M-Union-Whatever-Contra], which matches a union ground type $\bigsqcup\langle \overline{g} \rangle$ against α according to − and computes type bounds that make $\bigsqcup\langle \overline{g} \rangle$ a supertype of α. Because a union type $\bigsqcup\langle \overline{g} \rangle$ is a supertype of α if any component g_i is a supertype of α, multiple components may be supertypes of α, which may lead to multiple type bounds for each of those components. Since we cannot decide which one of the multiple type bounds is the one eventually satisfying all the constraints, the match rules compute all possible solutions, which are then winnowed in the solving step. To collect all the bounds, the match rules cover failure cases as well so that the rules do not stop collecting bounds even when one component fails type matching.

Type matching a ground type to an internal type: $\boxed{\Psi \vdash g \text{ match } [\![\overline{K}]\!]\ \alpha\ V \text{ bounds } \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u} \rangle} \right\rangle}$

$$\frac{g \text{ is not an intersection type} \qquad g \text{ is not a union type} \qquad \overline{l = (\text{if } (P = \underline{P_i}) \text{ then } \underline{g} \text{ else } \texttt{Bottom})} \qquad \overline{u = \texttt{Any}} \qquad 1 \le i \le \#(\overline{P}) = \#(\overline{u})}{\Psi \vdash g \text{ match } [\![\overline{\{\chi\} <: P <: \{\eta\}}]\!]\ P_i \text{ + bounds } \left\langle \langle \bar{l} \rangle \right\rangle \text{ and } \left\langle \langle \bar{u} \rangle \right\rangle} \text{ [M-Param-Co]}$$

$$\frac{\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \alpha\ \underline{V} \text{ bounds } \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u} \rangle} \right\rangle}}{\Psi \vdash (\overline{g}) \text{ match } [\![\overline{K}]\!]\ (\overline{\alpha})\ V \text{ bounds } \bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \bar{u} \rangle} \right\rangle} \text{ [M-Tuple-Tuple-Success]}$$

$$\frac{\#(\overline{g}) \neq \#(\overline{\alpha})}{\Psi \vdash (\overline{g}) \text{ match } _\ (\overline{\alpha})\ _ \text{ bounds } \langle\rangle \text{ and } \langle\rangle} \text{ [M-Tuple-Tuple-Failure]}$$

$$\frac{\alpha \text{ is not an intersection type} \qquad \alpha \text{ is not a union type} \qquad \overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \underline{\alpha} \text{ - bounds } \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u} \rangle} \right\rangle}}{\Psi \vdash \bigsqcup \langle \overline{g} \rangle \text{ match } [\![\overline{K}]\!]\ \alpha \text{ - bounds } \left\langle \overline{\overline{\langle \bar{l} \rangle}} \right\rangle \text{ and } \left\langle \overline{\overline{\langle \bar{u} \rangle}} \right\rangle} \text{ [M-Union-Whatever-Contra]}$$

Fig. 13. Type matching a ground type to an internal type (selected rules)

Transposing reduction on list of lists: $\boxed{\bigsqcup^{\mathsf{T}} \left\langle \overline{\langle \overline{\alpha} \rangle} \right\rangle = \langle \overline{\alpha} \rangle \quad \bigsqcap^{\mathsf{T}} \left\langle \overline{\langle \overline{\alpha} \rangle} \right\rangle = \langle \overline{\alpha} \rangle}$

$$\bigsqcup^{\mathsf{T}} \left\langle \overline{\langle \overline{\alpha_{i\,j}}^{\,j} \rangle}^{\,i} \right\rangle = \left\langle \overline{\bigsqcup \langle \overline{\alpha_{i\,j}}^{\,i} \rangle}^{\,j} \right\rangle \qquad \bigsqcap^{\mathsf{T}} \left\langle \overline{\langle \overline{\alpha_{i\,j}}^{\,j} \rangle}^{\,i} \right\rangle = \left\langle \overline{\bigsqcap \langle \overline{\alpha_{i\,j}}^{\,i} \rangle}^{\,j} \right\rangle$$

Reduction of a cross product: $\boxed{\bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\alpha} \rangle} \right\rangle = \langle \overline{\alpha} \rangle \quad \bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\alpha} \rangle} \right\rangle = \langle \overline{\alpha} \rangle}$

$$\bigsqcup^{\times}_{\overline{K}} \langle\rangle = \langle \overline{\texttt{Bottom}}^{\#(\overline{K})} \rangle$$

$$\bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\alpha} \rangle}, \langle \overline{\eta} \rangle \right\rangle = \left\langle \overline{\overline{\bigsqcup^{\mathsf{T}} \langle \langle \overline{\gamma_{i\,k}}^{\,k} \rangle, \langle \overline{\eta_{j\,k}}^{\,k} \rangle \rangle}^{\,j}}^{\,i} \right\rangle \quad \textbf{where} \quad \langle \overline{\gamma} \rangle = \bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\alpha} \rangle} \right\rangle$$

$$\bigsqcap^{\times}_{\overline{K}} \langle\rangle = \langle \overline{\texttt{Any}}^{\#(\overline{K})} \rangle$$

$$\bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\alpha} \rangle}, \langle \overline{\eta} \rangle \right\rangle = \left\langle \overline{\overline{\bigsqcap^{\mathsf{T}} \langle \langle \overline{\gamma_{i\,k}}^{\,k} \rangle, \langle \overline{\eta_{j\,k}}^{\,k} \rangle \rangle}^{\,j}}^{\,i} \right\rangle \quad \textbf{where} \quad \langle \overline{\gamma} \rangle = \bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\alpha} \rangle} \right\rangle$$

Fig. 14. Cartesian product reduction operators

For example, rule [M-Tuple-Tuple-Failure] specifies a failure case due to size mismatch of tuple types; rather than actually failing, it succeeds but delivers empty lists of solutions. The entire match rules are defined in Appendix B.

Lists of solutions for component types are combined in the manner of a Cartesian product as defined in Figure 14. For instance, rule [M-Tuple-Tuple-Success] reduces $\bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\langle \bar{l}_i \rangle} \rangle}^{\,i} \right\rangle \left(\bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\langle \bar{u}_i \rangle} \rangle}^{\,i} \right\rangle \right)$ by computing componentwise unions (intersections) for all the combinations of choosing one type

Solving type bounds: $\boxed{\Psi \vdash \text{solve } [\![\overline{K}]\!] \text{ bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle \text{ using } \sigma}$

$$\frac{}{\Psi \vdash \text{solve } [\![ ]\!] \text{ bounds } \langle\langle\rangle\rangle \text{ and } \langle\langle\rangle\rangle \text{ using } []} \text{ [Solve-Empty]}$$

$$\frac{\#\left(\overline{\langle \overline{l}, l' \rangle}\right) = \#\left(\overline{\langle \overline{u}, u' \rangle}\right) = n \geq 1 \qquad 1 \leq i \leq n \qquad g' = (u'_i \sqcap \bigsqcap \{\overline{\eta'}\}) \qquad \Psi \vdash l'_i <: g' \qquad \underline{\Psi} \vdash \underline{g'} \text{ match } [\![\overline{K}]\!]\ \chi' \text{ - bounds } \left\langle \overline{\langle \overline{l''} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u''} \rangle} \right\rangle \quad \Psi \vdash \text{solve } [\![\overline{K}]\!] \text{ bounds } \bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\langle \overline{l''} \rangle} \rangle}, \langle \langle \overline{l_{\underline{i}}} \rangle \rangle \right\rangle \text{ and } \bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\langle \overline{u''} \rangle} \rangle}, \langle \langle \overline{u_{\underline{i}}} \rangle \rangle \right\rangle \text{ using } [\overline{g/P}]}{\Psi \vdash \text{solve } [\![\overline{K}, \{\overline{\chi'}\} <: P' <: \{\overline{\eta'}\}]\!] \text{ bounds } \left\langle \overline{\langle \overline{l}, l' \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u}, u' \rangle} \right\rangle \text{ using } [\overline{g/P}, g'/P']} \text{ [Solve-Step]}$$

Fig. 15. Solving type bounds

bound from each i-th list of the type bounds. Hence, inductively matching n components whose i-th component has m_i type bounds produces a list of $m_1 \times \cdots \times m_n$ type bounds (thus if matching any component fails and produces $\langle\rangle$, the whole type matching fails and results in $\langle\rangle$). For example,

$$\bigsqcup^{\times}_{\overline{K}} \left\langle \langle \langle A, B \rangle, \langle C, D \rangle \rangle, \langle \langle E, F \rangle, \langle G, H \rangle, \langle I, J \rangle \rangle \right\rangle =$$
$$\langle \langle A \sqcup E, B \sqcup F \rangle, \langle A \sqcup G, B \sqcup H \rangle, \langle A \sqcup I, B \sqcup J \rangle, \langle C \sqcup E, D \sqcup F \rangle, \langle C \sqcup G, D \sqcup H \rangle, \langle C \sqcup I, D \sqcup J \rangle \rangle.$$

5.2.3 *Solving the Bounds.*

The last step of the dispatch semipredicate is to solve the computed type bounds for method type parameters as in Figure 15. The solve rules compute a valid substitution σ for all the method type parameters in $\overline{K}$, which satisfies the declared bounds in $\overline{K}$ and the computed bounds from type matching $\left\langle \overline{\langle \overline{l} \rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u} \rangle} \right\rangle$. The solving step succeeds if any of the type bounds leads to a valid substitution. For declared type bounds for method type parameters $\overline{P}, P'$ and their computed type bounds $\left\langle \overline{\langle \overline{l}, l' \rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u}, u' \rangle} \right\rangle$ from type matching, rule [Solve-Step] nondeterministically selects i-th type bounds $\langle \overline{l_{\underline{i}}}, l'_i \rangle$ and $\langle \overline{u_{\underline{i}}}, u'_i \rangle$ that successfully compute a valid substitution. Starting from the rightmost type parameter P', the rule computes a substitution $[g'/P']$ by taking the intersection of the upper bounds of P'. Because g' is a ground type, the solving step never instantiates method type parameters with `Bottom`. The match premises propagate the choice of g' for P' to the declared lower bounds of P' to collect additional constraints on $\overline{P}$. Then, the rule combines the additional constraints and the originally computed bounds, and inductively solves the combined bounds to compute the substitution $[\overline{g/P}]$. From this, it produces a valid substitution $[\overline{g/P}, g'/P']$.

Note that the rule selects i-th type bounds nondeterministically while it checks type parameters deterministically from the rightmost one. Because the scope of a well-formed method type parameter extends to the right and method type parameters can appear only in lower bounds, for well-formed type parameters $\{\overline{\chi}\} <: P <: \{\overline{\eta}\}$, the upper bounds $\{\overline{\eta}\}$ do not contain any of $\overline{P}$, and P_j does not appear in the lower bounds of P_i for $i \leq j$. Thus, the choice of g_i for P_i does not affect the upper bounds of P_j, which allows us to compute a substitution in the right-to-left order by taking intersections of upper bounds. Since the choice of g_j for P_j may place additional constraints on P_i's that appear in the lower bounds $\{\overline{\chi_{\underline{j}}}\}$ of P_j, [Solve-Step] propagates the choice of g_j for P_j to the lower bounds and computes additional constraints, to ensure that g_j is a supertype of all $\{\overline{\chi_{\underline{j}}}\}$.

5.2.4 *Example.*

Let us revisit the dynamic overloading resolution process with a concrete example. Suppose we have two covariant traits `List` and `SortedList` such that `SortedList`$[\![P]\!] <:$ `List`$[\![P]\!]$, two objects `Cons` and `SortedCons` such that `Cons`$[\![P]\!] <:$ `List`$[\![P]\!]$ and `SortedCons`$[\![P]\!] <:$ `SortedList`$[\![P]\!]$,

and a method *append* that concatenates two lists. Consider the following method declarations:

$$\mathit{append}[\![P, \{P\} <: Q]\!](x\colon \texttt{SortedList}[\![P]\!], y\colon \texttt{SortedList}[\![Q]\!])\colon \texttt{SortedList}[\![Q]\!] = _$$
$$\mathit{append}[\![P, \{P\} <: Q]\!](x\colon \texttt{List}[\![P]\!], y\colon \texttt{List}[\![Q]\!])\colon \texttt{List}[\![Q]\!] = _$$

and call them d_1 and d_2 in order. This is a valid overloading since it satisfies the overloading rules with d_1 being more specific than d_2. Suppose we have three monomorphic classes C_1, C_2, and C_3 such that $C_1 <: C_3$ and $C_2 <: C_3$, and consider the following method invocation expression:

$$\mathit{append}(l_1, l_2)$$

with two lists l_1 and l_2 whose static types are both $\texttt{List}[\![C_3]\!]$ and whose ilks are $\texttt{SortedCons}[\![C_1]\!]$ and $\texttt{Cons}[\![C_2]\!]$, respectively. Then, type checking this expression would produce the internal expression $\mathit{append}(l_1, l_2)\colon \texttt{List}[\![C_3]\!]$.

Now, let us see how this method call is dynamically dispatched. Given the ilk of the argument tuple $(\texttt{SortedCons}[\![C_1]\!], \texttt{Cons}[\![C_2]\!])$, the static return type $\texttt{List}[\![C_3]\!]$, and the topologically sorted list of method declarations $\langle d_1, d_2\rangle$, dynamic overloading resolution performs the dispatch semipredicate starting from the first declaration d_1 in the list. The dispatch semipredicate on d_1 would succeed if it computed a valid substitution. However, it fails because its first type matching step fails, producing an empty list of solutions:

$$\frac{\Psi \vdash \texttt{SortedCons}[\![C_1]\!] \text{ match } [\![\overline{K}]\!]\ \texttt{SortedList}[\![P]\!] \text{ + bounds } \left\langle \langle C_1, \texttt{Bottom}\rangle \right\rangle \text{ and } \left\langle \langle \texttt{Any}, \texttt{Any}\rangle \right\rangle \qquad \Psi \vdash \texttt{Cons}[\![C_2]\!] \text{ match } [\![\overline{K}]\!]\ \texttt{SortedList}[\![Q]\!] \text{ + bounds } \langle\rangle \text{ and } \langle\rangle}{\Psi \vdash (\texttt{SortedCons}[\![C_1]\!], \texttt{Cons}[\![C_2]\!]) \text{ match } [\![\overline{K}]\!]\ (\texttt{SortedList}[\![P]\!], \texttt{SortedList}[\![Q]\!]) \text{ + bounds } \langle\rangle \text{ and } \langle\rangle}$$

where $\overline{K} = \{\texttt{Bottom}\} <: P <: \{\texttt{Any}\}, \{P\} <: Q <: \{\texttt{Any}\}$. While the first component matching succeeds, the second component matching fails since $\texttt{Cons}[\![C_2]\!]$ is not a subtype of $\texttt{SortedList}[\![Q]\!]$. Although matching the static return type against the return type of d_1 succeeds, the combined type bounds for the solving step are the empty lists, which leads to the failure of the dispatch semipredicate:

$$\frac{\begin{array}{c}\mathit{arrow}(d_1) = \forall[\![\overline{K}]\!]((\texttt{SortedList}[\![P]\!], \texttt{SortedList}[\![Q]\!]) \to \texttt{SortedList}[\![Q]\!]) \\ \Psi \vdash (\texttt{SortedCons}[\![C_1]\!], \texttt{Cons}[\![C_2]\!]) \text{ match } [\![\overline{K}]\!]\ (\texttt{SortedList}[\![P]\!], \texttt{SortedList}[\![Q]\!]) \text{ + bounds } \langle\rangle \text{ and } \langle\rangle \\ \Psi \vdash \texttt{List}[\![C_3]\!] \text{ match } [\![\overline{K}]\!]\ \texttt{SortedList}[\![Q]\!] \text{ - bounds } \left\langle \langle \texttt{Bottom}, \texttt{Bottom}\rangle \right\rangle \text{ and } \left\langle \langle \texttt{Any}, C_3\rangle \right\rangle \\ \Psi \vdash \text{solve } [\![\overline{K}]\!] \text{ bounds } \langle\rangle \text{ and } \langle\rangle \text{ using } \times\end{array}}{\Psi \vdash (\texttt{SortedCons}[\![C_1]\!], \texttt{Cons}[\![C_2]\!]) \text{ and } \texttt{List}[\![C_3]\!] \text{ dispatch } d_1 \text{ using } \times}$$

Dynamic overloading resolution then moves on to the next method declaration in the list, d_2, and performs the dispatch semipredicate on it. In this case, both type matching steps succeed, and the overall result of the dispatch semipredicate depends on the result of the solving step:

$$\frac{\begin{array}{c}\mathit{arrow}(d_2) = \forall[\![\overline{K}]\!]((\texttt{List}[\![P]\!], \texttt{List}[\![Q]\!]) \to \texttt{List}[\![Q]\!]) \\ \Psi \vdash (\texttt{SortedCons}[\![C_1]\!], \texttt{Cons}[\![C_2]\!]) \text{ match } [\![\overline{K}]\!]\ (\texttt{List}[\![P]\!], \texttt{List}[\![Q]\!]) \text{ + bounds } \left\langle \langle C_1, C_2\rangle \right\rangle \text{ and } \left\langle \langle \texttt{Any}, \texttt{Any}\rangle \right\rangle \\ \Psi \vdash \texttt{List}[\![C_3]\!] \text{ match } [\![\overline{K}]\!]\ \texttt{List}[\![Q]\!] \text{ - bounds } \left\langle \langle \texttt{Bottom}, \texttt{Bottom}\rangle \right\rangle \text{ and } \left\langle \langle \texttt{Any}, C_3\rangle \right\rangle \\ \Psi \vdash \text{solve } [\![\overline{K}]\!] \text{ bounds } \left\langle \langle C_1, C_2\rangle \right\rangle \text{ and } \left\langle \langle \texttt{Any}, C_3\rangle \right\rangle \text{ using } \sigma\end{array}}{\Psi \vdash (\texttt{SortedCons}[\![C_1]\!], \texttt{Cons}[\![C_2]\!]) \text{ and } \texttt{List}[\![C_3]\!] \text{ dispatch } d_2 \text{ using } \sigma}$$

We start solving the type bounds from the rightmost type parameter Q. Since we have a single pair of type bounds, that is, $\#\left(\left\langle \langle C_1, C_2\rangle \right\rangle\right) = \#\left(\left\langle \langle \texttt{Any}, C_3\rangle \right\rangle\right) = 1$, there is no need for a nondeterministic choice in rule [Solve-Step]. The substitution for Q is the intersection of the computed upper bound and the declared upper bound: $C_3 \sqcap \texttt{Any} = C_3$. This choice of substitution C_3 for Q has to be propagated by matching it against the declared lower bound of Q, which is P, to make sure that C_3

is a supertype of P. We then combine the type bounds obtained from propagation and the originally given type bounds, and inductively solve the combined bounds to get the substitution for P, which results in $[C_3/P]$ by again taking the intersection of the upper bounds: $C_3 \sqcap \mathtt{Any} = C_3$. Thus, the solving step succeeds with a valid substitution $[C_3/P, C_3/Q]$:

$$\frac{\Psi \vdash C_2 <: C_3 \qquad \Psi \vdash C_3 \text{ match } [\![\mathtt{Bottom} <: P <: \mathtt{Any}]\!] \; P \text{ - bounds } \langle\langle \mathtt{Bottom} \rangle\rangle \text{ and } \langle\langle C_3 \rangle\rangle \qquad \Psi \vdash \text{solve } [\![\mathtt{Bottom} <: P <: \mathtt{Any}]\!] \text{ bounds } \langle\langle C_1 \rangle\rangle \text{ and } \langle\langle C_3 \rangle\rangle \text{ using } [C_3/P]}{\Psi \vdash \text{solve } [\![\mathtt{Bottom} <: P <: \mathtt{Any}, P <: Q <: \mathtt{Any}]\!] \text{ bounds } \langle\langle C_1, C_2 \rangle\rangle \text{ and } \langle\langle \mathtt{Any}, C_3 \rangle\rangle \text{ using } [C_3/P, C_3/Q]}$$

As a result, the dispatch semipredicate on d_2 succeeds with $[C_3/P, C_3/Q]$, and the method call expression is dispatched to the following instance of d_2:

$$D = \mathit{instantiate}(d_2, [C_3/P, C_3/Q]) = \mathit{append}(x{:}\,\mathtt{List}[\![C_3]\!], y{:}\,\mathtt{List}[\![C_3]\!]){:}\,\mathtt{List}[\![C_3]\!] = _$$

It shows that the dynamic overloading resolution selects an instance of the most specific applicable method declaration without any ambiguity, and the return type of the chosen instance is a subtype of the static return type, which means that the dynamic overloading resolution preserves type.

5.2.5 *Correctness of Dispatch Semipredicate.*

We prove the soundness and completeness of the dispatch semipredicate by showing that each of its steps is sound and complete.

We first show that the type matching step is sound and complete.

The match rules compute sound bounds for method type parameters by matching a ground type g against an internal type α according to a variance mark V. That is, any substitution σ satisfying the bounds makes $\sigma\alpha$ a supertype, a subtype, or an equivalent type of g when V is respectively +, −, or =.

Lemma 5.1. *(Soundness of Type Matching)*
Let $\overline{K = \{\overline{\chi}\} <: P <: \{\overline{\eta}\}}$, $\Psi \vdash [\![\overline{K}]\!]$ ok, $\Psi \vdash g$ ok, $\Psi \cup \{\overline{K}\} \vdash \alpha$ ok, *and* $\sigma = [\overline{g'/P}]$ *with* $\overline{\Psi \vdash g' \text{ ok}}$.

If $\Psi \vdash g$ match $[\![\overline{K}]\!]\ \alpha\ V$ bounds $\langle \overline{\langle \overline{l} \rangle} \rangle$ and $\langle \overline{\langle \overline{u} \rangle} \rangle$,
$\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{l_i} \rangle$ and $\langle \overline{u_i} \rangle$ *for some* $1 \le i \le \#(\overline{\langle \overline{l} \rangle}) = \#(\overline{\langle \overline{u} \rangle})$, *and*
$\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{\bigsqcup\{\overline{\chi}\}} \rangle$ and $\langle \overline{\bigsqcap\{\overline{\eta}\}} \rangle$,
then $\Psi \vdash g$ and $\sigma\alpha$ compare V.

Proof. See Appendix D. □

Suppose an internal type α do not contain `Bottom` and any union types and intersection types it contains do not contain any type parameters. For a ground type g and a variance mark V, if a substitution σ on $\langle \overline{P} \rangle$ makes $\sigma\alpha$ a supertype, a subtype, or an equivalent type of g when V is respectively +, −, or =, then the match rules find bounds of $\langle \overline{P} \rangle$ that σ satisfies.

Lemma 5.2. *(Completeness of Type Matching)*
Let $\overline{K = \{\overline{\chi}\} <: P <: \{\overline{\eta}\}}$, $\Psi \vdash [\![\overline{K}]\!]$ ok, $\Psi \vdash g$ ok, $\Psi \cup \{\overline{K}\} \vdash \alpha$ ok, *and* $\sigma = [\overline{g'/P}]$ *with* $\overline{\Psi \vdash g' \text{ ok}}$. *Suppose for all* $\mathcal{T}$ *and* $\mathcal{T}'$,

$$\alpha \neq \mathcal{T}\boxed{\mathtt{Bottom}}, \overline{(\underline{\alpha} = \mathcal{T}\boxed{\textstyle\bigsqcup\langle\overline{\gamma}\rangle}) \Rightarrow \overline{(\gamma \neq \mathcal{T}'\boxed{\underline{P}})}}, \text{ and } \overline{(\underline{\alpha} = \mathcal{T}\boxed{\textstyle\bigsqcap\langle\overline{\gamma}\rangle}) \Rightarrow \overline{(\gamma \neq \mathcal{T}'\boxed{\underline{P}})}}.$$

If $\Psi \vdash g$ and $\sigma\alpha$ compare V *and* $\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{\bigsqcup\{\overline{\chi}\}} \rangle$ and $\langle \overline{\bigsqcap\{\overline{\eta}\}} \rangle$,
then $\Psi \vdash g$ match $[\![\overline{K}]\!]\ \alpha\ V$ bounds $\langle \overline{\langle \overline{l} \rangle} \rangle$ and $\langle \overline{\langle \overline{u} \rangle} \rangle$ *and*
$\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{l_i} \rangle$ and $\langle \overline{u_i} \rangle$ *for some* $1 \le i \le \#(\overline{\langle \overline{l} \rangle}) = \#(\overline{\langle \overline{u} \rangle})$.

Proof. See Appendix D. □

A subtle point in Lemma 5.2 is the assumptions on type α. While the syntax of the match rules allows any internal types, each match rule is used only with α coming from method declarations. The explicitly annotated types in method declarations are external types, which do not contain `Bottom`, union types, nor intersection types. On the contrary, inferred types for method type parameters may contain such internal types. For instance, in the following example (in which type annotations appear for the two method calls):

$$\begin{array}{ll} \texttt{object}\ O'[\![Q]\!] & m'(x\colon Q)\colon Q = _ \quad \texttt{end} \\ \texttt{object}\ O & m[\![P]\!](x\colon P, y\colon P)\colon P = O'[\![P]\!].m'(x)\colon P \quad \texttt{end} \\ O.m(3, \texttt{true})\colon (\texttt{Int} \sqcup \texttt{Boolean}) & \end{array}$$

the method type parameter P of m is inferred as $(\texttt{Int} \sqcup \texttt{Boolean})$ by the dispatch semipredicate at run time. Since this inferred type then flows into the method declaration of m', the ilk and the static type of method invocation expression of m' are matched against the union type. However, because our type system requires that both static and dynamic type inference algorithms instantiate type parameters with non-`Bottom` types, α does not contain `Bottom`. In addition, when α contains union types or intersection types, all the type parameters in them are already instantiated with ground types.

Next, we show the soundness and completeness of the solving step.

The solve rules compute a valid substitution that satisfies both the declared bounds and computed bounds of method type parameters.

Lemma 5.3. *(Soundness of Solving Bounds)*
Let $\overline{K = \{\overline{\chi}\} <: P <: \{\overline{\eta}\}}$ *and* $\Psi \vdash [\![\overline{K}]\!]$ ok.
If $\Psi \vdash$ solve $[\![\overline{K}]\!]$ bounds $\left\langle \overline{\langle \overline{l} \rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u} \rangle} \right\rangle$ using σ,
then $\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{l_{\underline{i}}} \rangle$ and $\langle \overline{u_{\underline{i}}} \rangle$ *for some* $1 \le i \le \#(\overline{\langle \overline{l} \rangle}) = \#(\overline{\langle \overline{u} \rangle})$ *and*
$\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{\bigsqcup\{\overline{\chi}\}} \rangle$ and $\langle \overline{\bigsqcap\{\overline{\eta}\}} \rangle$.

Proof. See Appendix D. □

If a valid substitution for method type parameters exists, then the solve rules find one.

Lemma 5.4. *(Completeness of Solving Bounds)*
Let $\overline{K = \{\overline{\chi}\} <: P <: \{\overline{\eta}\}}$, $\Psi \vdash [\![\overline{K}]\!]$ ok, *and* $\sigma = [\overline{g/P}]$ *with* $\overline{\underline{\Psi} \vdash g \text{ ok}}$.
If $\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{\bigsqcup\{\overline{\chi}\}} \rangle$ and $\langle \overline{\bigsqcap\{\overline{\eta}\}} \rangle$ *and* $\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{l} \rangle$ and $\langle \overline{u} \rangle$
for some $\overline{l}$ *and* $\overline{u}$,
then $\Psi \vdash$ solve $[\![\overline{K}]\!]$ bounds $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle$ using σ' *for some* σ'
where $\langle \overline{l} \rangle = \langle \overline{l'_{\underline{i}}} \rangle$ *and* $\langle \overline{u} \rangle = \langle \overline{u'_{\underline{i}}} \rangle$ *for some* $1 \le i \le \#(\overline{\langle \overline{l'} \rangle}) = \#(\overline{\langle \overline{u'} \rangle})$.

Proof. See Appendix D. □

Finally, we prove that the dispatch semipredicate is sound and complete.

If the dispatch semipredicate on a method declaration d for an ilk k and a static return type g succeeds, then d is dispatchable to k and g.

Theorem 5.5. *(Soundness of Dispatch Semipredicate)*
Let $\Psi \vdash k$ ok, $\Psi \vdash g$ ok, $\overline{K = \{\overline{\chi}\} <: P <: \{\overline{\eta}\}}$, $\Psi \vdash [\![\overline{K}]\!]$ ok, $arrow(d) = \forall [\![\overline{K}]\!](\alpha \to \rho)$, $\Psi \cup \{\overline{K}\} \vdash \alpha$ ok, *and* $\Psi \cup \{\overline{K}\} \vdash \rho$ ok.
If $\Psi \vdash k$ and g dispatch d using σ,
then $\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{\bigsqcup\{\overline{\chi}\}} \rangle$ and $\langle \overline{\bigsqcap\{\overline{\eta}\}} \rangle$, $\Psi \vdash k <: \sigma\alpha$, *and* $\Psi \vdash \sigma\rho <: g$.

Proof. See Appendix D. □

If a method declaration d is dispatchable to an ilk k and a static return type g, then the dispatch semipredicate on d for k and g succeeds.

Theorem 5.6. *(Completeness of Dispatch Semipredicate)*
Let $\overline{K = \{\overline{\chi}\} <: P <: \{\overline{\eta}\}}$, $\Psi \vdash [\![\overline{K}]\!]$ ok, $\sigma = [\overline{g'/P}]$ *with* $\overline{\underline{\Psi} \vdash g' \text{ ok}}$, $\Psi \vdash k$ ok, $\Psi \vdash g$ ok, $\mathit{arrow}(d) = \forall[\![\overline{K}]\!](\alpha \to \rho)$, $\Psi \cup \{\overline{K}\} \vdash \alpha$ ok, *and* $\Psi \cup \{\overline{K}\} \vdash \rho$ ok.
If $\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{\bigsqcup\{\overline{\chi}\}} \rangle$ and $\langle \overline{\bigsqcap\{\overline{\eta}\}} \rangle$, $\Psi \vdash k <: \sigma\alpha$, *and* $\Psi \vdash \sigma\rho <: g$,
then $\Psi \vdash k$ and g dispatch d using σ' *for some* σ'.

Proof. See Appendix D. □

5.3 Type Soundness of FGFV

We now prove the type soundness of FGFV. First, we show that for any applicable method declaration instance for a given method invocation, we can find an instance of the most specific method declaration whose return type is a subtype of the return type of the applicable instance.

Lemma 5.7. *(Existence of a Type-safe Most Specific Method Instance)*
Let $\{\overline{d'_0}\} = \mathit{visible}(\Delta, (C, m))$ *for* $\Delta \vdash C$ ok, *and* $(d_0, \{\overline{\sigma_0}\}) \in \mathit{applicable}(\Delta, \{\overline{d'_0}\}, \alpha)$ *and* $(d, \{\overline{\sigma}\}) \in \mathit{mostSpecific}(\Delta, \mathit{applicable}(\Delta, \{\overline{d'_0}\}, \alpha))$ *for* $\Delta \vdash \alpha$ ok *and* $\alpha \neq \texttt{Bottom}$.
If $D_0 \in \left\{\overline{\mathit{instantiate}(\underline{d_0}, \sigma_0)}\right\}$ *with* $\mathit{arrow}(D_0) = \forall[\![\,]\!](_ \to \rho_0)$,
then $\exists\, D \in \left\{\overline{\mathit{instantiate}(\underline{d}, \sigma)}\right\}$ *with* $\mathit{arrow}(D) = \forall[\![\,]\!](_ \to \rho)$ *such that* $\Delta \vdash \rho <: \rho_0$.

Proof. See Appendix D. □

Next, we show that no matter which applicable method instance we choose for a method invocation expression at compile time, there exists a type-safe most specific method instance at run time.

Lemma 5.8. *(Existence of a Type-safe Most Specific Method Instance at Run Time)*
Let $\{\overline{d'_0}\} = \mathit{visible}(\Delta, (C, m))$ *and* $\{\overline{d'}\} = \mathit{visible}(\Psi, (G, m))$ *for* $\Delta \vdash C$ ok, $\Psi \vdash G$ ok, *and* $\Delta = \Psi \cup \{\overline{\{\overline{\chi}\} <: P <: \{\overline{\eta}\}}\}$. *Also, let* $(d_0, \{\overline{\sigma_0}\}) \in \mathit{applicable}(\Delta, \{\overline{d'_0}\}, \alpha)$ *for* $\Delta \vdash \alpha$ ok *and* $\alpha \neq \texttt{Bottom}$, *and* $(d, \{\overline{\sigma}\}) \in \mathit{mostSpecific}(\Psi, \mathit{applicable}(\Psi, \{\overline{d'}\}, k))$ *for* $\Psi \vdash k$ ok *with* $\Psi \vdash G <: \sigma C$ *and* $\Psi \vdash k <: \sigma\alpha$ *for* $\sigma = [\overline{g'/P}]$ *with* $\Psi \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{\bigsqcup\{\overline{\chi}\}} \rangle$ and $\langle \overline{\bigsqcap\{\overline{\eta}\}} \rangle$ *and* $\overline{\underline{\Psi} \vdash g' \text{ ok}}$.
If $D_0 \in \left\{\overline{\mathit{instantiate}(\underline{d_0}, \sigma_0)}\right\}$ *with* $\mathit{arrow}(D_0) = \forall[\![\,]\!](_ \to \rho_0)$
then $\exists\, D \in \left\{\overline{\mathit{instantiate}(\underline{d}, \sigma)}\right\}$ *with* $\mathit{arrow}(D) = \forall[\![\,]\!](_ \to h)$ *such that* $\Psi \vdash h <: \sigma\rho_0$.

Proof. See Appendix D. □

We now show that if a type-safe most specific method instance exists, the *firstDispatchable* rule finds one. This lemma follows from the soundness and completeness of the dispatch semipredicate.

Lemma 5.9. *(Validity of the firstDispatchable Rule)*
Let $\{\overline{d'}\} = \mathit{visible}(\Psi, (G, m))$ *for* $\Psi \vdash G$ ok.
If $(d, \{\overline{\sigma}\}) \in \mathit{mostSpecific}(\Psi, \mathit{applicable}(\Psi, \{\overline{d'}\}, k))$ *for* $\Psi \vdash k$ ok *and* $D \in \left\{\overline{\mathit{instantiate}(\underline{d}, \sigma)}\right\}$
with $\mathit{arrow}(D) = \forall[\![\,]\!](_ \to h)$ *and* $\Psi \vdash h <: g$ *for* $\Psi \vdash g$ ok,
then $\mathit{firstDispatchable}(\Psi, \mathit{topologicalSort}(\Psi, \{\overline{d'}\}), k, g) = D_1$
with $D_1 \in \left\{\overline{\mathit{instantiate}(\underline{d}, \sigma)}\right\}$, $\mathit{arrow}(D_1) = \forall[\![\,]\!](h_1 \to h_2)$, $\Psi \vdash k <: h_1$, *and* $\Psi \vdash h_2 <: g$.

Proof. See Appendix D. □

Finally, we prove the type soundness of FGFV. Since the main focus of this paper is resolution of overloaded method invocation and the other cases are conventional, we prove only the method

invocation case: a well-typed method invocation can be reduced via [R-Method] and the reduced expression preserves the type of the method invocation.

Theorem 5.10. *(Type Soundness of Method Invocation Expression)*
Let $\langle \overline{d} \rangle = \mathit{topologicalSort}(\Psi, \mathit{visible}(\Psi, (O[\![\overline{g''}]\!], m)))$ *for* `object` $O[\![__]\!](\overline{x\colon _})$ `__ end` $\in \Psi$ *and* $\Psi \vdash O[\![\overline{g''}]\!]$ ok.
If $\Psi; \bullet \vdash O[\![\overline{g''}]\!](\overline{v}).m(\overline{v'}) : (_, g)$,
then $(_, m[\![\,]\!](\overline{x'\colon _})\colon _ = \epsilon) = \mathit{firstDispatchable}(\Psi, \langle \overline{d} \rangle, (O[\![\overline{g''}]\!], \overline{\mathit{ilk}(v')}), g)$
and $\Psi; \bullet \vdash [O[\![\overline{g''}]\!](\overline{v})/\mathtt{self}][\overline{v/x}][\overline{v'/x'}]\epsilon : (_, g')$ *with* $\Psi \vdash g' <: g$.

Proof. See Appendix D. □

Theorem 5.11. *(Type Soundness of FGFV)*
If $\Psi; \bullet \vdash e : (\epsilon, g)$ *and* $\Psi \vdash \epsilon \longrightarrow^* v$, *then* $\Psi; \bullet \vdash v : (_, g')$ *and* $\Psi \vdash g' <: g$.

Proof. See Appendix D. □

5.4 Discussion

Defining a provable system for an object-oriented language requires addressing various technical challenges due to the structural asymmetries inherent in the language design. One can have a variable of type `Any`, but one cannot have a variable of type `Bottom`. The ilk of an actual argument must be a subtype, not a supertype, of the method's parameter type. The ilk of a returned value must be a subtype, not a supertype, of the method's return type. A trait may declare ancestors but not descendants. Such asymmetries prohibit seemingly symmetric features from being treated as duals. One might think at first glance that upper bounds and lower bounds of type parameters should be related symmetrically (perhaps treated as duals), but they cannot be.

For example, the "Ancestor Meet Rule" (expressed in this paper by rule [Anc-Same-Trait]) was introduced into Fortress in 2012 by Guy Steele and David Chase [Steele and Chase 2012] to design an expressive type system with union and intersection types. This rule permits proofs that (a) if $T[\![P]\!]$ is covariant in P then $T[\![(\alpha \sqcap \alpha')]\!] \equiv (T[\![\alpha]\!] \sqcap T[\![\alpha']\!])$, and (b) if $T[\![P]\!]$ is contravariant in P then $T[\![(\alpha \sqcup \alpha')]\!] \equiv (T[\![\alpha]\!] \sqcap T[\![\alpha']\!])$ (that last set operator is indeed $\sqcap$, not $\sqcup$). These equivalences allow simplification of type expressions by moving type union and type intersection operators out of type arguments. However, the "obvious other two cases" are not available. When $T[\![P]\!]$ is covariant in P, $T[\![(\alpha \sqcup \alpha')]\!]$ is not equivalent to some simple combination of $T[\![\alpha]\!]$ and $T[\![\alpha']\!]$; the best one can say is that $(T[\![\alpha]\!] \sqcup T[\![\alpha']\!]) <: T[\![(\alpha \sqcup \alpha')]\!]$.

In 2012, the developers of Fortress explored an alternate set of language restrictions that exploited these asymmetries to ensure that exponential blowup could occur at run time only when union types interacted with contravariance (the dual case of intersection types interacting with covariance could not occur). They then contemplated two alternatives for avoiding the exponential blowup. One possibility is to eliminate contravariant type parameters and contravariant type relationships. This has the consequence that arrow types must be treated as invariant, rather than contravariant, in the type to the left of the arrow. This could make it inconvenient to use higher-order functions such as *map*, but one can compensate by introducing rules of coercion, so that if a function of type $(\alpha \to \rho)$ is used in a situation where a function of type $(\alpha' \to \rho')$ is required, it can be coerced to a new function object if $\alpha' <: \alpha$ and $\rho <: \rho'$. The other possibility is to eliminate union types from the type theory; this might require restricting the language so that, for example, in an `if-then-else` expression, the type of the `then` subexpression must be either a supertype or a subtype of the `else` subexpression. Alternatively, some sort of pseudo-join (least common existing supertype) operation could be used. One could go even further, also eliminating intersection types from the type theory by further restricting the language and possibly using a pseudo-meet operation.

We carefully designed our dynamic dispatch algorithm to make a balanced tradeoff between static expressivity and dynamic efficiency. We accept the potential for exponential blowup because we believe that such cases are unlikely to arise in practice and the use of union and intersection types in the type theory, even if not directly expressible by the programmer, makes the language more expressive. Also, while one could in principle allow any lower bound to refer to any type parameter, dynamic overloading resolution would then have to solve a general set of type inequalities, requiring a complex technique with multiple iterations; we chose instead to impose left-to-right scoping on method type parameters, allowing use of a simple, one-pass right-to-left scan. We believe this tradeoff represents a relatively small sacrifice of expressivity for a large gain in dynamic efficiency.

Dynamically inferring type instantiations can incur run-time overhead. In simple invariant cases, we expect the run-time cost to be relatively low but nonzero. For example, if we pass a value of type $\texttt{List}[\![\texttt{String}]\!]$ to a method such as $foo[\![T]\!](x{:}\texttt{List}[\![T]\!])$, T is matched only once and no union or intersection types are involved. If `List` is invariant in T, it will come down to extracting the type `String` from the type $\texttt{List}[\![\texttt{String}]\!]$ and binding T to that. For more costly covariant and contravariant cases, as in certain JVM implementations, such techniques as caching or dynamic recompilation in reaction to the relative frequencies of types actually encountered may help.

The biggest challenge we faced in trying to find a provable system was grappling with the language asymmetries described above and trying to understand their consequences. In particular, the Fortress team had assumed, because it seemed from several working examples to be in accord with intuition, that once the tightest possible upper and lower bounds for a method type parameter have been found, then the union (or pseudo-union) of the lower bounds should be chosen as the type to be used for that type parameter. We considered this and the hybrid technique of using the intersection of upper bounds only when the union of lower bounds is `Bottom`, but we were unable to complete the proofs of correctness of both the techniques. Since the minimum requirement for a type to instantiate a method type parameter is that it satisfies all the lower and upper bounds of the type parameter, the dynamic dispatch algorithm presented here always chooses the intersection of the upper bounds.

In the work presented here, as in previous work, one of the most difficult parts is finding the necessary substitution σ to satisfy the rule [E-Sub] for existential subtyping during static analysis. This is more difficult than the corresponding calculation during dynamic dispatch because at run time one always matches an ilk (which contains no type parameters) to a type, but at compile time one must match a type against a type, where each type may contain type parameters. Indeed, we do not present a specific static type inference algorithm that finds the substitution σ in rule [E-Sub], but only refer to previous work in this area. However, we conjecture that the match rule presented here can serve this role after being extended to handle the case of matching two type parameters.

6 RELATED WORK

This paper extends the work of Allen et al. [2011] with variance for polymorphic types and a dynamic dispatch algorithm. Allen et al. proposed rules for defining overloaded functions that ensure type safety under symmetric multiple dispatch in an object-oriented language with multiple inheritance, and showed how to check the rules without requiring the entire type hierarchy. They proposed the overloading rules in the context of a set of class declarations and function declarations without defining a language with complete static and dynamic semantics. Therefore, their work does not guarantee that the static overloading rules and the way a language chooses a method to invoke at run time match correctly. While FGFV does not support features like modularity, nominal exclusion, and closed types in their work, they can be added in an obvious way. As the related work section of Allen et al. [2011] discusses in detail, most earlier systems like *Dubious* [Millstein and Chambers 1999, 2002; Millstein 2003], MultiJava [Clifton et al. 2006], and F(Eml) [Lee and Chambers

2006] supported overloading with symmetric multiple dynamic dispatch with multiple inheritance, but without support for polymorphic methods or types. While $ML_{\leq}$ [Bourdoncle and Merz 1997] integrates polymorphism and symmetric multiple dynamic dispatch, it lacks multiple inheritance.

ParaSail [Taft 2011, 2016] is a parallel programming language designed to support inherently safe and secure, highly parallel applications. It also constrains the scope of type parameters, prohibiting a bound on a type parameter from referring to type parameter bindings to its left, so that dynamic type inference can be fast. However, ParaSail does not support covariant and contravariant type parameters, and it does not provide any formal specification.

Julia [Bezanson et al. 2015, 2017] is a dynamic programming language with symmetric multiple dispatch. It provides a type declaration and type annotation sublanguage, but does not have a static type system. Thus, it does not check the validity of overloaded methods, and only uses the subtype relation to find the most specific method among overloaded methods at run time.

Various languages provide different subsets of the FGFV features. Scala [Odersky et al. 2016] supports polymorphic types with variance, but it does not support symmetric multiple dispatch. FHJ [Wang et al. 2018] proposed a dispatch algorithm, which uses static types at run time to resolve overloading conflicts from unrelated methods that happen to have the same name and signature due to multiple inheritance. Castagna et al. [2015] presented a statically-typed calculus with dynamic (explicit) dispatch through pattern-matching, polymorphism, recursive types, semantic subtyping, and union, intersection, and negation type connectives. Their local type inference algorithm computes sets of type substitutions to insert in appropriate places in expressions. While their type-case expression and intersection type behave similarly to method overloading, they do not have a similar notion of overloading rules. Since the type-case expression executes the first matching expression, it does not introduce any ambiguity. For OCaml, Garrigue [2001; 2002] introduced a type inference system for structural polymorphism such as polymorphic records, polymorphic variants, and accurate matching. Type constraints are described by constraint domains that are defined separately from its type system, and the constraints contain some notion of lower and upper bounds, which respectively represent required and available labels or cases. He also extended his work with CoQ mechanization [Garrigue 2010, 2015]. The work of Castagna and Garrigue may serve as a possible starting point for finding the necessary substitution σ to satisfy the rule [E-Sub] for existential subtyping during static analysis. We believe that our work opens the gate to adding either variance annotation in the work of Castagna or a form of type-case in the work of Garrigue.

7 CONCLUSION

We defined FGFV, a core calculus for an object-oriented language with symmetric multiple dispatch, multiple inheritance, and parametric polymorphism with variance. In order to ensure type soundness of FGFV, it must guarantee no ambiguous method invocations at run time, and preserve type even when a method invocation is dispatched to a different method at run time from the method chosen at compile time. Thus, we formalized overloading rules using existential types and universal types quantified over method type parameters to mechanically check whether overloaded polymorphic methods are valid. As a result, in a well-formed program that satisfies the overloading rules, there always exists the single most specific method to invoke with respect to both static argument types and dynamic argument types. Further, with the static types of method invocation expressions delivered to run time, the dynamic semantics of FGFV that uses the new dynamic dispatch algorithm we presented is proven to make type-sound choices among applicable method declarations.

REFERENCES

Eric Allen, David Chase, J. J. Hallett, Victor Luchangco, Jan-Willem Maessen, Sukyoung Ryu, Guy L. Steele Jr., and Sam Tobin-Hochstadt. 2008. The Fortress Language Specification Version 1.0. (March 2008).

Eric Allen, J. J. Hallett, Victor Luchangco, Sukyoung Ryu, and Guy L. Steele Jr. 2007. Modular Multiple Dispatch with Multiple Inheritance. In *Proceedings of the 2007 ACM Symposium on Applied Computing (SAC '07)*. ACM, New York, NY, USA, 1117–1121. https://doi.org/10.1145/1244002.1244245

Eric Allen, Justin Hilburn, Scott Kilpatrick, Victor Luchangco, Sukyoung Ryu, David Chase, and Guy L. Steele Jr. 2011. Type-checking Modular Multiple Dispatch with Parametric Polymorphism and Multiple Inheritance. In *Proceedings of the ACM International Conference on Object Oriented Programming, Systems, Languages, and Applications (OOPSLA '11)*. ACM, New York, NY, USA, 973–992. https://doi.org/10.1145/2048066.2048140

Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. 2015. Julia: A Fresh Approach to Numerical Computing (v4). *CoRR* abs/1411.1607 (July 2015), 1–37. http://arxiv.org/abs/1411.1607

Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98. https://doi.org/10.1137/141000671

François Bourdoncle and Stephan Merz. 1997. Type Checking Higher-Order Polymorphic Multi-methods. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*. ACM, New York, NY, USA, 302–315. https://doi.org/10.1145/263699.263743

Giuseppe Castagna, Giorgio Ghelli, and Giuseppe Longo. 1992. A Calculus for Overloaded Functions with Subtyping. In *Proceedings of the 1992 ACM Conference on LISP and Functional Programming (LFP '92)*. ACM, New York, NY, USA, 182–192. https://doi.org/10.1145/141471.141537

Giuseppe Castagna, Giorgio Ghelli, and Giuseppe Longo. 1995. A Calculus for Overloaded Functions with Subtyping. *Information and Computation* 117, 1 (1995), 115–135. https://doi.org/10.1006/inco.1995.1033

Giuseppe Castagna, Kim Nguyen, Zhiwu Xu, and Pietro Abate. 2015. Polymorphic Functions with Set-Theoretic Types: Part 2: Local Type Inference and Type Reconstruction. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '15)*. ACM, New York, NY, USA, 289–302. https://doi.org/10.1145/2676726.2676991

Curtis Clifton, Gary T. Leavens, Craig Chambers, and Todd Millstein. 2000. MultiJava: Modular Open Classes and Symmetric Multiple Dispatch for Java. In *Proceedings of the 15th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '00)*. ACM, New York, NY, USA, 130–145. https://doi.org/10.1145/353171.353181

Curtis Clifton, Todd D. Millstein, Gary T. Leavens, and Craig Chambers. 2006. MultiJava: Design Rationale, Compiler Implementation, and Applications. *ACM Transactions on Programming Languages and Systems* 28, 3 (May 2006), 517–575. https://doi.org/10.1145/1133651.1133655

Devin Coughlin and Bor-Yuh Evan Chang. 2014. Fissile Type Analysis: Modular Checking of Almost Everywhere Invariants. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '14)*. Association for Computing Machinery, New York, 73–85. https://doi.org/10.1145/2535838.2535855

EPFL. 2017. A class for immutable linked lists representing ordered collections of elements of type 'A' [implementation of class `List` for the Scala programming language]. https://github.com/scala/scala/blob/2.13.x/src/library/scala/collection/immutable/List.scala. (2017). Accessed: 2017-11-27.

Jacques Garrigue. 2001. Simple Type Inference for Structural Polymorphism. In *Proceedings of the Second Asian Workshop on Programming Languages and Systems (APLAS'01)*. 329–343. Also presented as [Garrigue 2002].

Jacques Garrigue. 2002. Simple Type Inference for Structural Polymorphism. In *Ninth International Workshop on Foundations of Object-Oriented Languages*. 1–11. http://www.cs.cmu.edu/ Also presented as [Garrigue 2001].

Jacques Garrigue. 2010. A Certified Implementation of ML with Structural Polymorphism. In *Programming Languages and Systems*, Kazunori Ueda (Ed.). Springer Berlin Heidelberg, 360–375. LNCS 6461. Proceedings of the Eighth Asian Symposium on Programming Languages and Systems (APLAS 2010), Shanghai, China. More complete journal version is [Garrigue 2015].

Jacques Garrigue. 2015. A Certified Implementation of ML with Structural Polymorphism and Recursive Types. *Mathematical Structures in Computer Science* 25, 4 (May 2015), 867–891. https://doi.org/10.1017/S0960129513000066 Special issue: selected papers from the Eighth Asian Symposium on Programming Languages and Systems (APLAS 2010), Shanghai, China. Extended version of [Garrigue 2010].

Atsushi Igarashi and Mirko Viroli. 2006. Variant Parametric Types: A Flexible Subtyping Scheme for Generics. *ACM Transactions on Programming Languages and Systems* 28, 5 (September 2006), 795–847. https://doi.org/10.1145/1152649.1152650

Jieung Kim and Sukyoung Ryu. 2011. Coq Mechanization of Featherweight Fortress with Multiple Dispatch and Multiple Inheritance. In *Certified Programs and Proofs (CPP '11)*, Jean-Pierre Jouannaud and Zhong Shao (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 264–279.

Keunwoo Lee and Craig Chambers. 2006. Parameterized Modules for Classes and Extensible Functions. In *ECOOP 2006 – Object-Oriented Programming*, Dave Thomas (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 353–378.

Todd Millstein and Craig Chambers. 1999. Modular Statically Typed Multimethods. In *ECOOP' 99 — Object-Oriented Programming*, Rachid Guerraoui (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 279–303.

Todd Millstein and Craig Chambers. 2002. Modular Statically Typed Multimethods. *Information and Computation* 175, 1 (2002), 76 – 118. https://doi.org/10.1006/inco.2002.3103

Todd David Millstein. 2003. *Reconciling Software Extensibility with Modular Program Reasoning*. Ph.D. Dissertation. University of Washington.

Martin Odersky, Lex Spoon, and Bill Venners. 2016. *Programming in Scala* (3rd ed.). Artima Press, Walnut Creek, California, USA. ISBN 978-0981531687.

Guy Steele and David Chase. 2012. Personal communication. (2012).

Guy L. Steele Jr. 2017. It's Time for a New Old Language. Published on YouTube. (May 2017). https://youtu.be/7HKbjYqqPPQ Video (TRT 1:12:11) of invited keynote talk at PPoPP '17: 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, Austin, Texas, February 6, 2017.

S. Tucker Taft. 2011. Multicore Programming in ParaSail: Parallel Specification and Implementation Language. In *Proceedings of the 16th Ada-Europe International Conference on Reliable Software Technologies (Ada-Europe'11)*. Springer-Verlag, Berlin, Heidelberg, 196–200. http://dl.acm.org/citation.cfm?id=2018027.2018049

S. Tucker Taft. 2016. Designing ParaSail, a new programming language (blog). http://parasail-programming-language.blogspot.com. (2016). Accessed: 2018-04-09.

Dimitrios Vytiniotis, Simon Peyton Jones, Koen Claessen, and Dan Rosén. 2013. HALO: Haskell to Logic through Denotational Semantics. In *Proceedings of the 40th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '13)*. Association for Computing Machinery, New York, 431–442. https://doi.org/10.1145/2429069.2429121

Yanlin Wang, Haoyuan Zhang, Bruno C. d. S. Oliveira, and Marco Servetto. 2018. FHJ: A Formal Model for Hierarchical Dispatching and Overriding. In *32nd European Conference on Object-Oriented Programming, ECOOP 2018, July 16-21, 2018, Amsterdam, The Netherlands*. 20:1–20:30.

Andrew K. Wright and Matthias Felleisen. 1994. A Syntactic Approach to Type Soundness. *Information and Computation* 115, 1 (Nov. 1994), 38–94. https://doi.org/10.1006/inco.1994.1093

APPENDIX A CONDITIONAL TYPE RELATIONS

Conditional subtyping: $\boxed{\Delta \vdash \alpha <: \alpha \Leftarrow C}$

$$\Delta \vdash P <: P \Leftarrow \text{true} \qquad \text{[Sub-Params-Same]}$$

$$\frac{P \in \mathit{parameters}(\Delta) \qquad \{\overline{\chi}\} <: Q <: _ \in \Delta \qquad \mathit{distinct}(P, Q) \qquad \overline{\underline{\Delta} \vdash \underline{P} <: \chi \Leftarrow C}}{\Delta \vdash P <: Q \Leftarrow \bigvee \{\overline{C}\}} \qquad \text{[Sub-Params-Different]}$$

α is none of the following: a type parameter, an intersection type, a union type

$$\frac{_ <: P <: \{\overline{\eta}\} \in \Delta \qquad \#(\overline{\eta}) > 0 \qquad \overline{\underline{\Delta} \vdash \eta <: \underline{\alpha} \Leftarrow C}}{\Delta \vdash P <: \alpha \Leftarrow \bigvee \{\overline{C}\}} \qquad \text{[Sub-LParam]}$$

α is none of the following: a type parameter, an intersection type, a union type

$$\frac{_ <: P <: \{\} \in \Delta \qquad \Delta \vdash \texttt{Any} <: \alpha \Leftarrow C}{\Delta \vdash P <: \alpha \Leftarrow C} \qquad \text{[Sub-LParam-Any]}$$

α is none of the following: a type parameter, an intersection type, a union type

$$\frac{\{\overline{\chi}\} <: P <: _ \in \Delta \qquad \#(\overline{\chi}) > 0 \qquad \overline{\underline{\Delta} \vdash \underline{\alpha} <: \chi \Leftarrow C}}{\Delta \vdash \alpha <: P \Leftarrow \bigvee \{\overline{C}\}} \qquad \text{[Sub-RParam]}$$

α is none of the following: a type parameter, an intersection type, a union type

$$\frac{\{\} <: P <: _ \in \Delta \qquad \Delta \vdash \alpha <: \texttt{Bottom} \Leftarrow C}{\Delta \vdash \alpha <: P \Leftarrow C} \qquad \text{[Sub-RParam-Bottom]}$$

α is none of the following: an intersection type, a union type

$$\frac{P \notin \mathit{parameters}(\Delta) \qquad (\alpha = Q) \Rightarrow (\mathit{distinct}(P, Q))}{\Delta \vdash P <: \alpha \Leftarrow P <: \alpha} \qquad \text{[Sub-LParam-Infer]}$$

α is none of the following: an intersection type, a union type

$$\frac{P \notin \mathit{parameters}(\Delta) \qquad (\alpha = Q) \Rightarrow (Q \in \mathit{parameters}(\Delta))}{\Delta \vdash \alpha <: P \Leftarrow \alpha <: P} \qquad \text{[Sub-RParam-Infer]}$$

$$\frac{\overline{\underline{\Delta} \vdash \alpha <: \eta \Leftarrow C}}{\Delta \vdash (\overline{\alpha}) <: (\overline{\eta}) \Leftarrow \bigwedge \{\overline{C}\}} \qquad \text{[Sub-Tuple]}$$

$$\frac{\#(\overline{\alpha}) \neq \#(\overline{\eta})}{\Delta \vdash (\overline{\alpha}) <: (\overline{\eta}) \Leftarrow \text{false}} \qquad \text{[Sub-Tuple-Size]}$$

Fig. 16. Subtyping rules that generate constraints (1 of 2)

$$\frac{\Delta \vdash \alpha' <: \alpha \Leftarrow C \qquad \Delta \vdash \rho <: \rho' \Leftarrow C'}{\Delta \vdash (\alpha \to \rho) <: (\alpha' \to \rho') \Leftarrow C \wedge C'} \quad \text{[Sub-Arrow]}$$

$$\frac{\overline{\underline{\Delta} \vdash \alpha \equiv \eta \Leftarrow C}}{\Delta \vdash O[\![\overline{\alpha}]\!] <: O[\![\overline{\eta}]\!] \Leftarrow \bigwedge \{\overline{C}\}} \quad \text{[Sub-Object-Same]}$$

$$\frac{\mathit{distinct}(O, O')}{\Delta \vdash O[\![\overline{\alpha}]\!] <: O'[\![\overline{\eta}]\!] \Leftarrow \text{false}} \quad \text{[Sub-Object-Different]}$$

$$\frac{\texttt{trait}\ T[\![\overline{V\ _ <: _}]\!]\ __\ \text{end} \in \Delta \qquad \overline{\underline{\Delta} \vdash \alpha \text{ and } \eta \text{ compare } V \Leftarrow C}}{\Delta \vdash T[\![\overline{\alpha}]\!] <: T[\![\overline{\eta}]\!] \Leftarrow \bigwedge \{\overline{C}\}} \quad \text{[Sub-Trait]}$$

$$\frac{\{\overline{J}\} = \mathit{parents}(\Delta, O[\![\overline{\alpha}]\!]) \qquad \overline{\underline{\Delta} \vdash J <: \underline{T[\![\overline{\eta}]\!]} \Leftarrow C}}{\Delta \vdash O[\![\overline{\alpha}]\!] <: T[\![\overline{\eta}]\!] \Leftarrow \bigvee \{\overline{C}\}} \quad \text{[Sub-O-Parents]}$$

$$\frac{\mathit{distinct}(T, T') \qquad \{\overline{J}\} = \mathit{parents}(\Delta, T[\![\overline{\alpha}]\!]) \qquad \overline{\underline{\Delta} \vdash J <: \underline{T'[\![\overline{\eta}]\!]} \Leftarrow C}}{\Delta \vdash T[\![\overline{\alpha}]\!] <: T'[\![\overline{\eta}]\!] \Leftarrow \bigvee \{\overline{C}\}} \quad \text{[Sub-T-Parents]}$$

$$\frac{\chi \text{ is not an intersection type} \qquad \Delta \vdash \alpha <: \chi \Leftarrow C \qquad \Delta \vdash \eta <: \chi \Leftarrow C'}{\Delta \vdash (\alpha \sqcup \eta) <: \chi \Leftarrow C \wedge C'} \quad \text{[Sub-LUnion]}$$

$$\frac{\chi \text{ is not an intersection type} \qquad \Delta \vdash \alpha <: \chi \Leftarrow C \qquad \Delta \vdash \eta <: \chi \Leftarrow C' \qquad \Delta \vdash \alpha \Diamond \eta \Leftarrow C''}{\Delta \vdash (\alpha \sqcap \eta) <: \chi \Leftarrow C \vee C' \vee C''} \quad \text{[Sub-LIntersection]}$$

$$\frac{\alpha \text{ is none of the following: an intersection type, a union type} \qquad \Delta \vdash \alpha <: \eta \Leftarrow C \qquad \Delta \vdash \alpha <: \chi \Leftarrow C'}{\Delta \vdash \alpha <: (\eta \sqcup \chi) \Leftarrow C \vee C'} \quad \text{[Sub-RUnion]}$$

$$\frac{\Delta \vdash \alpha <: \eta \Leftarrow C \qquad \Delta \vdash \alpha <: \chi \Leftarrow C'}{\Delta \vdash \alpha <: (\eta \sqcap \chi) \Leftarrow C \wedge C'} \quad \text{[Sub-RIntersection]}$$

Fig. 17. Subtyping rules that generate constraints (2 of 2)

Conditional exclusion: $\boxed{\Delta \vdash \alpha \Diamond \alpha \Leftarrow C}$

$$\Delta \vdash P \Diamond P \Leftarrow \text{false} \qquad \text{[Exc-Refl]}$$

$$\frac{_ <: P <: \{\overline{\eta}\} \in \Delta \qquad \alpha \neq \texttt{Bottom} \qquad (\alpha = Q) \Rightarrow (Q \in \mathit{parameters}(\Delta)) \qquad \overline{\underline{\Delta} \vdash \eta \Diamond \underline{\alpha} \Leftarrow C}}{\Delta \vdash P \Diamond \alpha \Leftarrow \bigvee \{\overline{C}\}} \qquad \text{[Exc-Param]}$$

$$\frac{P \notin \mathit{parameters}(\Delta) \qquad (\alpha = Q) \Rightarrow (\mathit{distinct}(P, Q)) \qquad \alpha \neq \texttt{Bottom}}{\Delta \vdash P \Diamond \alpha \Leftarrow P \Diamond \alpha} \qquad \text{[Exc-Param-Infer]}$$

χ is none of the following: a type parameter, an intersection type

$$\frac{\Delta \vdash \alpha \Diamond \chi \Leftarrow C \qquad \Delta \vdash \eta \Diamond \chi \Leftarrow C'}{\Delta \vdash (\alpha \sqcup \eta) \Diamond \chi \Leftarrow C \wedge C'} \qquad \text{[Exc-Union]}$$

χ is not a type parameter

$$\frac{\Delta \vdash \alpha \Diamond \chi \Leftarrow C \qquad \Delta \vdash \eta \Diamond \chi \Leftarrow C' \qquad \Delta \vdash (\alpha \sqcap \eta) <: \texttt{Bottom} \Leftarrow C''}{\Delta \vdash (\alpha \sqcap \eta) \Diamond \chi \Leftarrow C \vee C' \vee C''} \qquad \text{[Exc-Intersection]}$$

$$\frac{\overline{\underline{\Delta} \vdash \alpha \Diamond \eta \Leftarrow C}}{\Delta \vdash (\overline{\alpha}) \Diamond (\overline{\eta}) \Leftarrow \bigvee \{\overline{C}\}} \qquad \text{[Exc-Tuple]}$$

$$\frac{\#(\overline{\alpha}) \neq \#(\overline{\eta})}{\Delta \vdash (\overline{\alpha}) \Diamond (\overline{\eta}) \Leftarrow \text{true}} \qquad \text{[Exc-Tuple-Size]}$$

$$\frac{\overline{\underline{\Delta} \vdash \alpha \not\equiv \eta \Leftarrow C}}{\Delta \vdash O[\![\overline{\alpha}]\!] \Diamond O[\![\overline{\eta}]\!] \Leftarrow \bigvee \{\overline{C}\}} \qquad \text{[Exc-Object-Same]}$$

$$\frac{\mathit{distinct}(O, O')}{\Delta \vdash O[\![\overline{\alpha}]\!] \Diamond O'[\![\overline{\eta}]\!] \Leftarrow \text{true}} \qquad \text{[Exc-Object-Different]}$$

Fig. 18. Exclusion rules that generate constraints (1 of 2)

$$\frac{\begin{array}{c}\Delta \vdash T[\![\overline{\alpha}]\!] \not<: T'[\![\overline{\eta}]\!] \Leftarrow C \qquad \Delta \vdash T'[\![\overline{\eta}]\!] \not<: T[\![\overline{\alpha}]\!] \Leftarrow C' \\ \{\overline{J}\} = \mathit{ancestors}(\Delta, T[\![\overline{\alpha}]\!]) \qquad \{\overline{J'}\} = \mathit{ancestors}(\Delta, T'[\![\overline{\eta}]\!]) \\ \overline{\Delta \vdash J_i \Diamond_{\mathrm{m}} J'_j \Leftarrow C''_{ij}}^{i,j}\end{array}}{\Delta \vdash T[\![\overline{\alpha}]\!] \Diamond T'[\![\overline{\eta}]\!] \Leftarrow \bigwedge \{C, C', \bigvee \{\overline{\overline{C''}}\}\}} \quad [\textsc{Exc-Trait}]$$

$$\frac{\Delta \vdash O[\![\overline{\alpha}]\!] \not<: T[\![\overline{\eta}]\!] \Leftarrow C}{\Delta \vdash O[\![\overline{\alpha}]\!] \Diamond T[\![\overline{\eta}]\!] \Leftarrow C} \quad [\textsc{Exc-Object-Trait}]$$

Constructability of a subtype of two trait types: $\boxed{\Delta \vdash J \Diamond_{\mathrm{m}} J \Leftarrow C}$

$$\frac{\mathit{distinct}(T, T')}{\Delta \vdash T[\![\overline{\alpha}]\!] \Diamond_{\mathrm{m}} T'[\![\overline{\eta}]\!] \Leftarrow \mathrm{false}}$$

$$\frac{\texttt{trait}\ T[\![\overline{V\ _ <: _}]\!]\ __\ \texttt{end} \in \Delta}{\dfrac{(V = +) \Rightarrow (\Delta \vdash \alpha \Diamond \eta \Leftarrow C) \wedge (V = =) \Rightarrow (\Delta \vdash \alpha \not\equiv \eta \Leftarrow C) \wedge (V = -) \Rightarrow (C = \mathrm{false})}{\Delta \vdash T[\![\overline{\alpha}]\!] \Diamond_{\mathrm{m}} T[\![\overline{\eta}]\!] \Leftarrow \bigvee \{\overline{C}\}}}$$

Fig. 19. Exclusion rules that generate constraints (2 of 2)

APPENDIX B TYPE MATCHING RULES

Matching an argument to a signature: $\boxed{\Psi \vdash g \text{ match } [\![\overline{K}]\!]\ \alpha\ V \text{ bounds } \langle\overline{\langle \bar{l} \rangle}\rangle \text{ and } \langle\overline{\langle \bar{u} \rangle}\rangle}$

$$\frac{\begin{array}{c}\alpha \text{ is none of the following: a type parameter, } \texttt{Any}\text{, an intersection type, a union type} \\ \#(\bar{l}) = \#(\bar{u}) = \#(\overline{K}) \qquad \overline{l = \texttt{Bottom}} \qquad \overline{u = \texttt{Any}} \\ \Psi \cup \{\overline{K}\} \vdash \texttt{Any} \text{ and } \alpha \text{ compare } V\end{array}}{\Psi \vdash \texttt{Any} \text{ match } [\![\overline{K}]\!]\ \alpha\ V \text{ bounds } \langle\langle \bar{l} \rangle\rangle \text{ and } \langle\langle \bar{u} \rangle\rangle}\ \text{[M-Any-Whatever-Success]}$$

$$\frac{\begin{array}{c}\alpha \text{ is none of the following: a type parameter, } \texttt{Any}\text{, an intersection type, a union type} \\ \Psi \cup \{\overline{K}\} \vdash \texttt{Any} \text{ and } \alpha \text{ do not compare } V\end{array}}{\Psi \vdash \texttt{Any} \text{ match } [\![\overline{K}]\!]\ \alpha\ V \text{ bounds } \langle\rangle \text{ and } \langle\rangle}\ \text{[M-Any-Whatever-Failure]}$$

Fig. 20. Matching arguments to signatures (1 of 11: matching Any to a type)

$$\frac{\begin{array}{c}g \text{ is none of the following: an intersection type, a union type} \\ \#(\bar{l}) = \#(\bar{u}) = \#(\overline{K}) \qquad \overline{l = \texttt{Bottom}} \qquad \overline{u = \texttt{Any}} \\ \Psi \vdash g \text{ and } \texttt{Any} \text{ compare } V\end{array}}{\Psi \vdash g \text{ match } [\![\overline{K}]\!]\ \texttt{Any}\ V \text{ bounds } \langle\langle \bar{l} \rangle\rangle \text{ and } \langle\langle \bar{u} \rangle\rangle}\ \text{[M-Whatever-Any-Success]}$$

$$\frac{\begin{array}{c}g \text{ is none of the following: an intersection type, a union type} \\ \Psi \vdash g \text{ and } \texttt{Any} \text{ do not compare } V\end{array}}{\Psi \vdash g \text{ match } _\ \texttt{Any}\ V \text{ bounds } \langle\rangle \text{ and } \langle\rangle}\ \text{[M-Whatever-Any-Failure]}$$

Fig. 21. Matching arguments to signatures (2 of 11: matching a ground type to Any)

$$\frac{\begin{array}{c}g \text{ is none of the following: an intersection type, a union type} \\ 1 \le i \le \#(\overline{P}) \qquad \overline{l = (\text{if } (P = \underline{P_i}) \text{ then } \underline{g} \text{ else } \texttt{Bottom})} \qquad \overline{u = (\text{if } (P = \underline{P_i}) \text{ then } \underline{g} \text{ else } \texttt{Any})}\end{array}}{\Psi \vdash g \text{ match } [\![\overline{\{\chi\}} <: P <: \overline{\{\eta\}}]\!]\ P_i = \text{bounds } \left\langle \langle \bar{l} \rangle \right\rangle \text{ and } \left\langle \langle \bar{u} \rangle \right\rangle} \text{ [M-Param-In]}$$

$$\frac{\begin{array}{c}g \text{ is none of the following: an intersection type, a union type} \\ 1 \le i \le \#(\overline{P}) = \#(\overline{u}) \qquad \overline{l = (\text{if } (P = \underline{P_i}) \text{ then } \underline{g} \text{ else } \texttt{Bottom})} \qquad \overline{u = \texttt{Any}}\end{array}}{\Psi \vdash g \text{ match } [\![\overline{\{\chi\}} <: P <: \overline{\{\eta\}}]\!]\ P_i + \text{bounds } \left\langle \langle \bar{l} \rangle \right\rangle \text{ and } \left\langle \langle \bar{u} \rangle \right\rangle} \text{ [M-Param-Co]}$$

$$\frac{\begin{array}{c}g \text{ is none of the following: an intersection type, a union type} \\ 1 \le i \le \#(\overline{P}) = \#(\bar{l}) \qquad \overline{l = \texttt{Bottom}} \qquad \overline{u = (\text{if } (P = \underline{P_i}) \text{ then } \underline{g} \text{ else } \texttt{Any})}\end{array}}{\Psi \vdash g \text{ match } [\![\overline{\{\chi\}} <: P <: \overline{\{\eta\}}]\!]\ P_i - \text{bounds } \left\langle \langle \bar{l} \rangle \right\rangle \text{ and } \left\langle \langle \bar{u} \rangle \right\rangle} \text{ [M-Param-Contra]}$$

Fig. 22. Matching arguments to signatures (3 of 11: matching a type parameter)

$$\frac{\begin{array}{c}\Psi \vdash g \text{ match } [\![\overline{K}]\!]\ \alpha\ \mathit{flip}(V) \text{ bounds } \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u} \rangle} \right\rangle \\ \Psi \vdash h \text{ match } [\![\overline{K}]\!]\ \rho\ V \text{ bounds } \left\langle \overline{\langle \bar{l'} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u'} \rangle} \right\rangle\end{array}}{\Psi \vdash (g \to h) \text{ match } [\![\overline{K}]\!]\ (\alpha \to \rho)\ V \text{ bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\langle \bar{l} \rangle}, \overline{\langle \bar{l'} \rangle} \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\langle \bar{u} \rangle}, \overline{\langle \bar{u'} \rangle} \right\rangle} \text{ [M-Arrow-Arrow]}$$

$$\Psi \vdash (\overline{g}) \text{ match } _\ (\alpha \to \rho)\ _ \text{ bounds } \left\langle \right\rangle \text{ and } \left\langle \right\rangle \quad \text{[M-Tuple-Arrow]}$$

$$\Psi \vdash G \text{ match } _\ (\alpha \to \rho)\ _ \text{ bounds } \left\langle \right\rangle \text{ and } \left\langle \right\rangle \quad \text{[M-Const-Arrow]}$$

Fig. 23. Matching arguments to signatures (4 of 11: matching an arrow)

$$\frac{\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \alpha\ \underline{V} \text{ bounds } \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u} \rangle} \right\rangle}}{\Psi \vdash (\overline{g}) \text{ match } [\![\overline{K}]\!]\ (\overline{\alpha})\ V \text{ bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\langle \bar{u} \rangle} \right\rangle} \text{ [M-Tuple-Tuple-Success]}$$

$$\frac{\#(\overline{g}) \neq \#(\overline{\alpha})}{\Psi \vdash (\overline{g}) \text{ match } _\ (\overline{\alpha})\ _ \text{ bounds } \left\langle \right\rangle \text{ and } \left\langle \right\rangle} \quad \text{[M-Tuple-Tuple-Failure]}$$

$$\Psi \vdash (g \to h) \text{ match } _\ (\overline{\alpha})\ _ \text{ bounds } \left\langle \right\rangle \text{ and } \left\langle \right\rangle \quad \text{[M-Arrow-Tuple]}$$

$$\Psi \vdash G \text{ match } _\ (\overline{\alpha})\ _ \text{ bounds } \left\langle \right\rangle \text{ and } \left\langle \right\rangle \quad \text{[M-Const-Tuple]}$$

Fig. 24. Matching arguments to signatures (5 of 11: matching a tuple)

$$\frac{\#(\overline{l}) = \#(\overline{u}) = \#(\overline{K}) \qquad \overline{l = \texttt{Bottom}} \qquad \overline{u = \texttt{Any}}}{\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \bigsqcup\langle\overline{g}\rangle = \text{bounds } \left\langle\langle\overline{l}\rangle\right\rangle \text{ and } \left\langle\langle\overline{u}\rangle\right\rangle} \quad \text{[M-Union-In-Same]}$$

$$\frac{\alpha \text{ is not parameter} \qquad \alpha \neq \bigsqcup\langle\overline{g}\rangle}{\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } _\ \alpha = \text{bounds } \langle\,\rangle \text{ and } \langle\,\rangle} \quad \text{[M-Union-In-Different]}$$

$$\frac{1 \leq i \leq \#(\overline{P}) \qquad \overline{l = (\text{if } (P = \underline{P_i}) \text{ then } \underline{\bigsqcup\langle\overline{g}\rangle} \text{ else } \texttt{Bottom})} \qquad \overline{u = (\text{if } (P = \underline{P_i}) \text{ then } \underline{\bigsqcup\langle\overline{g}\rangle} \text{ else Any})}}{\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{\{\overline{\chi}\} <: P <: \{\overline{\eta}\}}]\!]\ P_i = \text{bounds } \left\langle\langle\overline{l}\rangle\right\rangle \text{ and } \left\langle\langle\overline{u}\rangle\right\rangle} \quad \text{[M-Union-In-Param]}$$

$$\frac{\dfrac{\alpha \text{ is not an intersection type}}{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \underline{\alpha} + \text{bounds } \left\langle\overline{\langle\overline{l}\rangle}\right\rangle \text{ and } \left\langle\overline{\langle\overline{u}\rangle}\right\rangle}}{\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!]\ \alpha + \text{bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle\overline{\langle\overline{l}\rangle}\right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle\overline{\langle\overline{u}\rangle}\right\rangle} \quad \text{[M-Union-Whatever-Co]}$$

$$\frac{\dfrac{\alpha \text{ is none of the following: an intersection type, a union type}}{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \underline{\alpha} - \text{bounds } \left\langle\overline{\langle\overline{l}\rangle}\right\rangle \text{ and } \left\langle\overline{\langle\overline{u}\rangle}\right\rangle}}{\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!]\ \alpha - \text{bounds } \left\langle\overline{\overline{\langle\overline{l}\rangle}}\right\rangle \text{ and } \left\langle\overline{\overline{\langle\overline{u}\rangle}}\right\rangle} \quad \text{[M-Union-Whatever-Contra]}$$

Fig. 25. Matching arguments to signatures (6 of 11: matching a union type to a type)

$$\frac{\#(\overline{l}) = \#(\overline{u}) = \#(\overline{K}) \qquad \overline{l = \texttt{Bottom}} \qquad \overline{u = \texttt{Any}}}{\Psi \vdash \bigcap\langle \overline{g} \rangle \text{ match } [\![\overline{K}]\!] \bigcap\langle \overline{g} \rangle = \text{bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle} \text{ [M-Intersection-In-Same]}$$

$$\frac{\alpha \text{ is not a type parameter} \qquad \alpha \neq \bigcap\langle \overline{g} \rangle}{\Psi \vdash \bigcap\langle \overline{g} \rangle \text{ match } _ \ \alpha = \text{bounds } \langle\rangle \text{ and } \langle\rangle} \text{ [M-Intersection-In-Different]}$$

$$\frac{1 \leq i \leq \#(\overline{P}) \qquad \overline{l = (\text{if } (P = \underline{P_i}) \text{ then } \underline{\bigcap\langle \overline{g} \rangle} \text{ else } \texttt{Bottom})} \qquad \overline{u = (\text{if } (P = \underline{P_i}) \text{ then } \underline{\bigcap\langle \overline{g} \rangle} \text{ else } \texttt{Any})}}{\Psi \vdash \bigcap\langle \overline{g} \rangle \text{ match } [\![\overline{\{\chi\}} <: P <: \overline{\{\eta\}}]\!] \ P_i = \text{bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle} \text{ [M-Intersection-In-Param]}$$

$$\frac{\dfrac{\alpha \text{ is not an intersection type}}{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]} \ \underline{\alpha} + \text{bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle}}{\Psi \vdash \bigcap\langle \overline{g} \rangle \text{ match } [\![\overline{K}]\!] \ \alpha + \text{bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \text{ [M-Intersection-Whatever-Co]}$$

$$\frac{\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]} \ \underline{\alpha} - \text{bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle}}{\Psi \vdash \bigcap\langle \overline{g} \rangle \text{ match } [\![\overline{K}]\!] \ \alpha - \text{bounds } \bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \bigcap^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \text{ [M-Intersection-Whatever-Contra]}$$

Fig. 26. Matching arguments to signatures (7 of 11: matching an intersection type to a type)

$$\frac{\begin{array}{c}g \text{ is none of the following: an intersection type, a union type} \\ \#(\overline{l}) = \#(\overline{u}) = \#(\overline{K}) \qquad \overline{l = \texttt{Bottom}} \qquad \overline{u = \texttt{Any}} \\ \Psi \cup \{\overline{K}\} \vdash g \text{ and } \bigsqcup\langle\overline{\alpha}\rangle \text{ compare } V\end{array}}{\Psi \vdash g \text{ match } [\![\overline{K}]\!]\ \bigsqcup\langle\overline{\alpha}\rangle\ V \text{ bounds } \langle\langle\overline{l}\rangle\rangle \text{ and } \langle\langle\overline{u}\rangle\rangle} \text{ [M-Whatever-Union-Success]}$$

$$\frac{\begin{array}{c}g \text{ is none of the following: an intersection type, a union type} \\ \Psi \cup \{\overline{K}\} \vdash g \text{ and } \bigsqcup\langle\overline{\alpha}\rangle \text{ do not compare } V\end{array}}{\Psi \vdash g \text{ match } [\![\overline{K}]\!]\ \bigsqcup\langle\overline{\alpha}\rangle\ V \text{ bounds } \langle\rangle \text{ and } \langle\rangle} \text{ [M-Whatever-Union-Failure]}$$

$$\frac{\begin{array}{c}\#(\overline{l}) = \#(\overline{u}) = \#(\overline{K}) \qquad \overline{l = \texttt{Bottom}} \qquad \overline{u = \texttt{Any}} \\ \Psi \cup \{\overline{K}\} \vdash \bigsqcup\langle\overline{g}\rangle \text{ and } \bigsqcup\langle\overline{\alpha}\rangle \text{ compare } -\end{array}}{\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!]\ \bigsqcup\langle\overline{\alpha}\rangle\ - \text{ bounds } \langle\langle\overline{l}\rangle\rangle \text{ and } \langle\langle\overline{u}\rangle\rangle} \text{ [M-Union-Union-Success]}$$

$$\frac{\Psi \cup \{\overline{K}\} \vdash \bigsqcup\langle\overline{g}\rangle \text{ and } \bigsqcup\langle\overline{\alpha}\rangle \text{ do not compare } -}{\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!]\ \bigsqcup\langle\overline{\alpha}\rangle\ - \text{ bounds } \langle\rangle \text{ and } \langle\rangle} \text{ [M-Union-Union-Failure]}$$

Fig. 27. Matching arguments to signatures (8 of 11: matching a union type)

$$\frac{\begin{array}{c}g \text{ is none of the following: an intersection type, a union type} \\ \#(\overline{l}) = \#(\overline{u}) = \#(\overline{K}) \qquad \overline{l = \texttt{Bottom}} \qquad \overline{u = \texttt{Any}} \\ \Psi \cup \{\overline{K}\} \vdash g \text{ and } \bigsqcap\langle\overline{\alpha}\rangle \text{ compare } V\end{array}}{\Psi \vdash g \text{ match } [\![\overline{K}]\!] \bigsqcap\langle\overline{\alpha}\rangle\ V \text{ bounds } \langle\langle\overline{l}\rangle\rangle \text{ and } \langle\langle\overline{u}\rangle\rangle} \text{ [M-Whatever-Intersection-Success]}$$

$$\frac{\begin{array}{c}g \text{ is none of the following: an intersection type, a union type} \\ \Psi \cup \{\overline{K}\} \vdash g \text{ and } \bigsqcap\langle\overline{\alpha}\rangle \text{ do not compare } V\end{array}}{\Psi \vdash g \text{ match } [\![\overline{K}]\!] \bigsqcap\langle\overline{\alpha}\rangle\ V \text{ bounds } \langle\rangle \text{ and } \langle\rangle} \text{ [M-Whatever-Intersection-Failure]}$$

$$\frac{\begin{array}{c}\#(\overline{l}) = \#(\overline{u}) = \#(\overline{K}) \qquad \overline{l = \texttt{Bottom}} \qquad \overline{u = \texttt{Any}} \\ (V = +) \vee (V = -) \qquad \Psi \cup \{\overline{K}\} \vdash \bigsqcup\langle\overline{g}\rangle \text{ and } \bigsqcap\langle\overline{\alpha}\rangle \text{ compare } V\end{array}}{\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \bigsqcap\langle\overline{\alpha}\rangle\ V \text{ bounds } \langle\langle\overline{l}\rangle\rangle \text{ and } \langle\langle\overline{u}\rangle\rangle} \text{ [M-Union-Intersection-Success]}$$

$$\frac{(V = +) \vee (V = -) \qquad \Psi \cup \{\overline{K}\} \vdash \bigsqcup\langle\overline{g}\rangle \text{ and } \bigsqcap\langle\overline{\alpha}\rangle \text{ do not compare } V}{\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \bigsqcap\langle\overline{\alpha}\rangle\ V \text{ bounds } \langle\rangle \text{ and } \langle\rangle} \text{ [M-Union-Intersection-Failure]}$$

$$\frac{\begin{array}{c}\#(\overline{l}) = \#(\overline{u}) = \#(\overline{K}) \qquad \overline{l = \texttt{Bottom}} \qquad \overline{u = \texttt{Any}} \\ \Psi \cup \{\overline{K}\} \vdash \bigsqcap\langle\overline{g}\rangle \text{ and } \bigsqcap\langle\overline{\alpha}\rangle \text{ compare } +\end{array}}{\Psi \vdash \bigsqcap\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \bigsqcap\langle\overline{\alpha}\rangle + \text{ bounds } \langle\langle\overline{l}\rangle\rangle \text{ and } \langle\langle\overline{u}\rangle\rangle} \text{ [M-Intersection-Intersection-Success]}$$

$$\frac{\Psi \cup \{\overline{K}\} \vdash \bigsqcap\langle\overline{g}\rangle \text{ and } \bigsqcap\langle\overline{\alpha}\rangle \text{ do not compare } +}{\Psi \vdash \bigsqcap\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \bigsqcap\langle\overline{\alpha}\rangle + \text{ bounds } \langle\rangle \text{ and } \langle\rangle} \text{ [M-Intersection-Intersection-Failure]}$$

Fig. 28. Matching arguments to signatures (9 of 11: matching an intersection type)

$$\frac{\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \alpha = \text{bounds } \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u} \rangle} \right\rangle}}{\Psi \vdash T[\![\bar{g}]\!] \text{ match } [\![\overline{K}]\!]\ T[\![\bar{\alpha}]\!] = \text{bounds } \bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \bar{u} \rangle} \right\rangle} \quad \text{[M-Trait-In-Success]}$$

$$\frac{\mathit{distinct}(T, T')}{\Psi \vdash T[\![\bar{g}]\!] \text{ match } [\![\overline{K}]\!]\ T'[\![\bar{\alpha}]\!] = \text{bounds } \langle\rangle \text{ and } \langle\rangle} \quad \text{[M-Trait-In-Failure]}$$

$$\frac{\begin{array}{c}\texttt{trait}\ T[\![\overline{V' _ <: _}]\!]\ __ \ \texttt{end} \in \Psi \\ \Psi \vdash G <: T[\![\bar{g}]\!] \qquad \Psi \vdash (G <: T[\![\overline{\alpha'}]\!]) \Rightarrow (T[\![\bar{g}]\!] <: T[\![\overline{\alpha'}]\!]) \\ \underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \alpha\ V' \text{ bounds } \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u} \rangle} \right\rangle\end{array}}{\Psi \vdash G \text{ match } [\![\overline{K}]\!]\ T[\![\bar{\alpha}]\!] + \text{bounds } \bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \bar{u} \rangle} \right\rangle} \quad \text{[M-Trait-Co]}$$

$$\frac{\begin{array}{c}\texttt{trait}\ T[\![\overline{V' _ <: _}]\!]\ __ \ \texttt{end} \in \Psi \\ \Psi \cup \{\overline{K}\} \vdash T'[\![\overline{\alpha'}]\!] <: T[\![\bar{\alpha}]\!] \qquad \Psi \cup \{\overline{K}\} \vdash (T'[\![\overline{\alpha'}]\!] <: T[\![\overline{\alpha''}]\!]) \Rightarrow (T[\![\bar{\alpha}]\!] <: T[\![\overline{\alpha''}]\!]) \\ \underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \alpha\ \mathit{flip}(V') \text{ bounds } \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u} \rangle} \right\rangle\end{array}}{\Psi \vdash T[\![\bar{g}]\!] \text{ match } [\![\overline{K}]\!]\ T'[\![\overline{\alpha'}]\!] - \text{bounds } \bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \bar{u} \rangle} \right\rangle} \quad \text{[M-Trait-Contra]}$$

$$\Psi \vdash O[\![__]\!] \text{ match } _\ T[\![__]\!] = \text{bounds } \langle\rangle \text{ and } \langle\rangle \quad \text{[M-Object-Trait-In]}$$

$$\Psi \vdash O[\![__]\!] \text{ match } _\ T[\![__]\!] - \text{bounds } \langle\rangle \text{ and } \langle\rangle \quad \text{[M-Object-Trait-Contra]}$$

$$\frac{\{\overline{C}\} = \mathit{ancestors}(\Psi, G) \qquad T \notin \{\overline{\mathit{name}(C)}\}}{\Psi \vdash G \text{ match } _\ T[\![__]\!] + \text{bounds } \langle\rangle \text{ and } \langle\rangle} \quad \text{[M-Trait-Co-Ancestor-Not-Found]}$$

$$\frac{\{\overline{C}\} = \mathit{ancestors}(\Psi, T'[\![\overline{\alpha'}]\!]) \qquad T \notin \{\overline{\mathit{name}(C)}\}}{\Psi \vdash T[\![__]\!] \text{ match } _\ T'[\![\overline{\alpha'}]\!] - \text{bounds } \langle\rangle \text{ and } \langle\rangle} \quad \text{[M-Trait-Contra-Ancestor-Not-Found]}$$

$$\Psi \vdash (\bar{g}) \text{ match } _\ T[\![__]\!]\ _ \text{ bounds } \langle\rangle \text{ and } \langle\rangle \quad \text{[M-Tuple-Trait]}$$

$$\Psi \vdash (g \to h) \text{ match } _\ T[\![__]\!]\ _ \text{ bounds } \langle\rangle \text{ and } \langle\rangle \quad \text{[M-Arrow-Trait]}$$

Fig. 29. Matching arguments to signatures (10 of 11: matching a trait type)

$$\frac{\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \alpha = \text{bounds } \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u} \rangle} \right\rangle}}{\Psi \vdash O[\![\bar{g}]\!] \text{ match } [\![\overline{K}]\!]\ O[\![\bar{\alpha}]\!]\ V \text{ bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\langle \overline{\langle \bar{l} \rangle} \rangle} \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\langle \overline{\langle \bar{u} \rangle} \rangle} \right\rangle} \text{ [M-Object-Object-Same]}$$

$$\frac{\mathit{distinct}(O, O')}{\Psi \vdash O[\![_]\!] \text{ match } _\ O'[\![_]\!]\ _ \text{ bounds } \langle\rangle \text{ and } \langle\rangle} \text{ [M-Object-Object-Different]}$$

$$\Psi \vdash T[\![_]\!] \text{ match } _\ O[\![_]\!] = \text{bounds } \langle\rangle \text{ and } \langle\rangle \quad \text{[M-Trait-Object-In]}$$

$$\Psi \vdash T[\![_]\!] \text{ match } _\ O[\![_]\!] + \text{bounds } \langle\rangle \text{ and } \langle\rangle \quad \text{[M-Trait-Object-Co]}$$

$$\frac{\begin{array}{c}\texttt{trait}\ T[\![\overline{V' _ <: _}]\!]\ __ \text{ end} \in \Psi \\ \Psi \cup \{\overline{K}\} \vdash O[\![\bar{\alpha}]\!] <: T[\![\overline{\alpha'}]\!] \qquad \Psi \cup \{\overline{K}\} \vdash (O[\![\bar{\alpha}]\!] <: T[\![\overline{\alpha''}]\!]) \Rightarrow (T[\![\overline{\alpha'}]\!] <: T[\![\overline{\alpha''}]\!]) \\ \overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \alpha'\ \mathit{flip}(V') \text{ bounds } \left\langle \overline{\langle \bar{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \bar{u} \rangle} \right\rangle}\end{array}}{\Psi \vdash T[\![\bar{g}]\!] \text{ match } [\![\overline{K}]\!]\ O[\![\bar{\alpha}]\!] - \text{bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\langle \overline{\langle \bar{l} \rangle} \rangle} \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\langle \overline{\langle \bar{u} \rangle} \rangle} \right\rangle} \text{ [M-Trait-Object-Contra]}$$

$$\frac{\{\overline{C}\} = \mathit{ancestors}(\Psi, O[\![\bar{\alpha}]\!]) \qquad T \notin \{\overline{\mathit{name}(C)}\}}{\Psi \vdash T[\![_]\!] \text{ match } _\ O[\![\bar{\alpha}]\!] - \text{bounds } \langle\rangle \text{ and } \langle\rangle} \text{ [M-Trait-Object-Ancestor-Not-Found]}$$

$$\Psi \vdash (\bar{g}) \text{ match } _\ O[\![\bar{\alpha}]\!]\ _ \text{ bounds } \langle\rangle \text{ and } \langle\rangle \quad \text{[M-Tuple-Object]}$$

$$\Psi \vdash (g \to h) \text{ match } _\ O[\![\bar{\alpha}]\!]\ _ \text{ bounds } \langle\rangle \text{ and } \langle\rangle \quad \text{[M-Arrow-Object]}$$

Fig. 30. Matching arguments to signatures (11 of 11: matching an object type)

APPENDIX C LEMMAS

Lemma C.1. *For* $\left\langle\overline{\langle\overline{l'}\rangle}\right\rangle = \bigsqcup_{\overline{K}}^{\times}\left\langle\overline{\langle\overline{\langle\overline{l}\rangle}\rangle}\right\rangle$ *and* $\left\langle\overline{\langle\overline{u'}\rangle}\right\rangle = \bigsqcap_{\overline{K}}^{\times}\left\langle\overline{\langle\overline{\langle\overline{u}\rangle}\rangle}\right\rangle$ *such that* $1 \le m = \#\left(\overline{\langle\overline{\langle\overline{l}\rangle}\rangle}\right) = \#\left(\overline{\langle\overline{\langle\overline{u}\rangle}\rangle}\right)$ *and* $\overline{\#\left(\overline{\langle\overline{l}\rangle}\right) = \#\left(\overline{\langle\overline{u}\rangle}\right)}$ *and* $\overline{\overline{\#(\overline{K}) = \#(\overline{l}) = \#(\overline{u})}}$ *and* $\overline{\overline{\overline{\underline{\Psi} \vdash l <: u}}}$, *there exists* $1 \le i \le \#\left(\overline{\langle\overline{l'}\rangle}\right) = \#\left(\overline{\langle\overline{u'}\rangle}\right)$ *such that*

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l'_{\underline{i}}}\rangle \text{ and } \langle\overline{u'_{\underline{i}}}\rangle$$

if and only if there exist $\overline{1 \le k \le \#\left(\overline{\langle\overline{l}\rangle}\right) = \#\left(\overline{\langle\overline{u}\rangle}\right)}^{1 \le j \le m}$ *such that for each* j,

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l_{\underline{jk_j}}}\rangle \text{ and } \langle\overline{u_{\underline{jk_j}}}\rangle$$

Proof. We prove by induction on m. First, if $m = 1$,

$$\left\langle\overline{\langle\overline{l'}\rangle}\right\rangle = \bigsqcup_{\overline{K}}^{\times}\left\langle\overline{\langle\overline{l}\rangle}\right\rangle = \left\langle\overline{\langle\overline{l}\rangle}\right\rangle \quad \text{and} \quad \left\langle\overline{\langle\overline{u'}\rangle}\right\rangle = \bigsqcap_{\overline{K}}^{\times}\left\langle\overline{\langle\overline{u}\rangle}\right\rangle = \left\langle\overline{\langle\overline{u}\rangle}\right\rangle$$

Therefore, the statement holds trivially.
Now, suppose the statement holds for $m-1$ where $m > 1$, and we show that it holds for m. For

$$\left\langle\overline{\langle\overline{l'}\rangle}\right\rangle = \bigsqcup_{\overline{K}}^{\times}\left\langle\overline{\langle\overline{l}\rangle}^{m-1}, \left\langle\overline{l_{\underline{\underline{m}}}}\right\rangle\right\rangle \quad \text{and} \quad \left\langle\overline{\langle\overline{u'}\rangle}\right\rangle = \bigsqcap_{\overline{K}}^{\times}\left\langle\overline{\langle\overline{u}\rangle}^{m-1}, \left\langle\overline{u_{\underline{\underline{m}}}}\right\rangle\right\rangle,$$

by letting $\left\langle\overline{\langle\overline{l''}\rangle}\right\rangle = \bigsqcup_{\overline{K}}^{\times}\left\langle\overline{\langle\overline{l}\rangle}^{m-1}\right\rangle$ and $\left\langle\overline{\langle\overline{u''}\rangle}\right\rangle = \bigsqcap_{\overline{K}}^{\times}\left\langle\overline{\langle\overline{u}\rangle}^{m-1}\right\rangle$, we have

$$\left\langle\overline{\langle\overline{l'}\rangle}\right\rangle = \left\langle\overline{\overline{\bigsqcup^{\mathsf{T}}\left\langle\langle\overline{l''_{i'k}}^{\,k}\rangle, \langle\overline{l_{mj'k}}^{\,k}\rangle\right\rangle}^{j'}}^{i'}\right\rangle \quad \text{and} \quad \left\langle\overline{\langle\overline{u'}\rangle}\right\rangle = \left\langle\overline{\overline{\bigsqcap^{\mathsf{T}}\left\langle\langle\overline{u''_{i'k}}^{\,k}\rangle, \langle\overline{u_{mj'k}}^{\,k}\rangle\right\rangle}^{j'}}^{i'}\right\rangle$$

For $\sigma = [\overline{g/P}]$, there exists $1 \le i \le \#\left(\overline{\langle\overline{l'}\rangle}\right) = \#\left(\overline{\langle\overline{u'}\rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l'_{\underline{i}}}\rangle \text{ and } \langle\overline{u'_{\underline{i}}}\rangle$$

if and only if

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \bigsqcup^{\mathsf{T}}\left\langle\langle\overline{l''_{\underline{i'}}}\rangle, \langle\overline{l_{\underline{mj'}}}\rangle\right\rangle \text{ and } \bigsqcap^{\mathsf{T}}\left\langle\langle\overline{u''_{\underline{i}}}\rangle, \langle\overline{u_{\underline{mj'}}}\rangle\right\rangle \text{ for some } i' \text{ and } j'$$

if and only if

$$\overline{\Psi \vdash \sigma \bigsqcup\langle l''_{i'k}, l_{mj'k}\rangle <: g_k}^{\,k} \quad \text{and} \quad \overline{\Psi \vdash g_k <: \sigma \bigsqcap\langle u''_{i'k}, u_{mj'k}\rangle}^{\,k}$$

if and only if

$$\overline{\underline{\Psi} \vdash \underline{\sigma} l''_{\underline{i'}} <: g} \quad \text{and} \quad \overline{\underline{\Psi} \vdash \underline{\sigma} l_{\underline{mj'}} <: g} \quad \text{and} \quad \overline{\underline{\Psi} \vdash g <: \underline{\sigma} u''_{\underline{i'}}} \quad \text{and} \quad \overline{\underline{\Psi} \vdash g <: \underline{\sigma} u_{\underline{mj'}}}$$

if and only if

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l''_{\underline{i'}}}\rangle \text{ and } \langle\overline{u''_{\underline{i'}}}\rangle \quad \text{and} \quad \Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l_{\underline{mj'}}}\rangle \text{ and } \langle\overline{u_{\underline{mj'}}}\rangle$$

if and only if there exist $\overline{1 \le k \le \#\left(\overline{\langle\overline{l}\rangle}\right) = \#\left(\overline{\langle\overline{u}\rangle}\right)}^{1 \le j \le m-1}$ and $k_m = j'$ such that for each $1 \le j \le m$,

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l_{\underline{jk_j}}}\rangle \text{ and } \langle\overline{u_{\underline{jk_j}}}\rangle$$

by inductive hypothesis. □

Lemma C.2. *For* $\overline{K = \{\overline{\chi}\} <: P <: \{\overline{\eta}\}}$ *and* $\sigma = [\overline{\gamma/P}]$ *with* $\overline{\overline{\underline{\Delta} \vdash \chi \text{ ok}}}$, $\overline{\overline{\underline{\Delta} \vdash \eta \text{ ok}}}$, $\overline{\underline{\Delta} \vdash \gamma \text{ ok}}$, *and* $\{\overline{P}\} \cap FV(\overline{\gamma}) = \emptyset$, *if*

$$\Delta \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{\bigsqcup\{\overline{\chi}\}} \rangle \text{ and } \langle \overline{\bigsqcap\{\overline{\eta}\}} \rangle$$

and

$$\Delta \cup \{\overline{K}\} \vdash \alpha <: \alpha'$$

then

$$\Delta \vdash \sigma\alpha <: \sigma\alpha'$$

Proof. If $FV(\alpha, \alpha') \cap \{\overline{P}\} = \emptyset$, the statement holds trivially. We prove by induction according to whichever subtype rule was used:

Rule [Sub-Params-Same]: $\Delta \vdash \sigma P_i <: \sigma P_i$ is trivially satisfied. ✓

Rule [Sub-Params-Different]:

If $\Delta \cup \{\overline{K}\} \vdash P_i <: P_j$ for $i \neq j$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash P_i <: \chi_{jk} \text{ for some } k$$

from which we can infer inductively that

$$\Delta \vdash \gamma_i <: \sigma\chi_{jk}$$

From obeys judgement, it follows that

$$\Delta \vdash \sigma \bigsqcup\{\overline{\chi_j}\} <: \gamma_j$$

which implies

$$\Delta \vdash \gamma_i <: \gamma_j$$

therefore

$$\Delta \vdash \sigma P_i <: \sigma P_j$$

If $\Delta \cup \{\overline{K}\} \vdash P_i <: Q$ for $Q \notin \{\overline{P}\}$ and $\{\overline{\chi'}\} <: Q <: _ \in \Delta$, it follows that for some k

$$\Delta \cup \{\overline{K}\} \vdash P_i <: \chi'_k \text{ and } \Delta \cup \{\overline{K}\} \vdash \chi'_k <: Q$$

from which we can infer inductively that

$$\Delta \vdash \gamma_i <: \sigma\chi'_k \text{ and } \Delta \vdash \sigma\chi'_k <: Q$$

which implies

$$\Delta \vdash \gamma_i <: Q$$

therefore

$$\Delta \vdash \sigma P_i <: \sigma Q$$

If $\Delta \cup \{\overline{K}\} \vdash Q <: P_j$ for $Q \notin \{\overline{P}\}$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash Q <: \chi_{jk} \text{ for some k}$$

from which we can infer inductively that

$$\Delta \vdash Q <: \sigma\chi_{jk}$$

From obeys judgement, it follows that

$$\Delta \vdash \sigma \bigsqcup\{\overline{\chi_j}\} <: \gamma_j$$

which implies

$$\Delta \vdash Q <: \gamma_j$$

therefore

$$\Delta \vdash \sigma Q <: \sigma P_j$$

✓

Rule [Sub-LParam]:

If $\Delta \cup \{\overline{K}\} \vdash P_i <: \alpha$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash \eta_{ik} <: \alpha \text{ for some } k$$

from which we can infer inductively that

$$\Delta \vdash \sigma\eta_{ik} <: \sigma\alpha$$

From obeys judgement, it follows that

$$\Delta \vdash \gamma_i <: \sigma \bigsqcap \{\overline{\eta_{\underline{i}}}\}$$

which implies

$$\Delta \vdash \gamma_i <: \sigma\alpha$$

therefore

$$\Delta \vdash \sigma P_i <: \sigma\alpha$$

If $\Delta \cup \{\overline{K}\} \vdash Q <: \alpha$ for $Q \notin \{\overline{P}\}$ and $_ <: Q <: \{\overline{\eta'}\} \in \Delta$, it follows that for some k

$$\Delta \cup \{\overline{K}\} \vdash Q <: \eta'_k \text{ and } \Delta \cup \{\overline{K}\} \vdash \eta'_k <: \alpha$$

from which we can infer inductively that

$$\Delta \vdash Q <: \sigma\eta'_k \text{ and } \Delta \vdash \sigma\eta'_k <: \sigma\alpha$$

which implies

$$\Delta \vdash Q <: \sigma\alpha$$

therefore

$$\Delta \vdash \sigma Q <: \sigma\alpha$$

✓

Rule [Sub-LParam-Any]: If $\Delta \cup \{\overline{K}\} \vdash P <: \alpha$ for $_ <: P <: \{\,\} \in \Delta \cup \{\overline{K}\}$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash \mathsf{Any} <: \alpha$$

from which we can infer inductively that

$$\Delta \vdash \mathsf{Any} <: \sigma\alpha$$

therefore

$$\Delta \vdash \sigma P <: \sigma\alpha$$

✓

Rule [Sub-RParam]:

If $\Delta \cup \{\overline{K}\} \vdash \alpha <: P_j$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash \alpha <: \chi_{jk} \text{ for some } k$$

from which we can infer inductively that

$$\Delta \vdash \sigma\alpha <: \sigma\chi_{jk}$$

From obeys judgement, it follows that

$$\Delta \vdash \sigma \bigsqcup \{\overline{\chi_{\underline{j}}}\} <: \gamma_j$$

which implies

$$\Delta \vdash \sigma\alpha <: \gamma_j$$

therefore

$$\Delta \vdash \sigma\alpha <: \sigma P_j$$

If $\Delta \cup \{\overline{K}\} \vdash \alpha <: Q$ for $Q \notin \{\overline{P}\}$ and $\{\overline{\chi'}\} <: Q <: _ \in \Delta$, it follows that for some k

$$\Delta \cup \{\overline{K}\} \vdash \alpha <: \chi'_k \text{ and } \Delta \cup \{\overline{K}\} \vdash \chi'_k <: Q$$

from which we can infer inductively that

$$\Delta \vdash \sigma\alpha <: \sigma\chi'_k \text{ and } \Delta \vdash \sigma\chi'_k <: Q$$

which implies

$$\Delta \vdash \sigma\alpha <: Q$$

therefore

$$\Delta \vdash \sigma\alpha <: \sigma Q$$

✓

Rule [Sub-RParam-Bottom]: If $\Delta \cup \{\overline{K}\} \vdash \alpha <: P$ for $\{\} <: P <: _ \in \Delta \cup \{\overline{K}\}$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash \alpha <: \texttt{Bottom}$$

from which we can infer inductively that

$$\Delta \vdash \sigma\alpha <: \texttt{Bottom}$$

therefore

$$\Delta \vdash \sigma\alpha <: \sigma P$$

✓

Rule [Sub-Tuple]: From $\Delta \cup \{\overline{K}\} \vdash (\overline{\alpha}) <: (\overline{\eta})$, it follows that

$$\overline{\Delta \cup \{\overline{K}\} \vdash \alpha <: \eta}$$

from which we can infer inductively that

$$\overline{\underline{\Delta} \vdash \underline{\sigma}\alpha <: \underline{\sigma}\eta}$$

therefore

$$\Delta \vdash \sigma(\overline{\alpha}) <: \sigma(\overline{\eta})$$

✓

Rule [Sub-Arrow]: From $\Delta \cup \{\overline{K}\} \vdash (\alpha \to \rho) <: (\alpha' \to \rho')$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash \alpha' <: \alpha \text{ and } \Delta \cup \{\overline{K}\} \vdash \rho <: \rho'$$

from which we can infer inductively that

$$\Delta \vdash \sigma\alpha' <: \sigma\alpha \text{ and } \Delta \vdash \sigma\rho <: \sigma\rho'$$

therefore

$$\Delta \vdash \sigma(\alpha \to \rho) <: \sigma(\alpha' \to \rho')$$

✓

Rule [Sub-Object-Same]: From $\Delta \cup \{\overline{K}\} \vdash O[\![\overline{\alpha}]\!] <: O[\![\overline{\eta}]\!]$, it follows that

$$\overline{\Delta \cup \{\overline{K}\} \vdash \alpha \equiv \eta}$$

from which we can infer inductively that

$$\overline{\underline{\Delta} \vdash \underline{\sigma}\alpha \equiv \underline{\sigma}\eta}$$

which implies

$$\Delta \vdash O[\![\overline{\underline{\sigma}\alpha}]\!] <: O[\![\overline{\underline{\sigma}\eta}]\!]$$

therefore

$$\Delta \vdash \sigma O[\![\overline{\alpha}]\!] <: \sigma O[\![\overline{\eta}]\!]$$

✓

Rule [Sub-Trait]: From $\Delta \cup \{\overline{K}\} \vdash T[\![\overline{\alpha}]\!] <: T[\![\overline{\eta}]\!]$, it follows that

$$\overline{\Delta \cup \{\overline{K}\} \vdash \alpha \text{ and } \eta \text{ compare } V}$$

from which we can infer inductively that

$$\overline{\underline{\Delta} \vdash \underline{\sigma}\alpha \text{ and } \underline{\sigma}\eta \text{ compare } V}$$

which implies

$$\Delta \vdash T[\![\overline{\sigma\alpha}]\!] <: T[\![\overline{\sigma\eta}]\!]$$

therefore

$$\Delta \vdash \sigma T[\![\overline{\alpha}]\!] <: \sigma T[\![\overline{\eta}]\!]$$

✓

Rule [Sub-O-Parents]: From $\Delta \cup \{\overline{K}\} \vdash O[\![\overline{\alpha}]\!] <: T[\![\overline{\eta}]\!]$, it follows that

$$\texttt{object}\ O[\![\overline{Q <: _}]\!](_) <: \{\overline{T[\![\overline{\chi}]\!]}\}\ _\ \texttt{end} \in \Delta \cup \{\overline{K}\}$$

and for some $T_k[\![\overline{[\overline{\alpha/Q}]\chi_{\underline{k}}}]\!] \in \mathit{parents}(\Delta \cup \{\overline{K}\}, O[\![\overline{\alpha}]\!])$,

$$\Delta \cup \{\overline{K}\} \vdash T_k[\![\overline{[\overline{\alpha/Q}]\chi_{\underline{k}}}]\!] <: T[\![\overline{\eta}]\!]$$

from which we can infer inductively that

$$\Delta \vdash \sigma T_k[\![\overline{[\overline{\alpha/Q}]\chi_{\underline{k}}}]\!] <: \sigma T[\![\overline{\eta}]\!]$$

which implies

$$\Delta \vdash T_k[\![\overline{[\overline{\sigma\alpha/Q}]\chi_{\underline{k}}}]\!] <: \sigma T[\![\overline{\eta}]\!]$$

Since $T_k[\![\overline{[\overline{\sigma\alpha/Q}]\chi_{\underline{k}}}]\!] \in \mathit{parents}(\Delta, O[\![\overline{\sigma\alpha}]\!]) = \mathit{parents}(\Delta, \sigma O[\![\overline{\alpha}]\!])$, we can conclude that

$$\Delta \vdash \sigma O[\![\overline{\alpha}]\!] <: \sigma T[\![\overline{\eta}]\!]$$

✓

Rule [Sub-T-Parents]: From $\Delta \cup \{\overline{K}\} \vdash T[\![\overline{\alpha}]\!] <: T'[\![\overline{\eta}]\!]$, it follows that

$$\texttt{trait}\ T[\![\overline{_\ Q <: _}]\!] <: \{\overline{T[\![\overline{\chi}]\!]}\}\ _\ \texttt{end} \in \Delta \cup \{\overline{K}\}$$

and for some $T_k[\![\overline{[\overline{\alpha/Q}]\chi_{\underline{k}}}]\!] \in \mathit{parents}(\Delta \cup \{\overline{K}\}, T[\![\overline{\alpha}]\!])$,

$$\Delta \cup \{\overline{K}\} \vdash T_k[\![\overline{[\overline{\alpha/Q}]\chi_{\underline{k}}}]\!] <: T'[\![\overline{\eta}]\!]$$

from which we can infer inductively that

$$\Delta \vdash \sigma T_k[\![\overline{[\overline{\alpha/Q}]\chi_{\underline{k}}}]\!] <: \sigma T'[\![\overline{\eta}]\!]$$

which implies

$$\Delta \vdash T_k[\![\overline{[\overline{\sigma\alpha/Q}]\chi_{\underline{k}}}]\!] <: \sigma T'[\![\overline{\eta}]\!]$$

Since $T_k[\![\overline{[\overline{\sigma\alpha/Q}]\chi_{\underline{k}}}]\!] \in \mathit{parents}(\Delta, T[\![\overline{\sigma\alpha}]\!]) = \mathit{parents}(\Delta, \sigma T[\![\overline{\alpha}]\!])$, we can conclude that

$$\Delta \vdash \sigma T[\![\overline{\alpha}]\!] <: \sigma T'[\![\overline{\eta}]\!]$$

✓

Rule [Sub-LUnion]: From $\Delta \cup \{\overline{K}\} \vdash (\alpha \sqcup \eta) <: \chi$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash \alpha <: \chi \text{ and } \Delta \cup \{\overline{K}\} \vdash \eta <: \chi$$

from which we can infer inductively that

$$\Delta \vdash \sigma\alpha <: \sigma\chi \text{ and } \Delta \vdash \sigma\eta <: \sigma\chi$$

which implies

$$\Delta \vdash (\sigma\alpha \sqcup \sigma\eta) <: \sigma\chi$$

therefore

$$\Delta \vdash \sigma(\alpha \sqcup \eta) <: \sigma\chi$$

✓

Rule [Sub-LIntersection]: From $\Delta \cup \{\overline{K}\} \vdash (\alpha \sqcap \eta) <: \chi$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash \alpha <: \chi \text{ or } \Delta \cup \{\overline{K}\} \vdash \eta <: \chi \text{ or } \Delta \cup \{\overline{K}\} \vdash \alpha \lozenge \eta$$

If $\Delta \cup \{\overline{K}\} \vdash \alpha \lozenge \eta$,

$$\Delta \cup \{\overline{K}\} \vdash (\alpha \sqcap \eta) <: \texttt{Bottom}$$

which implies

$$\Delta \vdash \sigma(\alpha \sqcap \eta) <: \texttt{Bottom}$$

therefore

$$\Delta \vdash \sigma(\alpha \sqcap \eta) <: \sigma\chi$$

If $\Delta \cup \{\overline{K}\} \vdash \alpha \not\lozenge \eta$,

$$\Delta \cup \{\overline{K}\} \vdash \alpha <: \chi \text{ or } \Delta \cup \{\overline{K}\} \vdash \eta <: \chi$$

from which we can infer inductively that

$$\Delta \vdash \sigma\alpha <: \sigma\chi \text{ or } \Delta \vdash \sigma\eta <: \sigma\chi$$

which implies

$$\Delta \vdash (\sigma\alpha \sqcap \sigma\eta) <: \sigma\chi$$

therefore

$$\Delta \vdash \sigma(\alpha \sqcap \eta) <: \sigma\chi$$

✓

Rule [Sub-RUnion]: From $\Delta \cup \{\overline{K}\} \vdash \alpha <: (\eta \sqcup \chi)$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash \alpha <: \eta \text{ or } \Delta \cup \{\overline{K}\} \vdash \alpha <: \chi$$

from which we can infer inductively that

$$\Delta \vdash \sigma\alpha <: \sigma\eta \text{ or } \Delta \vdash \sigma\alpha <: \sigma\chi$$

which implies

$$\Delta \vdash \sigma\alpha <: (\sigma\eta \sqcup \sigma\chi)$$

therefore

$$\Delta \vdash \sigma\alpha <: \sigma(\eta \sqcup \chi)$$

✓

Rule [Sub-RIntersection]: From $\Delta \cup \{\overline{K}\} \vdash \alpha <: (\eta \sqcap \chi)$, it follows that

$$\Delta \cup \{\overline{K}\} \vdash \alpha <: \eta \text{ and } \Delta \cup \{\overline{K}\} \vdash \alpha <: \chi$$

from which we can infer inductively that

$$\Delta \vdash \sigma\alpha <: \sigma\eta \text{ and } \Delta \vdash \sigma\alpha <: \sigma\chi$$

which implies

$$\Delta \vdash \sigma\alpha <: (\sigma\eta \sqcap \sigma\chi)$$

therefore

$$\Delta \vdash \sigma\alpha <: \sigma(\eta \sqcap \chi)$$

✓

And so all cases are covered.

□

Lemma C.3. *If* $\Delta \vdash C <: T[\![\overline{\alpha}]\!]$, *then there exists* $T[\![\overline{\alpha'}]\!]$ *such that*

$$\Delta \vdash C <: T[\![\overline{\alpha'}]\!] \text{ and } \Delta \vdash (C <: T[\![\overline{\alpha''}]\!]) \Rightarrow (T[\![\overline{\alpha'}]\!] <: T[\![\overline{\alpha''}]\!])$$

Proof. Let $A = \left\{T[\![\overline{\alpha}]\!] \mid \Delta \vdash C <: T[\![\overline{\alpha}]\!]\right\}$, then A is a partially ordered set with respect to subtype relation. It is sufficient to show that if $|A| \geq 1$, A has the least element. If $|A| = 1$, $T[\![\overline{\alpha}]\!] \in A$ itself is the least element. If $|A| > 1$, we show that for any pair $T[\![\overline{\alpha'}]\!], T[\![\overline{\alpha''}]\!] \in A$, there exists $T[\![\overline{\alpha'''}]\!] \in A$ such that $\Delta \vdash T[\![\overline{\alpha'''}]\!] <: T[\![\overline{\alpha'}]\!]$ and $\Delta \vdash T[\![\overline{\alpha'''}]\!] <: T[\![\overline{\alpha''}]\!]$, then the rest is a straightforward induction.

Note that C is either

$$C = T'[\![\overline{\eta}]\!] \text{ with } \texttt{trait } T'[\![\overline{_ \; P <: \{\overline{\xi}\}}]\!] <: \left\{_\right\} \; _ \; \texttt{end} \in \Delta$$

or

$$C = O[\![\overline{\eta}]\!] \text{ with } \texttt{object } O[\![\overline{P <: \{\overline{\xi}\}}]\!](_) <: \left\{_\right\} \; _ \; \texttt{end} \in \Delta,$$

and let $C' = T'[\![\overline{P}]\!]$ or $C' = O[\![\overline{P}]\!]$ for respective case. If $C = T'[\![\overline{\eta}]\!]$ and $T' = T$, $T'[\![\overline{\eta}]\!]$ itself is the least element. Otherwise, if $T[\![\overline{\alpha'}]\!], T[\![\overline{\alpha''}]\!] \in A$,

$$T[\![\overline{\alpha'}]\!] = [\overline{\eta/P}]T[\![\overline{\chi'}]\!] \text{ and } T[\![\overline{\alpha''}]\!] = [\overline{\eta/P}]T[\![\overline{\chi''}]\!]$$

for $T[\![\overline{\chi'}]\!], T[\![\overline{\chi''}]\!] \in \mathit{properAncestors}(\Psi \cup \left\{\overline{\{\,\} <: P <: \{\overline{\xi}\}}\right\}, C')$ and $\sigma = [\overline{\eta/P}]$ such that $\Delta \vdash \sigma$ on $\langle \overline{P} \rangle$ obeys $\langle \overline{\texttt{Bottom}} \rangle$ and $\langle \overline{\bigsqcap\{\overline{\xi}\}} \rangle$. Since the trait T' or the object O is well-formed, its proper ancestors are well-formed, and so there exists $T[\![\overline{\chi'''}]\!] \in \mathit{properAncestors}(\Psi \cup \left\{\overline{\{\,\} <: P <: \{\overline{\xi}\}}\right\}, C')$ such that

$$\begin{gathered}
\Psi \cup \left\{\overline{\{\,\} <: P <: \{\overline{\xi}\}}\right\} \vdash C' <: T[\![\overline{\chi'''}]\!] \\
\Psi \cup \left\{\overline{\{\,\} <: P <: \{\overline{\xi}\}}\right\} \vdash T[\![\overline{\chi'''}]\!] <: T[\![\overline{\chi'}]\!] \\
\Psi \cup \left\{\overline{\{\,\} <: P <: \{\overline{\xi}\}}\right\} \vdash T[\![\overline{\chi'''}]\!] <: T[\![\overline{\chi''}]\!]
\end{gathered}$$

which implies

$$\begin{gathered}
\Delta \cup \left\{\overline{\{\,\} <: P <: \{\overline{\xi}\}}\right\} \vdash C' <: T[\![\overline{\chi'''}]\!] \\
\Delta \cup \left\{\overline{\{\,\} <: P <: \{\overline{\xi}\}}\right\} \vdash T[\![\overline{\chi'''}]\!] <: T[\![\overline{\chi'}]\!] \\
\Delta \cup \left\{\overline{\{\,\} <: P <: \{\overline{\xi}\}}\right\} \vdash T[\![\overline{\chi'''}]\!] <: T[\![\overline{\chi''}]\!]
\end{gathered}$$

By Lemma C.2,

$$\begin{gathered}
\Delta \vdash C <: \sigma T[\![\overline{\chi'''}]\!] \\
\Delta \vdash \sigma T[\![\overline{\chi'''}]\!] <: T[\![\overline{\alpha'}]\!] \\
\Delta \vdash \sigma T[\![\overline{\chi'''}]\!] <: T[\![\overline{\alpha''}]\!]
\end{gathered}$$

therefore $T[\![\overline{\alpha'''}]\!] = \sigma T[\![\overline{\chi'''}]\!] \in A$ such that $\Delta \vdash T[\![\overline{\alpha'''}]\!] <: T[\![\overline{\alpha'}]\!]$ and $\Delta \vdash T[\![\overline{\alpha'''}]\!] <: T[\![\overline{\alpha''}]\!]$. □

Lemma C.4. *If* $\Delta \vdash \Xi_1 \lesssim \Xi_2$, $\Delta \vdash \Xi_1 \xrightarrow{\equiv} \Xi_1'$, *and* $\Delta \vdash \Xi_2 \xrightarrow{\equiv} \Xi_2'$, *then* $\Delta \vdash \Xi_1' \lesssim \Xi_2'$.

Proof. Let

$$\begin{gathered}
\Xi_1 = \exists[\![\overline{\{\overline{\chi_1}\} <: P_1 <: \{\overline{\eta_1}\}}]\!]\alpha_1 \text{ and } \Xi_2 = \exists[\![\overline{\{\overline{\chi_2}\} <: P_2 <: \{\overline{\eta_2}\}}]\!]\alpha_2, \\
\Xi_1' = \exists[\![\{\overline{\chi_1'}\} <: P_1' <: \{\overline{\eta_1'}\}]\!]\alpha_1' \text{ and } \Xi_2' = \exists[\![\{\overline{\chi_2'}\} <: P_2' <: \{\overline{\eta_2'}\}]\!]\alpha_2', \\
\Delta_1 = \left\{\overline{\{\overline{\chi_1}\} <: P_1 <: \{\overline{\eta_1}\}}\right\}, \text{ and } \Delta_1' = \left\{\overline{\{\overline{\chi_1'}\} <: P_1' <: \{\overline{\eta_1'}\}}\right\}.
\end{gathered}$$

If $\Delta \vdash \Xi_1 \xrightarrow{\equiv} \Xi_1'$ using σ_1 and $\Delta \vdash \Xi_2 \xrightarrow{\equiv} \Xi_2'$ using σ_2, then $\alpha_1' = \sigma_1\alpha_1$ and $\alpha_2' = \sigma_2\alpha_2$, and neither of them is `Bottom`. Also, from the existential reduction, it follows that

$$\Delta \cup \Delta_1' \vdash \sigma_1 \text{ on } \langle \overline{P_1} \rangle \text{ obeys } \left\langle \overline{\bigsqcup\{\overline{\chi_1}\}} \right\rangle \text{ and } \left\langle \overline{\bigsqcap\{\overline{\eta_1}\}} \right\rangle$$

If $\Delta \vdash \Xi_1 \lesssim \Xi_2$ using σ, then

$$\Delta \cup \Delta_1 \vdash \alpha_1 <: \sigma\alpha_2$$

, hence

$$\Delta \cup \Delta_1 \cup \Delta_1' \vdash \alpha_1 <: \sigma\alpha_2.$$

Therefore, by Lemma C.2,

$$\Delta \cup \Delta_1' \vdash \sigma_1\alpha_1 <: \sigma_1\sigma\alpha_2,$$

thus $\sigma_1\sigma\alpha_2$ is not `Bottom`. Since σ_2 represents the minimum requirement that prevents $\sigma_2\alpha_2$ from being `Bottom`, there exists σ' such that $\sigma_1 \circ \sigma = \sigma' \circ \sigma_2$. Therefore,

$$\Delta \cup \Delta_1' \vdash \sigma_1\alpha_1 <: \sigma'\sigma_2\alpha_2$$

which implies

$$\Delta \cup \Delta_1' \vdash \alpha_1' <: \sigma'\alpha_2'$$

therefore

$$\Delta \vdash \Xi_1' \lesssim \Xi_2' \text{ using } \sigma'$$

□

Lemma C.5. *If* $\Delta \vdash \Xi_1 \lesssim \Xi_2$ using σ_1 *and* $\Delta \vdash \Xi_2 \lesssim \Xi_3$ using σ_2, *then* $\Delta \vdash \Xi_1 \lesssim \Xi_3$ using $\sigma_1 \circ \sigma_2$

Proof. Let

$$\begin{aligned}
\Xi_1 &= \exists[\![\overline{\{\overline{\chi_1}\} <: P_1 <: \{\overline{\eta_1}\}}]\!]\alpha_1,\\
\Xi_2 &= \exists[\![\overline{\{\overline{\chi_2}\} <: P_2 <: \{\overline{\eta_2}\}}]\!]\alpha_2,\\
\Xi_3 &= \exists[\![\overline{\{\overline{\chi_3}\} <: P_3 <: \{\overline{\eta_3}\}}]\!]\alpha_3,\\
\Delta_1 &= \left\{\overline{\{\overline{\chi_1}\} <: P_1 <: \{\overline{\eta_1}\}}\right\},\\
\Delta_2 &= \left\{\overline{\{\overline{\chi_2}\} <: P_2 <: \{\overline{\eta_2}\}}\right\},
\end{aligned}$$

and $\sigma_1 = [\overline{\gamma_1/P_2}]$ and $\sigma_2 = [\overline{\gamma_2/P_3}]$. From $\Delta \vdash \Xi_2 \lesssim \Xi_3$ using σ_2, it follows that

$$\overline{\underline{\Delta \cup \Delta_1 \cup \Delta_2} \vdash \underline{\sigma_2}\left(\bigsqcup\{\overline{\chi_3}\}\right) <: \gamma_2} \text{ and } \overline{\underline{\Delta \cup \Delta_1 \cup \Delta_2} \vdash \gamma_2 <: \underline{\sigma_2}\left(\bigsqcap\{\overline{\eta_3}\}\right)}$$

From $\Delta \vdash \Xi_1 \lesssim \Xi_2$ using σ_1, it follows that

$$\Delta \cup \Delta_1 \vdash \sigma_1 \text{ on } \langle \overline{P_2} \rangle \text{ obeys } \left\langle \overline{\bigsqcup\{\overline{\chi_2}\}} \right\rangle \text{ and } \left\langle \overline{\bigsqcap\{\overline{\eta_2}\}} \right\rangle$$

Therefore by lemma C.2,

$$\overline{\underline{\Delta \cup \Delta_1} \vdash \underline{\sigma_1\sigma_2}\left(\bigsqcup\{\overline{\chi_3}\}\right) <: \underline{\sigma_1\gamma_2}} \text{ and } \overline{\underline{\Delta \cup \Delta_1} \vdash \underline{\sigma_1\gamma_2} <: \underline{\sigma_1\sigma_2}\left(\bigsqcap\{\overline{\eta_3}\}\right)}$$

therefore,

$$\Delta \cup \Delta_1 \vdash \sigma_1 \circ \sigma_2 \text{ on } \langle \overline{P_3} \rangle \text{ obeys } \left\langle \overline{\bigsqcup\{\overline{\chi_3}\}} \right\rangle \text{ and } \left\langle \overline{\bigsqcap\{\overline{\eta_3}\}} \right\rangle$$

From $\Delta \vdash \Xi_2 \lesssim \Xi_3$ using σ_2, it follows that

$$\Delta \cup \Delta_1 \cup \Delta_2 \vdash \alpha_2 <: \sigma_2\alpha_3$$

Since $\Delta \cup \Delta_1 \vdash \sigma_1$ on $\langle \overline{P_2} \rangle$ obeys $\left\langle \overline{\bigsqcup\{\overline{\chi_2}\}} \right\rangle$ and $\left\langle \overline{\bigsqcap\{\overline{\eta_2}\}} \right\rangle$

$$\Delta \cup \Delta_1 \vdash \sigma_1\alpha_2 <: \sigma_1\sigma_2\alpha_3$$

by lemma C.2. Since it follows that $\Delta \cup \Delta_1 \vdash \alpha_1 <: \sigma_1\alpha_2$ from $\Delta \vdash \Xi_1 \lesssim \Xi_2$ using σ_1,

$$\Delta \cup \Delta_1 \vdash \alpha_1 <: \sigma_1\sigma_2\alpha_3$$

by transitivity. Therefore

$$\Delta \vdash \Xi_1 \lesssim \Xi_3 \text{ using } \sigma_1 \circ \sigma_2$$

□

Lemma C.6. *If* $\Delta \vdash \Xi_1 \sqsubseteq \Xi_2$ *and* $\Delta \vdash \Xi_2 \sqsubseteq \Xi_3$*, then* $\Delta \vdash \Xi_1 \sqsubseteq \Xi_3$.

Proof. Let Ξ_1' and Ξ_2' be the existential types reduced from Ξ_1 and Ξ_2, respectively. From $\Delta \vdash \Xi_1 \sqsubseteq \Xi_2$, it follows that $\Delta \vdash \Xi_1' \lesssim \Xi_2$. By Lemma C.4, we have $\Delta \vdash \Xi_1' \lesssim \Xi_2'$, and since $\Delta \vdash \Xi_2 \sqsubseteq \Xi_3$ implies $\Delta \vdash \Xi_2' \lesssim \Xi_3$, we can conclude that $\Delta \vdash \Xi_1' \lesssim \Xi_3$ by Lemma C.5. Therefore, $\Delta \vdash \Xi_1 \sqsubseteq \Xi_3$. □

APPENDIX D PROOFS OF THEOREMS

Proof of Theorem 4.1

Proof. Let $\{\overline{(d', \{\overline{\sigma}\})}\} = \mathit{applicable}(\Delta, \{\overline{d}\}, \alpha)$ where $\{\overline{d}\} = \mathit{visible}(\Delta, n)$ and $\alpha \neq \texttt{Bottom}$. For the sake of brevity, let $\preceq$ relation represent $\mathit{moreSpecific}(\Delta)$. We now show that $\left(\{\overline{d'}\}, \preceq\right)$ forms a meet semilattice by showing that 1) it is a partially ordered set, and 2) the greatest lower bound of any pair resides in the set.
$\left(\{\overline{d'}\}, \preceq\right)$ forms a partially ordered set since $\preceq$ is:
(1) Clearly reflexive.
(2) Transitive by lemma C.6.
(3) Antisymmetric by No Duplicates Rule.
Next, we must show that for every $d_1, d_2 \in \{\overline{d'}\}$, there exists $d_0 \in \{d'\}$ such that $d_0 \preceq d_1$ and $d_0 \preceq d_2$, and if there exists $d_0' \in \{\overline{d'}\}$ such that $d_0' \preceq d_1$ and $d_0' \preceq d_2$ then $d_0' \preceq d_0$. Let

$$\mathit{dom}(d_1) = \exists[\![\overline{\{\overline{\chi_1}\} <: P_1 <: \{\overline{\eta_1}\}}]\!]\alpha_1 \text{ and } \mathit{dom}(d_2) = \exists[\![\overline{\{\overline{\chi_2}\} <: P_2 <: \{\overline{\eta_2}\}}]\!]\alpha_2$$

From

$$\Delta \vdash \exists[\![ ]\!]\alpha \sqsubseteq \mathit{dom}(d_1) \text{ using } \sigma_1 \text{ and } \Delta \vdash \exists[\![ ]\!]\alpha \sqsubseteq \mathit{dom}(d_2) \text{ using } \sigma_2$$

it follows that

$$\Delta \vdash \alpha <: \sigma_1\alpha_1 \text{ and } \Delta \vdash \alpha <: \sigma_2\alpha_2$$

which implies

$$\Delta \vdash \alpha <: \sigma_0(\alpha_1 \sqcap \alpha_2)$$

where $\sigma_0 = \sigma_1 \circ \sigma_2$, and from

$$\Delta \vdash \sigma_1 \text{ on } \langle \overline{P_1} \rangle \text{ obeys } \left\langle \overline{\bigsqcup\{\overline{\chi_1}\}} \right\rangle \text{ and } \left\langle \overline{\bigsqcap\{\overline{\eta_1}\}} \right\rangle$$
$$\Delta \vdash \sigma_2 \text{ on } \langle \overline{P_2} \rangle \text{ obeys } \left\langle \overline{\bigsqcup\{\overline{\chi_2}\}} \right\rangle \text{ and } \left\langle \overline{\bigsqcap\{\overline{\eta_2}\}} \right\rangle,$$

it follows that

$$\Delta \vdash \sigma_0 \text{ on } \langle \overline{P_1}, \overline{P_2} \rangle \text{ obeys } \left\langle \overline{\bigsqcup\{\overline{\chi_1}\}}, \overline{\bigsqcup\{\overline{\chi_2}\}} \right\rangle \text{ and } \left\langle \overline{\bigsqcap\{\overline{\eta_1}\}}, \overline{\bigsqcap\{\overline{\eta_2}\}} \right\rangle$$

Since $d_1, d_2 \in \{\overline{d}\}$, they must satisfy the Meet Rule. Clearly, [Meet-Triv] and [Meet-Excl] are not applicable. If they satisfy [Meet-Spec-Less] by $\Delta \vdash \mathit{dom}(d_1) \sqsubseteq \mathit{dom}(d_2)$, then $d_0 = d_1$ is the greatest lower bound. If they satisfy [Meet-Spec-Grt] by $\Delta \vdash \mathit{dom}(d_2) \sqsubseteq \mathit{dom}(d_1)$, then $d_0 = d_2$ is the greatest lower bound. If they satisfy [Meet-Third], then there exists $d_0 \in \{\overline{d}\}$ such that

$$\mathit{dom}(d_0) = \exists[\![\overline{\{\overline{\chi_1}\} <: P_1 <: \{\overline{\eta_1}\}}, \overline{\{\overline{\chi_2}\} <: P_2 <: \{\overline{\eta_2}\}}]\!](\alpha_1 \sqcap \alpha_2)$$

which implies

$$\Delta \vdash \mathit{dom}(d_0) \sqsubseteq \mathit{dom}(d_1) \text{ and } \Delta \vdash \mathit{dom}(d_0) \sqsubseteq \mathit{dom}(d_2)$$

Moreover, $d_0 \in \{\overline{d'}\}$ since

$$\Delta \vdash \alpha <: \sigma_0(\alpha_1 \sqcap \alpha_2)$$

and

$$\Delta \vdash \sigma_0 \text{ on } \langle \overline{P_1}, \overline{P_2} \rangle \text{ obeys } \left\langle \overline{\bigsqcup\{\overline{\chi_1}\}}, \overline{\bigsqcup\{\overline{\chi_2}\}} \right\rangle \text{ and } \left\langle \overline{\bigsqcap\{\overline{\eta_1}\}}, \overline{\bigsqcap\{\overline{\eta_2}\}} \right\rangle$$

Now, we show that d_0 is the greatest lower bound. Suppose there exists $d_0' \in \{\overline{d'}\}$ such that

$$dom(d_0') = \exists[\![\overline{\{\overline{\chi_0'}\} <: P_0' <: \{\overline{\eta_0'}\}}]\!]\alpha_0'$$

and

$$\Delta \vdash dom(d_0') \sqsubseteq dom(d_1) \text{ using } \sigma_1' \text{ and } \Delta \vdash dom(d_0') \sqsubseteq dom(d_2) \text{ using } \sigma_2'.$$

Then, for $\Xi_r = \exists[\![\overline{\{\overline{\chi_r'}\} <: P_r' <: \{\overline{\eta_r'}\}}]\!]\alpha_r'$ such that $\Delta \vdash dom(d_0') \xrightarrow{\equiv} \Xi_r$,

$$\Delta, \Delta_r' \vdash \alpha_r' <: \sigma_1'\alpha_1 \text{ and } \Delta, \Delta_r' \vdash \alpha_r' <: \sigma_2'\alpha_2$$

where $\Delta_r' = \{\overline{\{\overline{\chi_r'}\} <: P_r' <: \{\overline{\eta_r'}\}}\}$, which implies

$$\Delta, \Delta_r' \vdash \alpha_r' <: \sigma_0'(\alpha_1 \sqcap \alpha_2)$$

where $\sigma_0' = \sigma_1' \circ \sigma_2'$,therefore

$$\Delta \vdash dom(d_0') \sqsubseteq dom(d_0) \text{ using } \sigma_0'$$

Since for every pair of a partially ordered set $\left(\{\overline{d'}\}, \preceq\right)$ their greatest lower bound is also in $\{\overline{d'}\}$, it forms a finite meet semilattice. Now it is easy to show that $\{\overline{d'}\}$ has a single most specific declaration by simple straightforward induction. □

Proof of Lemma 5.1

Proof. If $\Psi \vdash _$ match $_\ _\ _$ bounds $\langle\,\rangle$ and $\langle\,\rangle$, the theorem holds vacuously. Therefore, we will prove by induction on whichever successful match rule was used:

Rule [M-Any-Whatever-Success]: Since $\Psi \vdash \sigma\mathsf{Any} \equiv \mathsf{Any}$, by applying lemma C.2 to the premise

$$\Psi \cup \{\overline{K}\} \vdash \mathsf{Any} \text{ and } \alpha \text{ compare } V$$

it follows that

$$\Psi \vdash \mathsf{Any} \text{ and } \sigma\alpha \text{ compare } V$$

✓

Rule [M-Whatever-Any-Success]: Since $\Psi \vdash \sigma\mathsf{Any} \equiv \mathsf{Any}$, it follows from the premise that

$$\Psi \vdash g \text{ and } \sigma\mathsf{Any} \text{ compare } V$$

✓

Rule [M-Param-In]: From the premises and the hypothesis

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l}\rangle \text{ and } \langle\overline{u}\rangle$$

it follows that $g = l_i = u_i = g_i'$. Therefore $\Psi \vdash g \equiv g_i'$ from which we can conclude that

$$\Psi \vdash g \text{ and } \sigma P_i \text{ compare } =$$

✓

Rule [M-Param-Co]: From the premise and the hypothesis

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l}\rangle \text{ and } \langle\overline{u}\rangle$$

it follows that $g = l_i$ and $\Psi \vdash l_i <: g_i'$ from which we can conclude that

$$\Psi \vdash g \text{ and } \sigma P_i \text{ compare } +$$

✓

Rule [M-Param-Contra]: From the premise and the hypothesis

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l}\rangle \text{ and } \langle\overline{u}\rangle$$

it follows that $u_i = g$ and $\Psi \vdash g_i' <: u_i$ from which we can conclude that

$$\Psi \vdash g \text{ and } \sigma P_i \text{ compare } -$$

✓

Rule [M-Arrow-Arrow]: Let $\left\langle \overline{\langle \overline{l''} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \left\langle \overline{\langle \overline{l} \rangle} \right\rangle, \left\langle \overline{\langle \overline{l'} \rangle} \right\rangle \right\rangle$ and $\left\langle \overline{\langle \overline{u''} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \left\langle \overline{\langle \overline{u} \rangle} \right\rangle, \left\langle \overline{\langle \overline{u'} \rangle} \right\rangle \right\rangle$.
Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l''_{\underline{i}}} \rangle \text{ and } \langle \overline{u''_{\underline{i}}} \rangle$$

for some $1 \le i \le \#\left(\overline{\langle \overline{l''} \rangle}\right) = \#\left(\overline{\langle \overline{u''} \rangle}\right)$, by lemma C.1, there exist $1 \le j \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)$ and $1 \le j' \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{j}}} \rangle \text{ and } \langle \overline{u_{\underline{j}}} \rangle \text{ and } \Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{j'}}} \rangle \text{ and } \langle \overline{u'_{\underline{j'}}} \rangle$$

Then we can infer inductively from the premises that

$$\Psi \vdash g \text{ and } \sigma\alpha \text{ compare } \mathit{flip}(V) \text{ and } \Psi \vdash g' \text{ and } \sigma\rho \text{ compare } V$$

from which we can conclude that

$$\Psi \vdash (g \to g') \text{ and } \sigma(\alpha \to \rho) \text{ compare } V$$

✓

Rule [M-Tuple-Tuple-Success]: Let $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$, $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$, and $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$. Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i}}} \rangle \text{ and } \langle \overline{u'_{\underline{i}}} \rangle$$

for some $1 \le i \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$, by lemma C.1, there exist $\overline{1 \le k \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)}^{1 \le j \le m}$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{jk_j}}} \rangle \text{ and } \langle \overline{u_{\underline{jk_j}}} \rangle$$

Then we can infer inductively from the premises that

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha \text{ compare } \underline{V}}$$

from which we can conclude that

$$\Psi \vdash (\overline{g}) \text{ and } \sigma(\overline{\alpha}) \text{ compare } V$$

✓

Rule [M-Union-In-Same]: Since $\bigsqcup\langle \overline{g} \rangle$ is a ground type, $\Psi \vdash \sigma\left(\bigsqcup\langle \overline{g} \rangle\right) \equiv \bigsqcup\langle \overline{g} \rangle$ from which we can conclude that

$$\Psi \vdash \bigsqcup\langle \overline{g} \rangle \text{ and } \sigma\left(\bigsqcup\langle \overline{g} \rangle\right) \text{ compare } =$$

✓

Rule [M-Union-In-Param]: From the premises and the hypothesis

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

it follows that $\bigsqcup\langle \overline{g} \rangle = l_i = u_i = g'_i$. Therefore $\Psi \vdash \bigsqcup\langle \overline{g} \rangle \equiv g'_i$ from which we can conclude that

$$\Psi \vdash \bigsqcup\langle \overline{g} \rangle \text{ and } \sigma P_i \text{ compare } =$$

✓

Rule [M-Union-Whatever-Co]: Let $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$, $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$, and $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$. Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i}}} \rangle \text{ and } \langle \overline{u'_{\underline{i}}} \rangle$$

for some $1 \le i \le \#(\overline{\langle \overline{l'} \rangle}) = \#(\overline{\langle \overline{u'} \rangle})$, by lemma C.1, there exist $\overline{1 \le k \le \#(\overline{\langle \overline{l} \rangle}) = \#(\overline{\langle \overline{u} \rangle})}^{1 \le j \le m}$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{jk_j}}} \rangle \text{ and } \langle \overline{u_{\underline{jk_j}}} \rangle$$

Then we can infer inductively from the premises that

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma\alpha} \text{ compare } +}$$

from which we can conclude that

$$\Psi \vdash \bigsqcup\langle \overline{g} \rangle \text{ and } \sigma\alpha \text{ compare } +$$

✓

Rule [M-Union-Whatever-Contra]: Let $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \left\langle \overline{\langle \overline{l}^{\,k} \rangle}^{\,j} \right\rangle^{i}$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \left\langle \overline{\langle \overline{u}^{\,k} \rangle}^{\,j} \right\rangle^{i}$. Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i'}}} \rangle \text{ and } \langle \overline{u'_{\underline{i'}}} \rangle$$

for some $1 \le i' \le \#(\overline{\langle \overline{l'} \rangle}) = \#(\overline{\langle \overline{u'} \rangle})$,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{ij}}} \rangle \text{ and } \langle \overline{u_{\underline{ij}}} \rangle$$

for some $1 \le i \le \#(\overline{\langle \overline{l} \rangle}) = \#(\overline{\langle \overline{u} \rangle})$ and $1 \le j \le \#(\overline{\langle \overline{l_{\underline{i}}} \rangle}) = \#(\overline{\langle \overline{u_{\underline{i}}} \rangle})$. Then we can infer inductively from the i-th match premise that

$$\Psi \vdash g_i \text{ and } \sigma\alpha \text{ compare } -$$

from which we can conclude that

$$\Psi \vdash \bigsqcup\langle \overline{g} \rangle \text{ and } \sigma\alpha \text{ compare } -$$

✓

Rule [M-Intersection-In-Same]: Since $\bigsqcap\langle \overline{g} \rangle$ is a ground type, $\Psi \vdash \sigma\left(\bigsqcap\langle \overline{g} \rangle\right) \equiv \bigsqcap\langle \overline{g} \rangle$ from which we can conclude that

$$\Psi \vdash \bigsqcap\langle \overline{g} \rangle \text{ and } \sigma\left(\bigsqcap\langle \overline{g} \rangle\right) \text{ compare } =$$

✓

Rule [M-Intersection-In-Param]: From the premises and the hypothesis

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

it follows that $\bigsqcap\langle \overline{g} \rangle = l_i = u_i = g'_i$. Therefore $\Psi \vdash \bigsqcap\langle \overline{g} \rangle \equiv g'_i$ from which we can conclude that

$$\Psi \vdash \bigsqcap\langle \overline{g} \rangle \text{ and } \sigma P_i \text{ compare } =$$

✓

Rule [M-Intersection-Whatever-Co]: Let $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \left\langle \overline{\langle \overline{l}^{\,k} \rangle}^{\,j} \right\rangle^{i}$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \left\langle \overline{\langle \overline{u}^{\,k} \rangle}^{\,j} \right\rangle^{i}$. Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i'}}} \rangle \text{ and } \langle \overline{u'_{\underline{i'}}} \rangle$$

for some $1 \le i' \le \#(\overline{\langle \overline{l'} \rangle}) = \#(\overline{\langle \overline{u'} \rangle})$,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{ij}}} \rangle \text{ and } \langle \overline{u_{\underline{ij}}} \rangle$$

for some $1 \le i \le \#(\overline{\langle \overline{l} \rangle}) = \#(\overline{\langle \overline{u} \rangle})$ and $1 \le j \le \#(\overline{\langle \overline{l_{\underline{i}}} \rangle}) = \#(\overline{\langle \overline{u_{\underline{i}}} \rangle})$. Then we can infer inductively from the i-th match premise that

$$\Psi \vdash g_i \text{ and } \sigma\alpha \text{ compare } +$$

from which we can conclude that

$$\Psi \vdash \bigsqcap\langle \overline{g} \rangle \text{ and } \sigma\alpha \text{ compare } +$$

✓

Rule [M-Intersection-Whatever-Contra]: Let $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$, $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$, and $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$. Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_i} \rangle \text{ and } \langle \overline{u'_i} \rangle$$

for some $1 \le i \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$, by lemma C.1, there exist $\overline{1 \le k \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)}^{1 \le j \le m}$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{jk_j}} \rangle \text{ and } \langle \overline{u_{jk_j}} \rangle$$

Then we can infer inductively from the premises that

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma\alpha} \text{ compare } -}$$

from which we can conclude that

$$\Psi \vdash \bigsqcap \langle \overline{g} \rangle \text{ and } \sigma\alpha \text{ compare } -$$

✓

Rule [M-Whatever-Union-Success]: From the premise

$$\Psi \cup \{\overline{K}\} \vdash g \text{ and } \bigsqcup \langle \overline{\alpha} \rangle \text{ compare } V$$

it follows by Lemma C.2 that

$$\Delta \vdash \sigma g \text{ and } \sigma\left(\bigsqcup \langle \overline{\alpha} \rangle\right) \text{ compare } V$$

therefore

$$\Psi \vdash g \text{ and } \sigma\left(\bigsqcup \langle \overline{\alpha} \rangle\right) \text{ compare } V$$

✓

Rule [M-Union-Union-Success]: From the premise

$$\Psi \cup \{\overline{K}\} \vdash \bigsqcup \langle \overline{g} \rangle \text{ and } \bigsqcup \langle \overline{\alpha} \rangle \text{ compare } -$$

it follows by Lemma C.2 that

$$\Delta \vdash \sigma\left(\bigsqcup \langle \overline{g} \rangle\right) \text{ and } \sigma\left(\bigsqcup \langle \overline{\alpha} \rangle\right) \text{ compare } -$$

therefore

$$\Psi \vdash \bigsqcup \langle \overline{g} \rangle \text{ and } \sigma\left(\bigsqcup \langle \overline{\alpha} \rangle\right) \text{ compare } -$$

✓

Rule [M-Whatever-Intersection-Success]: From the premise

$$\Psi \cup \{\overline{K}\} \vdash g \text{ and } \bigsqcap \langle \overline{\alpha} \rangle \text{ compare } V$$

it follows by Lemma C.2 that

$$\Delta \vdash \sigma g \text{ and } \sigma\left(\bigsqcap \langle \overline{\alpha} \rangle\right) \text{ compare } V$$

therefore

$$\Psi \vdash g \text{ and } \sigma\left(\bigsqcap \langle \overline{\alpha} \rangle\right) \text{ compare } V$$

✓

Rule [M-Union-Intersection-Success]: From the premise

$$\Psi \cup \{\overline{K}\} \vdash \bigsqcup \langle \overline{g} \rangle \text{ and } \bigsqcap \langle \overline{\alpha} \rangle \text{ compare } -$$

it follows by Lemma C.2 that

$$\Delta \vdash \sigma\left(\bigsqcup \langle \overline{g} \rangle\right) \text{ and } \sigma\left(\bigsqcap \langle \overline{\alpha} \rangle\right) \text{ compare } -$$

therefore

$$\Psi \vdash \bigsqcup \langle \overline{g} \rangle \text{ and } \sigma\left(\bigsqcap \langle \overline{\alpha} \rangle\right) \text{ compare } -$$

✓

Rule [M-Intersection-Intersection-Success]: From the premise

$$\Psi \cup \{\overline{K}\} \vdash \bigsqcap\langle \overline{g} \rangle \text{ and } \bigsqcap\langle \overline{\alpha} \rangle \text{ compare } -$$

it follows by Lemma C.2 that

$$\Delta \vdash \sigma\big(\bigsqcap\langle \overline{g} \rangle\big) \text{ and } \sigma\big(\bigsqcap\langle \overline{\alpha} \rangle\big) \text{ compare } -$$

therefore

$$\Psi \vdash \bigsqcap\langle \overline{g} \rangle \text{ and } \sigma\big(\bigsqcap\langle \overline{\alpha} \rangle\big) \text{ compare } -$$

✓

Rule [M-Trait-In-Success]: Let $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$, $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$, and $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$. Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i}}} \rangle \text{ and } \langle \overline{u'_{\underline{i}}} \rangle$$

for some $1 \le i \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$, by lemma C.1, there exist $\overline{1 \le k \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)}^{1 \le j \le m}$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{jk_{\underline{j}}}} \rangle \text{ and } \langle \overline{u_{jk_{\underline{j}}}} \rangle$$

Then we can infer inductively from the premises that

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha \text{ compare } =}$$

from which we can conclude that

$$\Psi \vdash T[\![\overline{g}]\!] \text{ and } \sigma T[\![\overline{\alpha}]\!] \text{ compare } =$$

✓

Rule [M-Trait-Co]: Let $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$, $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$, and $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$. Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i}}} \rangle \text{ and } \langle \overline{u'_{\underline{i}}} \rangle$$

for some $1 \le i \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$, by lemma C.1, there exist $\overline{1 \le k \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)}^{1 \le j \le m}$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{jk_{\underline{j}}}} \rangle \text{ and } \langle \overline{u_{jk_{\underline{j}}}} \rangle$$

Then we can infer inductively from the premises that

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha \text{ compare } V'}$$

from which we can conclude that

$$\Psi \vdash T[\![\overline{g}]\!] \text{ and } \sigma T[\![\overline{\alpha}]\!] \text{ compare } +$$

Since $\Psi \vdash G$ and $T[\![\overline{g}]\!]$ compare +,

$$\Psi \vdash G \text{ and } \sigma T[\![\overline{\alpha}]\!] \text{ compare } +$$

✓

Rule [M-Trait-Contra]: Let $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$, $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$, and $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$. Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i}}} \rangle \text{ and } \langle \overline{u'_{\underline{i}}} \rangle$$

for some $1 \le i \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$, by lemma C.1, there exist $\overline{1 \le k \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)}^{1 \le j \le m}$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{jk_j}}} \rangle \text{ and } \langle \overline{u_{\underline{jk_j}}} \rangle$$

Then we can infer inductively from the premises that

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha \text{ compare } \mathit{flip}(V')}$$

from which we can conclude that

$$\Psi \vdash T[\![\overline{g}]\!] \text{ and } \sigma T[\![\overline{\alpha}]\!] \text{ compare } -$$

By applying lemma C.2 to the premise,

$$\Psi \vdash \sigma T[\![\overline{\alpha}]\!] \text{ and } \sigma T'[\![\overline{\alpha'}]\!] \text{ compare } -$$

therefore

$$\Psi \vdash T[\![\overline{g}]\!] \text{ and } \sigma T'[\![\overline{\alpha'}]\!] \text{ compare } -$$

✓

Rule [M-Object-Object-Same]: Let $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\langle \overline{\langle \overline{l} \rangle} \rangle} \right\rangle$, $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\langle \overline{\langle \overline{u} \rangle} \rangle} \right\rangle$, and $m = \#\left(\overline{\langle \overline{\langle \overline{l} \rangle} \rangle}\right) = \#\left(\overline{\langle \overline{\langle \overline{u} \rangle} \rangle}\right)$. Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i}}} \rangle \text{ and } \langle \overline{u'_{\underline{i}}} \rangle$$

for some $1 \le i \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$, by lemma C.1, there exist $\overline{1 \le k \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)}^{1 \le j \le m}$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{jk_j}}} \rangle \text{ and } \langle \overline{u_{\underline{jk_j}}} \rangle$$

Then we can infer inductively from the premises that

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha \text{ compare } =}$$

from which we can conclude that

$$\Psi \vdash O[\![\overline{g}]\!] \text{ and } \sigma O[\![\overline{\alpha}]\!] \text{ compare } =$$

which implies

$$\Psi \vdash O[\![\overline{g}]\!] \text{ and } \sigma O[\![\overline{\alpha}]\!] \text{ compare } V$$

✓

Rule [M-Trait-Object-Contra]: Let $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\langle \overline{\langle \overline{l} \rangle} \rangle} \right\rangle$, $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\langle \overline{\langle \overline{u} \rangle} \rangle} \right\rangle$, and $m = \#\left(\overline{\langle \overline{\langle \overline{l} \rangle} \rangle}\right) = \#\left(\overline{\langle \overline{\langle \overline{u} \rangle} \rangle}\right)$. Since

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i}}} \rangle \text{ and } \langle \overline{u'_{\underline{i}}} \rangle$$

for some $1 \le i \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$, by lemma C.1, there exist $\overline{1 \le k \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)}^{1 \le j \le m}$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{jk_j}}} \rangle \text{ and } \langle \overline{u_{\underline{jk_j}}} \rangle$$

Then we can infer inductively from the premises that

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha' \text{ compare } \mathit{flip}(V')}$$

from which we can conclude that

$$\Psi \vdash T[\![\overline{g}]\!] \text{ and } \sigma T[\![\overline{\alpha'}]\!] \text{ compare } -$$

By applying lemma C.2 to the premise,

$$\Psi \vdash \sigma T[\![\overline{\alpha'}]\!] \text{ and } \sigma O[\![\overline{\alpha}]\!] \text{ compare } -$$

therefore

$$\Psi \vdash T[\![\overline{g}]\!] \text{ and } \sigma O[\![\overline{\alpha}]\!] \text{ compare } -$$

✓

□

Proof of Lemma 5.2

Proof. By induction on the structure of g and α:

Case 1 $g = \mathsf{Any}$ and α is none of the following: a type parameter, Any, an intersection type, a union type :

$\Psi \vdash \mathsf{Any}$ and $\sigma\alpha$ compare V holds only when $V = -$. Let

$$\overline{l = \mathtt{Bottom}}^{\#(\overline{K})} \text{ and } \overline{u = \mathsf{Any}}^{\#(\overline{K})}$$

Then since $\Psi \cup \{\overline{K}\} \vdash \mathsf{Any}$ and α compare $-$, by [M-Any-Whatever-Success],

$$\Psi \vdash \mathsf{Any} \text{ match } [\![\overline{K}]\!]\ \alpha - \text{bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle$$

and

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

✓

Case 2 g is none of the following: an intersection type, a union type, and $\alpha = \mathsf{Any}$:

$\Psi \vdash g$ and $\sigma\mathsf{Any}$ compare V implies $\Psi \cup \{\overline{K}\} \vdash g$ and Any compare V. Let

$$\overline{l = \mathtt{Bottom}}^{\#(\overline{K})} \text{ and } \overline{u = \mathsf{Any}}^{\#(\overline{K})}$$

Then by [M-Whatever-Any-Success],

$$\Psi \vdash g \text{ match } [\![\overline{K}]\!]\ \mathsf{Any} + \text{bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle$$

and

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

✓

Case 3 g is none of the following: an intersection type, a union type, and $\alpha = P_i$:

If $V = =$, let

$$\overline{l = (\text{if } (P = \underline{P_i}) \text{ then } \underline{g} \text{ else } \mathtt{Bottom})}^{\#(\overline{K})} \text{ and } \overline{u = (\text{if } (P = \underline{P_i}) \text{ then } \underline{g} \text{ else } \mathsf{Any})}^{\#(\overline{K})}$$

Then by [M-Param-In],

$$\Psi \vdash g \text{ match } [\![\overline{K}]\!]\ P_i = \text{bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle$$

and since $\Psi \vdash g$ and σP_i compare $=$,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

If $V = +$, let

$$\overline{l = (\text{if } (P = \underline{P_i}) \text{ then } \underline{g} \text{ else } \mathtt{Bottom})}^{\#(\overline{K})} \text{ and } \overline{u = \mathsf{Any}}^{\#(\overline{K})}$$

Then by [M-Param-Co],

$$\Psi \vdash g \text{ match } [\![\overline{K}]\!]\ P_i + \text{bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle$$

and since $\Psi \vdash g$ and σP_i compare $+$,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

If $V = -$, let

$$\overline{l = \mathtt{Bottom}}^{\#(\overline{K})} \text{ and } \overline{u = (\text{if } (P = \underline{P_i}) \text{ then } \underline{g} \text{ else } \mathsf{Any})}^{\#(\overline{K})}$$

Then by [M-Parma-Contra],

$$\Psi \vdash g \text{ match } [\![\overline{K}]\!] \; P_i \text{ - bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle$$

and since $\Psi \vdash g$ and σP_i compare -,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

✓

Case 4 $g = (g_1 \to g_2)$ and $\alpha = (\alpha_1 \to \alpha_2)$:

Since $\Psi \vdash (g_1 \to g_2)$ and $\sigma(\alpha_1 \to \alpha_2)$ compare V,

$$\Psi \vdash g_1 \text{ and } \sigma\alpha_1 \text{ compare } \mathit{flip}(V) \text{ and } \Psi \vdash g_2 \text{ and } \sigma\alpha_2 \text{ compare } V$$

from which we can infer inductively that

$$\Psi \vdash g_1 \text{ match } [\![\overline{K}]\!] \; \alpha_1 \; \mathit{flip}(V) \text{ bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle$$
$$\Psi \vdash g_2 \text{ match } [\![\overline{K}]\!] \; \alpha_2 \; V \text{ bounds } \left\langle \overline{\langle \overline{l'} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u'} \rangle} \right\rangle$$

and

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{j}}} \rangle \text{ and } \langle \overline{u_{\underline{j}}} \rangle \text{ and } \Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{k}}} \rangle \text{ and } \langle \overline{u'_{\underline{k}}} \rangle$$

for some $1 \le j \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)$ and some $1 \le k \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$. Therefore, by [M-Arrow-Arrow],

$$\Psi \vdash (g_1 \to g_2) \text{ match } [\![\overline{K}]\!] \; (\alpha_1 \to \alpha_2) \; V \text{ bounds } \bigsqcup^{\times}_{\overline{K}} \left\langle \left\langle \overline{\langle \overline{l} \rangle} \right\rangle, \left\langle \overline{\langle \overline{l'} \rangle} \right\rangle \right\rangle \text{ and } \bigsqcap^{\times}_{\overline{K}} \left\langle \left\langle \overline{\langle \overline{u} \rangle} \right\rangle, \left\langle \overline{\langle \overline{u'} \rangle} \right\rangle \right\rangle$$

and by lemma C.1, there exists $1 \le i \le \#\left(\overline{\langle \overline{l''} \rangle}\right) = \#\left(\overline{\langle \overline{u''} \rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l''_{\underline{i}}} \rangle \text{ and } \langle \overline{u''_{\underline{i}}} \rangle$$

where $\left\langle \overline{\langle \overline{l''} \rangle} \right\rangle = \bigsqcup^{\times}_{\overline{K}} \left\langle \left\langle \overline{\langle \overline{l} \rangle} \right\rangle, \left\langle \overline{\langle \overline{l'} \rangle} \right\rangle \right\rangle$ and $\left\langle \overline{\langle \overline{u''} \rangle} \right\rangle = \bigsqcap^{\times}_{\overline{K}} \left\langle \left\langle \overline{\langle \overline{u} \rangle} \right\rangle, \left\langle \overline{\langle \overline{u'} \rangle} \right\rangle \right\rangle$. ✓

Case 5 $g = (\overline{g})$ and $\alpha = (\overline{\alpha})$:

Since $\Psi \vdash (\overline{g})$ and $\sigma(\overline{\alpha})$ compare V,

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha \text{ compare } \underline{V}}$$

from which we can infer inductively that

$$\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]} \; \alpha \; \underline{V} \text{ bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle}$$

and there exist $\overline{1 \le k \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)}^{1 \le j \le m}$ where $m = \#\left(\left\langle \overline{\langle \overline{l} \rangle} \right\rangle\right) = \#\left(\left\langle \overline{\langle \overline{u} \rangle} \right\rangle\right)$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{jk_j}}} \rangle \text{ and } \langle \overline{u_{\underline{jk_j}}} \rangle$$

Therefore, by [M-Tuple-Tuple],

$$\Psi \vdash (\overline{g}) \text{ match } [\![\overline{K}]\!] \; (\overline{\alpha}) \; V \text{ bounds } \bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\langle \overline{l} \rangle} \rangle} \right\rangle \text{ and } \bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\langle \overline{u} \rangle} \rangle} \right\rangle$$

and by lemma C.1, there exists $1 \le i \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i}}} \rangle \text{ and } \langle \overline{u'_{\underline{i}}} \rangle$$

where $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\langle \overline{l} \rangle} \rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap^{\times}_{\overline{K}} \left\langle \overline{\langle \overline{\langle \overline{u} \rangle} \rangle} \right\rangle$. ✓

Case 6 $g = \bigsqcup\langle \overline{g} \rangle$, $\alpha = \bigsqcup\langle \overline{g} \rangle$, and $V = =$:

Let

$$\overline{l = \texttt{Bottom}}^{\#(\overline{K})} \text{ and } \overline{u = \texttt{Any}}^{\#(\overline{K})}$$

Then by [M-Union-In-Same],

$$\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \bigsqcup\langle\overline{g}\rangle = \text{bounds } \left\langle\langle\overline{l}\rangle\right\rangle \text{ and } \left\langle\langle\overline{u}\rangle\right\rangle$$

and

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l}\rangle \text{ and } \langle\overline{u}\rangle$$

✓

Case 7 $g = \bigsqcup\langle\overline{g}\rangle$, $\alpha = P_i$, and $V = =$:

Let

$$\overline{l = (\text{if } (P = \underline{P_i}) \text{ then } \underline{\bigsqcup\langle\overline{g}\rangle} \text{ else } \texttt{Bottom})}^{\#(\overline{K})} \text{ and } \overline{u = (\text{if } (P = \underline{P_i}) \text{ then } \underline{\bigsqcup\langle\overline{g}\rangle} \text{ else } \texttt{Any})}^{\#(\overline{K})}$$

Then by [M-Union-In-Param],

$$\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \; P_i = \text{bounds } \left\langle\langle\overline{l}\rangle\right\rangle \text{ and } \left\langle\langle\overline{u}\rangle\right\rangle$$

and since $\Psi \vdash \bigsqcup\langle\overline{g}\rangle$ and σP_i compare $=$,

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l}\rangle \text{ and } \langle\overline{u}\rangle$$

✓

Case 8 $g = \bigsqcup\langle\overline{g}\rangle$, α is not an intersection type, and $V = +$:

Since $\Psi \vdash \bigsqcup\langle\overline{g}\rangle$ and $\sigma\alpha$ compare $+$,

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma\alpha} \text{ compare } +}$$

from which we can infer inductively that

$$\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]} \; \underline{\alpha} + \text{bounds } \left\langle\overline{\langle\overline{l}\rangle}\right\rangle \text{ and } \left\langle\overline{\langle\overline{u}\rangle}\right\rangle}$$

and there exist $\overline{1 \le k \le \#\left(\overline{\langle\overline{l}\rangle}\right) = \#\left(\overline{\langle\overline{u}\rangle}\right)}^{1 \le j \le m}$ where $m = \#\left(\overline{\left\langle\overline{\langle\overline{l}\rangle}\right\rangle}\right) = \#\left(\overline{\left\langle\overline{\langle\overline{u}\rangle}\right\rangle}\right)$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l_{\underline{jk_j}}}\rangle \text{ and } \langle\overline{u_{\underline{jk_j}}}\rangle$$

Therefore, by [M-Union-Whatever-Co],

$$\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \; \alpha + \text{bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle\overline{\langle\overline{\langle\overline{l}\rangle}\rangle}\right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle\overline{\langle\overline{\langle\overline{u}\rangle}\rangle}\right\rangle$$

and by lemma C.1, there exists $1 \le i \le \#\left(\overline{\langle\overline{l'}\rangle}\right) = \#\left(\overline{\langle\overline{u'}\rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l'_{\underline{i}}}\rangle \text{ and } \langle\overline{u'_{\underline{i}}}\rangle$$

where $\left\langle\overline{\langle\overline{l'}\rangle}\right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle\overline{\langle\overline{\langle\overline{l}\rangle}\rangle}\right\rangle$ and $\left\langle\overline{\langle\overline{u'}\rangle}\right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle\overline{\langle\overline{\langle\overline{u}\rangle}\rangle}\right\rangle$. ✓

Case 9 $g = \bigsqcup\langle\overline{g}\rangle$, α is none of the following: an intersectiontype, a union type, and $V = -$:

Since $\Psi \vdash \bigsqcup\langle\overline{g}\rangle$ and $\sigma\alpha$ compare $-$,

$$\Psi \vdash g_j \text{ and } \sigma\alpha \text{ compare } -$$

for some $1 \le j \le \#(\overline{g})$. We can infer inductively that

$$\Psi \vdash g_j \text{ match } [\![\overline{K}]\!] \; \alpha - \text{bounds } \left\langle\overline{\langle\overline{l}\rangle}\right\rangle \text{ and } \left\langle\overline{\langle\overline{u}\rangle}\right\rangle$$

and there exists $1 \le k \le \#\left(\overline{\langle\overline{l}\rangle}\right) = \#\left(\overline{\langle\overline{u}\rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l_{\underline{k}}}\rangle \text{ and } \langle\overline{u_{\underline{k}}}\rangle$$

Therefore, by [M-Union-Whatever-Contra],

$$\Psi \vdash \bigsqcup\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \; \alpha - \text{bounds } \left\langle\overline{\langle\overline{l'}\rangle}\right\rangle \text{ and } \left\langle\overline{\langle\overline{u'}\rangle}\right\rangle$$

and there exists $1 \le i \le \#\left(\overline{\langle\overline{l'}\rangle}\right) = \#\left(\overline{\langle\overline{u'}\rangle}\right)$ such that $\langle\overline{l'_{\underline{i}}}\rangle = \langle\overline{l_{\underline{k}}}\rangle$ and $\langle\overline{u'_{\underline{i}}}\rangle = \langle\overline{u_{\underline{k}}}\rangle$ which implies

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l'_{\underline{i}}}\rangle \text{ and } \langle\overline{u'_{\underline{i}}}\rangle$$

✓

Case 10 $g = \bigsqcap\langle\overline{g}\rangle$, $\alpha = \bigsqcap\langle\overline{g}\rangle$, and $V = =$:
Let

$$\overline{l = \texttt{Bottom}}^{\#(\overline{K})} \text{ and } \overline{u = \texttt{Any}}^{\#(\overline{K})}$$

Then by [M-Intersection-In-Same],

$$\Psi \vdash \bigsqcap\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \bigsqcap\langle\overline{g}\rangle = \text{bounds } \left\langle\langle\overline{l}\rangle\right\rangle \text{ and } \left\langle\langle\overline{u}\rangle\right\rangle$$

and

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l}\rangle \text{ and } \langle\overline{u}\rangle$$

✓

Case 11 $g = \bigsqcap\langle\overline{g}\rangle$, $\alpha = P_i$, and $V = =$:
Let

$$\overline{l = (\text{if } (P = \underline{P_i}) \text{ then } \underline{\bigsqcap\langle\overline{g}\rangle} \text{ else } \texttt{Bottom})}^{\#(\overline{K})} \text{ and } \overline{u = (\text{if } (P = \underline{P_i}) \text{ then } \underline{\bigsqcap\langle\overline{g}\rangle} \text{ else Any})}^{\#(\overline{K})}$$

Then by [M-Intersection-In-Param],

$$\Psi \vdash \bigsqcap\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] P_i = \text{bounds } \left\langle\langle\overline{l}\rangle\right\rangle \text{ and } \left\langle\langle\overline{u}\rangle\right\rangle$$

and since $\Psi \vdash \bigsqcap\langle\overline{g}\rangle$ and σP_i compare $=$,

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l}\rangle \text{ and } \langle\overline{u}\rangle$$

✓

Case 12 $g = \bigsqcap\langle\overline{g}\rangle$, α is not an intersection type, and $V = +$:
Since $\Psi \vdash \bigsqcap\langle\overline{g}\rangle$ and $\sigma\alpha$ compare $+$,

$$\Psi \vdash g_j \text{ and } \sigma\alpha \text{ compare } +$$

for some $1 \le j \le \#(\overline{g})$. We can infer inductively that

$$\Psi \vdash g_j \text{ match } [\![\overline{K}]\!] \alpha + \text{bounds } \left\langle\overline{\langle\overline{l}\rangle}\right\rangle \text{ and } \left\langle\overline{\langle\overline{u}\rangle}\right\rangle$$

and there exists $1 \le k \le \#\left(\overline{\langle\overline{l}\rangle}\right) = \#\left(\overline{\langle\overline{u}\rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l_{\underline{k}}}\rangle \text{ and } \langle\overline{u_{\underline{k}}}\rangle$$

Therefore, by [M-Intersection-Whatever-Co],

$$\Psi \vdash \bigsqcap\langle\overline{g}\rangle \text{ match } [\![\overline{K}]\!] \alpha + \text{bounds } \left\langle\overline{\langle\overline{l'}\rangle}\right\rangle \text{ and } \left\langle\overline{\langle\overline{u'}\rangle}\right\rangle$$

and there exists $1 \le i \le \#\left(\overline{\langle\overline{l'}\rangle}\right) = \#\left(\overline{\langle\overline{u'}\rangle}\right)$ such that $\langle\overline{l'_{\underline{i}}}\rangle = \langle\overline{l_{\underline{k}}}\rangle$ and $\langle\overline{u'_{\underline{i}}}\rangle = \langle\overline{u_{\underline{k}}}\rangle$ which implies

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l'_{\underline{i}}}\rangle \text{ and } \langle\overline{u'_{\underline{i}}}\rangle$$

✓

Case 13 $g = \bigsqcap\langle\overline{g}\rangle$, and $V = -$:
Since $\Psi \vdash \bigsqcap\langle\overline{g}\rangle$ and $\sigma\alpha$ compare $-$,

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma\alpha} \text{ compare } -}$$

from which we can infer inductively that

$$\frac{\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]} \ \underline{\alpha} - \text{bounds } \left\langle\overline{\langle\overline{l}\rangle}\right\rangle \text{ and } \left\langle\overline{\langle\overline{u}\rangle}\right\rangle}}{1 \le k \le \#\left(\overline{\langle\overline{l}\rangle}\right) = \#\left(\overline{\langle\overline{u}\rangle}\right)}^{1 \le j \le m}$$

where $m = \#\left(\left\langle\overline{\langle\overline{l}\rangle}\right\rangle\right) = \#\left(\left\langle\overline{\langle\overline{u}\rangle}\right\rangle\right)$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l_{\underline{jk_j}}}\rangle \text{ and } \langle\overline{u_{\underline{jk_j}}}\rangle$$

Therefore, by [M-Intersection-Whatever-Contra],

$$\Psi \vdash \bigsqcap\langle \overline{g} \rangle \text{ match } [\![\overline{K}]\!] \ \alpha \text{ - bounds } \bigsqcup\nolimits^{\times}_{\overline{K}} \left\langle \overline{\langle \langle \overline{l} \rangle \rangle} \right\rangle \text{ and } \bigsqcap\nolimits^{\times}_{\overline{K}} \left\langle \overline{\langle \langle \overline{u} \rangle \rangle} \right\rangle$$

and by lemma C.1, there exists $1 \leq i \leq \#\big(\overline{\langle \overline{l'} \rangle}\big) = \#\big(\overline{\langle \overline{u'} \rangle}\big)$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_i} \rangle \text{ and } \langle \overline{u'_i} \rangle$$

where $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup\nolimits^{\times}_{\overline{K}} \left\langle \overline{\langle \langle \overline{l} \rangle \rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap\nolimits^{\times}_{\overline{K}} \left\langle \overline{\langle \langle \overline{u} \rangle \rangle} \right\rangle$. ✓

Case 14 g is none of the following: an intersection type, a union type, and $\alpha = \bigsqcup\langle \overline{\alpha} \rangle$:

Since $\Psi \vdash g$ and $\sigma\big(\bigsqcup\langle \overline{\alpha} \rangle\big)$ compare V and $FV(\overline{\alpha}) \cap \{\overline{P}\} = \emptyset$,

$$\Psi \cup \{\overline{K}\} \vdash g \text{ and } \bigsqcup\langle \overline{\alpha} \rangle \text{ compare } V$$

Let

$$\overline{l = \texttt{Bottom}}^{\#(\overline{K})} \text{ and } \overline{u = \texttt{Any}}^{\#(\overline{K})}$$

Then, by [M-Whatever-Union-Success],

$$\Psi \vdash g \text{ match } [\![\overline{K}]\!] \bigsqcup\langle \overline{\alpha} \rangle \ V \text{ bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle$$

and

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

✓

Case 15 $g = \bigsqcup\langle \overline{g} \rangle$, $\alpha = \bigsqcup\langle \overline{\alpha} \rangle$, and $V = -$:

Since $\Psi \vdash \bigsqcup\langle \overline{g} \rangle$ and $\sigma\big(\bigsqcup\langle \overline{\alpha} \rangle\big)$ compare - and $FV(\overline{\alpha}) \cap \{\overline{P}\} = \emptyset$,

$$\Psi \cup \{\overline{K}\} \vdash \bigsqcup\langle \overline{g} \rangle \text{ and } \bigsqcup\langle \overline{\alpha} \rangle \text{ compare } -$$

Let

$$\overline{l = \texttt{Bottom}}^{\#(\overline{K})} \text{ and } \overline{u = \texttt{Any}}^{\#(\overline{K})}$$

Then, by [M-Union-Union-Success],

$$\Psi \vdash \bigsqcup\langle \overline{g} \rangle \text{ match } [\![\overline{K}]\!] \bigsqcup\langle \overline{\alpha} \rangle \text{ - bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle$$

and

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

✓

Case 16 g is none of the following: an intersection type, a union type, and $\alpha = \bigsqcap\langle \overline{\alpha} \rangle$:

Since $\Psi \vdash g$ and $\sigma\big(\bigsqcap\langle \overline{\alpha} \rangle\big)$ compare V and $FV(\overline{\alpha}) \cap \{\overline{P}\} = \emptyset$,

$$\Psi \cup \{\overline{K}\} \vdash g \text{ and } \bigsqcap\langle \overline{\alpha} \rangle \text{ compare } V$$

Let

$$\overline{l = \texttt{Bottom}}^{\#(\overline{K})} \text{ and } \overline{u = \texttt{Any}}^{\#(\overline{K})}$$

Then, by [M-Whatever-Intersection-Success],

$$\Psi \vdash g \text{ match } [\![\overline{K}]\!] \bigsqcap\langle \overline{\alpha} \rangle \ V \text{ bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle$$

and

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

✓

Case 17 $g = \bigsqcup\langle \overline{g} \rangle$, $\alpha = \bigsqcap\langle \overline{\alpha} \rangle$, and $(V = +) \vee (V = -)$:

Since $\Psi \vdash \bigsqcup\langle \overline{g} \rangle$ and $\sigma\big(\bigsqcap\langle \overline{\alpha} \rangle\big)$ compare V and $FV(\overline{\alpha}) \cap \{\overline{P}\} = \emptyset$,

$$\Psi \cup \{\overline{K}\} \vdash \bigsqcup\langle \overline{g} \rangle \text{ and } \bigsqcap\langle \overline{\alpha} \rangle \text{ compare } V$$

Let

$$\overline{l = \texttt{Bottom}}^{\#(\overline{K})} \text{ and } \overline{u = \texttt{Any}}^{\#(\overline{K})}$$

Then, by [M-Union-Intersection-Success],

$$\Psi \vdash \bigsqcup\langle \overline{g} \rangle \text{ match } [\![\overline{K}]\!] \bigsqcap\langle \overline{\alpha} \rangle \ V \text{ bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle$$

and

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

✓

Case 18 $g = \bigsqcap\langle \overline{g} \rangle$, $\alpha = \bigsqcap\langle \overline{\alpha} \rangle$, and $V = +$:

Since $\Psi \vdash \bigsqcap\langle \overline{g} \rangle$ and $\sigma\big(\bigsqcap\langle \overline{\alpha} \rangle\big)$ compare + and $FV(\overline{\alpha}) \cap \{\overline{P}\} = \emptyset$,

$$\Psi \cup \{\overline{K}\} \vdash \bigsqcap\langle \overline{g} \rangle \text{ and } \bigsqcap\langle \overline{\alpha} \rangle \text{ compare } +$$

Let

$$\overline{l = \texttt{Bottom}}^{\#(\overline{K})} \text{ and } \overline{u = \texttt{Any}}^{\#(\overline{K})}$$

Then, by [M-Intersection-Intersection-Success],

$$\Psi \vdash \bigsqcap\langle \overline{g} \rangle \text{ match } [\![\overline{K}]\!] \bigsqcap\langle \overline{\alpha} \rangle + \text{ bounds } \left\langle \langle \overline{l} \rangle \right\rangle \text{ and } \left\langle \langle \overline{u} \rangle \right\rangle$$

and

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l} \rangle \text{ and } \langle \overline{u} \rangle$$

✓

Case 19 $g = T[\![\overline{g}]\!]$, $\alpha = T[\![\overline{\alpha}]\!]$, and $V = =$:

Since $\Psi \vdash T[\![\overline{g}]\!]$ and $\sigma T[\![\overline{\alpha}]\!]$ compare =,

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha \text{ compare } =}$$

from which we can infer inductively that

$$\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]} \ \alpha = \text{ bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle}$$

and there exist $\overline{1 \le k \le \#\big(\overline{\langle \overline{l} \rangle}\big) = \#\big(\overline{\langle \overline{u} \rangle}\big)}^{1 \le j \le m}$ where $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{\underline{l_{jk_j}}} \rangle \text{ and } \langle \overline{\underline{u_{jk_j}}} \rangle$$

Therefore, by [M-Trait-In-Success],

$$\Psi \vdash T[\![\overline{g}]\!] \text{ match } [\![\overline{K}]\!] \ T[\![\overline{\alpha}]\!] = \text{ bounds } \bigsqcup\nolimits_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle \text{ and } \bigsqcap\nolimits_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$$

and by lemma C.1, there exists $1 \le i \le \#\big(\overline{\langle \overline{l'} \rangle}\big) = \#\big(\overline{\langle \overline{u'} \rangle}\big)$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{i}}} \rangle \text{ and } \langle \overline{u'_{\underline{i}}} \rangle$$

where $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$. ✓

Case 20 $g = G$, $\alpha = T[\![\overline{\alpha}]\!]$, and $V = +$:

Suppose $\texttt{trait } T[\![\overline{V' \ _ <: _}]\!] \ __ \ \texttt{end} \in \Psi$. Since $\Psi \vdash G$ and $\sigma T[\![\overline{\alpha}]\!]$ compare +, by lemma C.3, there exists $T[\![\overline{g'}]\!]$ such that

$$\Psi \vdash G <: T[\![\overline{g'}]\!] \text{ and } \Psi \vdash (G <: T[\![\overline{g''}]\!]) \Rightarrow (T[\![\overline{g'}]\!] <: T[\![\overline{g''}]\!])$$

Therefore, $\Psi \vdash T[\![\overline{g'}]\!]$ and $\sigma T[\![\overline{\alpha}]\!]$ compare + which implies

$$\overline{\underline{\Psi} \vdash g' \text{ and } \underline{\sigma}\alpha \text{ compare } V'}$$

from which we can infer inductively that

$$\overline{\underline{\Psi} \vdash g' \text{ match } \underline{[\![\overline{K}]\!]} \ \alpha \ V' \text{ bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle}$$

and there exist $\overline{1 \le k \le \#(\overline{\langle \overline{l} \rangle}) = \#(\overline{\langle \overline{u} \rangle})}^{1 \le j \le m}$ where $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{jk_j}} \rangle \text{ and } \langle \overline{u_{jk_j}} \rangle$$

Therefore, by [M-Trait-Co],

$$\Psi \vdash G \text{ match } [\![\overline{K}]\!]\ T[\![\overline{\alpha}]\!] \text{ + bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$$

and by lemma C.1, there exists $1 \le i \le \#(\overline{\langle \overline{l'} \rangle}) = \#(\overline{\langle \overline{u'} \rangle})$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_i} \rangle \text{ and } \langle \overline{u'_i} \rangle$$

where $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$. ✓

Case 21 $g = T[\![\overline{g}]\!]$, $\alpha = T'[\![\overline{\alpha'}]\!]$, and $V = -$:

Suppose `trait` $T[\![\overline{V' _ <: _}]\!] __ \text{ end} \in \Psi$. Since $\Psi \vdash T[\![\overline{g}]\!]$ and $\sigma T'[\![\overline{\alpha'}]\!]$ compare $-$, there exists $T[\![\overline{\alpha''}]\!]$ such that

$$\Psi \vdash \sigma T[\![\overline{\alpha''}]\!] \equiv T[\![\overline{g}]\!] \text{ and } \Psi \cup \{\overline{K}\} \vdash T'[\![\overline{\alpha'}]\!] <: T[\![\overline{\alpha''}]\!]$$

Therefore, by lemma C.3, there exists $T[\![\overline{\alpha}]\!]$ such that

$$\Psi \cup \{\overline{K}\} \vdash T'[\![\overline{\alpha'}]\!] <: T[\![\overline{\alpha}]\!] \text{ and } \Psi \cup \{\overline{K}\} \vdash (T'[\![\overline{\alpha'}]\!] <: T[\![\overline{\alpha'''}]\!]) \Rightarrow (T[\![\overline{\alpha}]\!] <: T[\![\overline{\alpha'''}]\!])$$

which implies

$$\Psi \cup \{\overline{K}\} \vdash T[\![\overline{\alpha}]\!] <: T[\![\overline{\alpha''}]\!]$$

Therefore, by lemma C.2,

$$\Psi \vdash T[\![\overline{g}]\!] \text{ and } \sigma T[\![\overline{\alpha}]\!] \text{ compare } -$$

which implies

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha \text{ compare } \mathit{flip}(V')}$$

from which we can infer inductively that

$$\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \alpha\ \mathit{flip}(V') \text{ bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle}$$

and there exist $\overline{1 \le k \le \#(\overline{\langle \overline{l} \rangle}) = \#(\overline{\langle \overline{u} \rangle})}^{1 \le j \le m}$ where $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{jk_j}} \rangle \text{ and } \langle \overline{u_{jk_j}} \rangle$$

Therefore, by [M-Trait-Contra],

$$\Psi \vdash T[\![\overline{g}]\!] \text{ match } [\![\overline{K}]\!]\ T'[\![\overline{\alpha'}]\!] \text{ - bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$$

and by lemma C.1, there exists $1 \le i \le \#(\overline{\langle \overline{l'} \rangle}) = \#(\overline{\langle \overline{u'} \rangle})$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_i} \rangle \text{ and } \langle \overline{u'_i} \rangle$$

where $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$. ✓

Case 22 $g = O[\![\overline{g}]\!]$ and $\alpha = O[\![\overline{\alpha}]\!]$:

Since $\Psi \vdash O[\![\overline{g}]\!]$ and $\sigma O[\![\overline{\alpha}]\!]$ compare V,

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha \text{ compare } =}$$

from which we can infer inductively that

$$\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \alpha = \text{bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle}$$

and there exist $\overline{1 \le k \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)}^{1 \le j \le m}$ where $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{jk_j}} \rangle \text{ and } \langle \overline{u_{jk_j}} \rangle$$

Therefore, by [M-Object-Object-Same],

$$\Psi \vdash O[\![\overline{g}]\!] \text{ match } [\![\overline{K}]\!]\ O[\![\overline{\alpha}]\!]\ V \text{ bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$$

and by lemma C.1, there exists $1 \le i \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_i} \rangle \text{ and } \langle \overline{u'_i} \rangle$$

where $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$. ✓

Case 23 $g = T[\![\overline{g}]\!]$, $\alpha = O[\![\overline{\alpha}]\!]$, and $V = -$:

Suppose $\texttt{trait}\ T[\![\overline{V' \ _ <: _}]\!] __ \ \texttt{end} \in \Psi$. Since $\Psi \vdash T[\![\overline{g}]\!]$ and $\sigma O[\![\overline{\alpha}]\!]$ compare $-$, there exists $T[\![\overline{\alpha''}]\!]$ such that

$$\Psi \vdash \sigma T[\![\overline{\alpha''}]\!] \equiv T[\![\overline{g}]\!] \text{ and } \Psi \cup \{\overline{K}\} \vdash O[\![\overline{\alpha}]\!] <: T[\![\overline{\alpha''}]\!]$$

Therefore, by lemma C.3, there exists $T[\![\overline{\alpha'}]\!]$ such that

$$\Psi \cup \{\overline{K}\} \vdash O[\![\overline{\alpha}]\!] <: T[\![\overline{\alpha'}]\!] \text{ and } \Psi \cup \{\overline{K}\} \vdash (O[\![\overline{\alpha'}]\!] <: T[\![\overline{\alpha'''}]\!]) \Rightarrow (T[\![\overline{\alpha'}]\!] <: T[\![\overline{\alpha'''}]\!])$$

which implies

$$\Psi \cup \{\overline{K}\} \vdash T[\![\overline{\alpha'}]\!] <: T[\![\overline{\alpha''}]\!]$$

Therefore, by lemma C.2,

$$\Psi \vdash T[\![\overline{g}]\!] \text{ and } \sigma T[\![\overline{\alpha'}]\!] \text{ compare } -$$

which implies

$$\overline{\underline{\Psi} \vdash g \text{ and } \underline{\sigma}\alpha' \text{ compare } \mathit{flip}(V')}$$

from which we can infer inductively that

$$\overline{\underline{\Psi} \vdash g \text{ match } \underline{[\![\overline{K}]\!]}\ \alpha'\ \mathit{flip}(V') \text{ bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle}$$

and there exist $\overline{1 \le k \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)}^{1 \le j \le m}$ where $m = \#\left(\overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle}\right) = \#\left(\overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle}\right)$ such that for each j,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{jk_j}} \rangle \text{ and } \langle \overline{u_{jk_j}} \rangle$$

Therefore, by [M-Trait-Object-Contra],

$$\Psi \vdash T[\![\overline{g}]\!] \text{ match } [\![\overline{K}]\!]\ O[\![\overline{\alpha}]\!]\ - \text{ bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$$

and by lemma C.1, there exists $1 \le i \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_i} \rangle \text{ and } \langle \overline{u'_i} \rangle$$

where $\left\langle \overline{\langle \overline{l'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{l} \rangle} \right\rangle} \right\rangle$ and $\left\langle \overline{\langle \overline{u'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{u} \rangle} \right\rangle} \right\rangle$.

✓

□

Proof of Lemma 5.3

Proof. By induction, according to whichever proof rule was used for solve:

Rule [Solve-Empty]: Since

$$\Psi \vdash [\,] \text{ on } \langle\rangle \text{ obeys } \langle\rangle \text{ and } \langle\rangle$$

is satisfied trivially, this case is trivial. ✓

Rule [Solve-Step]: Suppose that

$$\overline{K = \{\overline{\chi}\} <: P <: \{\overline{\eta}\}}$$

and

$$\Psi \vdash \text{solve } [\![\overline{K}, \{\overline{\chi'}\} <: P' <: \{\overline{\eta'}\}]\!] \text{ bounds } \left\langle \overline{\langle \bar{l}, l' \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u}, u' \rangle} \right\rangle \text{ using } [\overline{g/P}, g'/P']$$

Let $\sigma = [\overline{g/P}, g'/P']$ and $\sigma' = [\overline{g/P}]$. From the premises, there exists $1 \le i \le \#\big(\overline{\langle \bar{l}, l' \rangle}\big) = \#\big(\overline{\langle \overline{u}, u' \rangle}\big)$ such that

$$g' = \big(u'_i \sqcap \bigsqcap \{\overline{\eta'}\}\big)$$

It follows that

$$\Psi \vdash g' <: u'_i \text{ and } \Psi \vdash g' <: \bigsqcap \{\overline{\eta'}\}$$

Since $\{\overline{P}, P'\} \cap FV(u'_i, \overline{\eta'}) = \emptyset$,

$$\Psi \vdash g' <: \sigma u'_i \tag{1}$$

and

$$\Psi \vdash g' <: \sigma\big(\bigsqcap \{\overline{\eta'}\}\big) \tag{2}$$

Since $\Psi \vdash l'_i <: g'$ from the premises and $FV(l'_i) = \emptyset$,

$$\Psi \vdash \sigma l'_i <: g' \tag{3}$$

For

$$\overline{\underline{\Psi} \vdash \underline{g'} \text{ match } \underline{[\![\overline{K}]\!]}\ \chi' \text{ - bounds } \left\langle \overline{\langle \overline{l''} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u''} \rangle} \right\rangle}$$

let

$$\left\langle \overline{\langle \overline{l'''} \rangle} \right\rangle = \bigsqcup\nolimits^{\times}_{\overline{K}} \left\langle \overline{\left\langle \overline{\langle \overline{l''} \rangle}, \langle \underline{\overline{l_i}} \rangle \right\rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u'''} \rangle} \right\rangle = \bigsqcap\nolimits^{\times}_{\overline{K}} \left\langle \overline{\left\langle \overline{\langle \overline{u''} \rangle}, \langle \underline{\overline{u_i}} \rangle \right\rangle} \right\rangle$$

From

$$\Psi \vdash \text{solve } [\![\overline{K}]\!] \text{ bounds } \left\langle \overline{\langle \overline{l'''} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u'''} \rangle} \right\rangle \text{ using } \sigma'$$

we can infer inductively that

$$\Psi \vdash \sigma' \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{\bigsqcup \{\overline{\chi}\}} \rangle \text{ and } \langle \overline{\bigsqcap \{\overline{\eta}\}} \rangle \tag{4}$$

and there exists $1 \le j \le \#\big(\overline{\langle \overline{l'''} \rangle}\big) = \#\big(\overline{\langle \overline{u'''} \rangle}\big)$ such that

$$\Psi \vdash \sigma' \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \underline{\overline{l'''_j}} \rangle \text{ and } \langle \underline{\overline{u'''_j}} \rangle$$

Therefore, by lemma C.1,

$$\Psi \vdash \sigma' \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \underline{\overline{l_i}} \rangle \text{ and } \langle \underline{\overline{u_i}} \rangle \tag{5}$$

and there exist $\overline{1 \le k \le \#\big(\overline{\langle \overline{l''} \rangle}\big) = \#\big(\overline{\langle \overline{u''} \rangle}\big)}^{\,1 \le i' \le m}$ such that

$$\Psi \vdash \sigma' \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \underline{\overline{l''_{i'k_{i'}}}} \rangle \text{ and } \langle \underline{\overline{u''_{i'k_{i'}}}} \rangle$$

where $m = \#\left(\left\langle \overline{\langle \overline{l''} \rangle} \right\rangle\right) = \#\left(\left\langle \overline{\langle \overline{u''} \rangle} \right\rangle\right) = \#(\overline{\chi'})$. Therefore, by applying lemma 5.1 to each match judgement premise,

$$\overline{\underline{\Psi} \vdash \underline{g'} \text{ and } \underline{\sigma'\chi'} \text{ compare -}}$$

which implies

$$\Psi \vdash \sigma'\big(\bigsqcup \{\overline{\chi'}\}\big) <: g'$$

Since $P' \notin FV(\overline{\chi'})$,

$$\Psi \vdash \sigma\big(\bigsqcup\{\overline{\chi'}\}\big) <: g' \tag{6}$$

Since $\underline{\{P'\} \cap FV(\overline{\chi}, \overline{\eta}, \underline{l_{\underline{i}}}, \underline{u_{\underline{i}}}) = \emptyset}$,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P}, P' \rangle \text{ obeys } \langle \overline{\underline{l_{\underline{i}}}}, l'_i \rangle \text{ and } \langle \overline{\underline{u_{\underline{i}}}}, u'_i \rangle$$

from (1), (3), and (5), and

$$\Psi \vdash \sigma \text{ on } \langle \overline{P}, P' \rangle \text{ obeys } \langle \overline{\bigsqcup\{\overline{\chi}\}}, \bigsqcup\{\overline{\chi'}\} \rangle \text{ and } \langle \overline{\bigsqcap\{\overline{\eta}\}}, \bigsqcap\{\overline{\eta'}\} \rangle$$

from (2), (4), and (6). ✓

And so all cases are covered. □

Proof of Lemma 5.4

Proof. We prove by induction on $\#(\overline{P})$.
If $\#(\overline{P}) = 0$,

$$\Psi \vdash \text{solve } [\![\,]\!] \text{ bounds } \big\langle \langle\rangle \big\rangle \text{ and } \big\langle \langle\rangle \big\rangle \text{ using } [\,]$$

is satisfied trivially by **Rule** [Solve-Empty].
Now, suppose that the statement holds for $\#(\overline{P}) = n$ where $n \geq 0$, and we show that it holds for $\#(\overline{P}, P') = n + 1$. Let $\sigma = [\overline{g/P}, g'/P']$ and $\sigma' = [\overline{g/P}]$, and suppose that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P}, P' \rangle \text{ obeys } \langle \overline{\bigsqcup\{\overline{\chi}\}}, \bigsqcup\{\overline{\chi'}\} \rangle \text{ and } \langle \overline{\bigsqcap\{\overline{\eta}\}}, \bigsqcap\{\overline{\eta'}\} \rangle$$

$$\Psi \vdash \sigma \text{ on } \langle \overline{P}, P' \rangle \text{ obeys } \langle \overline{l}, l' \rangle \text{ and } \langle \overline{u}, u' \rangle$$

and

$$\langle \overline{l}, l' \rangle = \langle \overline{l''_{\underline{i}}}, l'''_i \rangle \text{ and } \langle \overline{u}, u' \rangle = \langle \overline{u''_{\underline{i}}}, u'''_i \rangle$$

for some $\big\langle \overline{\langle \overline{l''}, l''' \rangle} \big\rangle$, $\big\langle \overline{\langle \overline{u''}, u''' \rangle} \big\rangle$ and some $1 \leq i \leq \#(\overline{\langle \overline{l''}, l''' \rangle}) = \#(\overline{\langle \overline{u''}, u''' \rangle})$. From the hypothesis,

$$\Psi \vdash \sigma l' <: g' \text{ and } \overline{\underline{\Psi} \vdash \underline{\sigma \chi'} <: \underline{g'}}$$

$$\Psi \vdash g' <: \sigma u' \text{ and } \overline{\underline{\Psi} \vdash \underline{g'} <: \underline{\sigma \eta'}}$$

Then, since $FV(l', u', \overline{\eta'}) = \emptyset$ and $\{P'\} \cap FV(\overline{\chi'}) = \emptyset$,

$$\Psi \vdash l' <: g' \text{ and } \Psi \vdash g' <: (u' \sqcap \bigsqcap \{\overline{\eta'}\}) \tag{1}$$

and

$$\overline{\underline{\Psi} \vdash \underline{g'} \text{ and } \underline{\sigma' \chi'} \text{ compare } -} \tag{2}$$

Also, since $\underline{\{P'\} \cap FV(\overline{\chi}, \overline{\eta}, l, u) = \emptyset}$, $\overline{l = l''_{\underline{i}}}$ and $\overline{u = u''_{\underline{i}}}$,

$$\Psi \vdash \sigma' \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{\bigsqcup\{\overline{\chi}\}} \rangle \text{ and } \langle \overline{\bigsqcap\{\overline{\eta}\}} \rangle \tag{3}$$

and

$$\Psi \vdash \sigma' \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l''_{\underline{i}}} \rangle \text{ and } \langle \overline{u''_{\underline{i}}} \rangle \tag{4}$$

For $\overline{K = \{\overline{\chi}\} <: P <: \{\overline{\eta}\}}$ and $m = \#(\overline{\chi'})$, by applying lemma 5.2 to (2) and (3),

$$\overline{\underline{\Psi} \vdash \underline{g'} \text{ match } \underline{[\![\overline{K}]\!]}\ \chi' - \text{ bounds } \big\langle \overline{\langle \overline{a} \rangle} \big\rangle \text{ and } \big\langle \overline{\langle \overline{b} \rangle} \big\rangle} \tag{5}$$

and there exist $1 \le k \le \#\left(\overline{\langle \overline{a} \rangle}\right) = \#\left(\overline{\langle \overline{\overline{b}} \rangle}\right)^{1 \le j \le m}$ such that for each j,

$$\Psi \vdash \sigma' \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{a_{\underline{jk_j}}} \rangle \text{ and } \langle \overline{b_{\underline{jk_j}}} \rangle \tag{6}$$

By applying lemma C.1 to (4) and (6), there exists $1 \le j' \le \#\left(\overline{\langle \overline{a'} \rangle}\right) = \#\left(\overline{\langle \overline{\overline{b'}} \rangle}\right)$ such that

$$\Psi \vdash \sigma' \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{a'_{\underline{j'}}} \rangle \text{ and } \langle \overline{b'_{\underline{j'}}} \rangle \tag{7}$$

where $\left\langle \overline{\langle \overline{a'} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{a} \rangle} \right\rangle}, \left\langle \langle \overline{l''_{\underline{i}}} \rangle \right\rangle \right\rangle$ and $\left\langle \overline{\langle \overline{b'} \rangle} \right\rangle = \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{b} \rangle} \right\rangle}, \left\langle \langle \overline{u''_{\underline{i}}} \rangle \right\rangle \right\rangle$. Then we can infer inductively from (3) and (7) that there exists σ'' such that

$$\Psi \vdash \text{solve } [\![\overline{K}]\!] \text{ bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{a} \rangle} \right\rangle}, \left\langle \langle \overline{l''_{\underline{i}}} \rangle \right\rangle \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \overline{\left\langle \overline{\langle \overline{b} \rangle} \right\rangle}, \left\langle \langle \overline{u''_{\underline{i}}} \rangle \right\rangle \right\rangle \text{ using } \sigma'' \tag{8}$$

Since $FV(u', \overline{\eta'}) = \emptyset$, $(u' \sqcap \bigsqcap \{\overline{\eta'}\})$ is a ground type. Let $g'' = (u' \sqcap \bigsqcap \{\overline{\eta'}\})$, then since $l'''_i = l'$, and $\Psi \vdash l' <: g'$ and $\Psi \vdash g' <: (u' \sqcap \bigsqcap \{\overline{\eta'}\})$ from (1),

$$\Psi \vdash l'''_i <: g'' \tag{9}$$

By **Rule** [Solve-Step], from (5), (8), and (9),

$$\Psi \vdash \text{solve } [\![\overline{K}, \{\overline{\chi}'\} <: P' <: \{\overline{\eta}'\}]\!] \text{ bounds } \left\langle \overline{\langle \overline{l''}, l''' \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u''}, u''' \rangle} \right\rangle \text{ using } \sigma'''$$

where $\sigma''' = \sigma'' \circ [g''/P']$. □

Proof of Theorem 5.5

Proof. Since $\Psi \vdash k$ and g dispatch d using σ,

$$\Psi \vdash k \text{ match } [\![\overline{K}]\!] \ \alpha \text{ + bounds } \left\langle \overline{\langle \overline{l} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u} \rangle} \right\rangle$$
$$\Psi \vdash g \text{ match } [\![\overline{K}]\!] \ \rho \text{ - bounds } \left\langle \overline{\langle \overline{l'} \rangle} \right\rangle \text{ and } \left\langle \overline{\langle \overline{u'} \rangle} \right\rangle$$

and

$$\Psi \vdash \text{solve } [\![\overline{K}]\!] \text{ bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \left\langle \overline{\langle \overline{l} \rangle} \right\rangle, \left\langle \overline{\langle \overline{l'} \rangle} \right\rangle \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \left\langle \overline{\langle \overline{u} \rangle} \right\rangle, \left\langle \overline{\langle \overline{u'} \rangle} \right\rangle \right\rangle \text{ using } \sigma$$

Let

$$\left\langle \overline{\langle \overline{l''} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \left\langle \overline{\langle \overline{l} \rangle} \right\rangle, \left\langle \overline{\langle \overline{l'} \rangle} \right\rangle \right\rangle \text{ and } \left\langle \overline{\langle \overline{u''} \rangle} \right\rangle = \bigsqcup_{\overline{K}}^{\times} \left\langle \left\langle \overline{\langle \overline{u} \rangle} \right\rangle, \left\langle \overline{\langle \overline{u'} \rangle} \right\rangle \right\rangle$$

with $n = \#\left(\overline{\langle \overline{l''} \rangle}\right) = \#\left(\overline{\langle \overline{u''} \rangle}\right)$. By lemma 5.3,

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{\bigsqcup \{\overline{\chi}\}} \rangle \text{ and } \langle \overline{\bigsqcap \{\overline{\eta}\}} \rangle$$

and there exists $1 \le i \le n$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l''_{\underline{i}}} \rangle \text{ and } \langle \overline{u''_{\underline{i}}} \rangle$$

By lemma C.1, there exist $1 \le j \le \#\left(\overline{\langle \overline{l} \rangle}\right) = \#\left(\overline{\langle \overline{u} \rangle}\right)$ and $1 \le j' \le \#\left(\overline{\langle \overline{l'} \rangle}\right) = \#\left(\overline{\langle \overline{u'} \rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l_{\underline{j}}} \rangle \text{ and } \langle \overline{u_{\underline{j}}} \rangle \text{ and } \Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{l'_{\underline{j'}}} \rangle \text{ and } \langle \overline{u'_{\underline{j'}}} \rangle$$

By lemma 5.1,

$$\Psi \vdash k \text{ and } \sigma\alpha \text{ compare +}$$

and

$$\Psi \vdash g \text{ and } \sigma\rho \text{ compare } -$$

□

Proof of Theorem 5.6

Proof. Note that for $arrow(d) = \forall[\![\overline{K}]\!](\alpha \to \rho)$, as mentioned in Section 5.2.5, α and ρ do not contain `Bottom`, and any union types and intersection types they contain do not contain any type parameters. Thus, for all $\mathcal{T}$ and $\mathcal{T}'$,

$$\alpha \neq \mathcal{T}\ \boxed{\texttt{Bottom}},\ \overline{(\underline{\alpha} = \mathcal{T}\ \boxed{\bigsqcup\langle\overline{\gamma}\rangle}) \Rightarrow (\overline{\gamma \neq \mathcal{T}'\ \boxed{\underline{P}}})},\text{ and } \overline{(\underline{\alpha} = \mathcal{T}\ \boxed{\bigsqcap\langle\overline{\gamma}\rangle}) \Rightarrow (\overline{\gamma \neq \mathcal{T}'\ \boxed{\underline{P}}})}$$
$$\rho \neq \mathcal{T}\ \boxed{\texttt{Bottom}},\ \overline{(\underline{\rho} = \mathcal{T}\ \boxed{\bigsqcup\langle\overline{\gamma}\rangle}) \Rightarrow (\overline{\gamma \neq \mathcal{T}'\ \boxed{\underline{P}}})},\text{ and } \overline{(\underline{\rho} = \mathcal{T}\ \boxed{\bigsqcap\langle\overline{\gamma}\rangle}) \Rightarrow (\overline{\gamma \neq \mathcal{T}'\ \boxed{\underline{P}}})}$$

Since

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \left\langle\overline{\bigsqcup\{\overline{\chi}\}}\right\rangle \text{ and } \left\langle\overline{\bigsqcap\{\overline{\eta}\}}\right\rangle$$

and

$$\Psi \vdash k \text{ and } \sigma\alpha \text{ compare } +$$

lemma 5.2 implies that

$$\Psi \vdash k \text{ match } [\![\overline{K}]\!]\ \alpha + \text{ bounds } \left\langle\overline{\langle\overline{l}\rangle}\right\rangle \text{ and } \left\langle\overline{\langle\overline{u}\rangle}\right\rangle \tag{1}$$

and

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l_{\underline{i}}}\rangle \text{ and } \langle\overline{u_{\underline{i}}}\rangle$$

for some $1 \le i \le \#(\overline{\langle\overline{l}\rangle}) = \#(\overline{\langle\overline{u}\rangle})$. Also, since

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \left\langle\overline{\bigsqcup\{\overline{\chi}\}}\right\rangle \text{ and } \left\langle\overline{\bigsqcap\{\overline{\eta}\}}\right\rangle$$

and

$$\Psi \vdash g \text{ and } \sigma\rho \text{ compare } -$$

lemma 5.2 implies that

$$\Psi \vdash g \text{ match } [\![\overline{K}]\!]\ \rho - \text{ bounds } \left\langle\overline{\langle\overline{l'}\rangle}\right\rangle \text{ and } \left\langle\overline{\langle\overline{u'}\rangle}\right\rangle \tag{2}$$

and

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l'_{\underline{j}}}\rangle \text{ and } \langle\overline{u'_{\underline{j}}}\rangle$$

for some $1 \le j \le \#(\overline{\langle\overline{l'}\rangle}) = \#(\overline{\langle\overline{u'}\rangle})$. Therefore, by lemma C.1, there exists $1 \le k \le \#\left(\overline{\langle\overline{l''}\rangle}\right) = \#\left(\overline{\langle\overline{u''}\rangle}\right)$ such that

$$\Psi \vdash \sigma \text{ on } \langle\overline{P}\rangle \text{ obeys } \langle\overline{l''_{\underline{k}}}\rangle \text{ and } \langle\overline{u''_{\underline{k}}}\rangle$$

where

$$\left\langle\overline{\langle\overline{l''}\rangle}\right\rangle = \bigsqcup_{\overline{K}}^{\times}\left\langle\left\langle\overline{\langle\overline{l}\rangle}\right\rangle, \left\langle\overline{\langle\overline{l'}\rangle}\right\rangle\right\rangle \text{ and } \left\langle\overline{\langle\overline{u''}\rangle}\right\rangle = \bigsqcap_{\overline{K}}^{\times}\left\langle\left\langle\overline{\langle\overline{u}\rangle}\right\rangle, \left\langle\overline{\langle\overline{u'}\rangle}\right\rangle\right\rangle$$

By lemma 5.4, there exists a substitution σ' such that

$$\Psi \vdash \text{solve } [\![\overline{K}]\!] \text{ bounds } \bigsqcup_{\overline{K}}^{\times} \left\langle \left\langle \overline{\langle \overline{l} \rangle} \right\rangle, \left\langle \overline{\langle \overline{l'} \rangle} \right\rangle \right\rangle \text{ and } \bigsqcap_{\overline{K}}^{\times} \left\langle \left\langle \overline{\langle \overline{u} \rangle} \right\rangle, \left\langle \overline{\langle \overline{u'} \rangle} \right\rangle \right\rangle \text{ using } \sigma' \tag{3}$$

Therefore, we can conclude from (1), (2), and (3) that

$$\Psi \vdash k \text{ and } g \text{ dispatch } d \text{ using } \sigma'$$

□

Proof of Lemma 5.7

Proof. Let $arrow(d) = \forall [\![\overline{K_1}]\!](\alpha_1 \to \rho_1)$ and $arrow(d_0) = \forall [\![\overline{K_2}]\!](\alpha_2 \to \rho_2)$.
Since $\Delta \vdash dom(d) \sqsubseteq dom(d_0)$, by [Return-Test],

$$\Delta \vdash \forall [\![\overline{K_1}]\!](\alpha_1 \to \rho_1) \sqsubseteq \forall [\![\overline{K_1}, \overline{K_2}]\!]((\alpha_1 \sqcap \alpha_2) \to \rho_2) \text{ using } \sigma'$$

which implies

$$\Delta \vdash \forall [\![\overline{K_1}, \overline{K_2}]\!]((\alpha_1 \sqcap \alpha_2) \to \rho_2) \xrightarrow{\equiv} \forall [\![\overline{K_3}]\!](\alpha_3 \to \rho_3) \text{ using } \sigma_r \tag{1}$$

$$\Delta \vdash \forall [\![\overline{K_1}]\!](\alpha_1 \to \rho_1) \lesssim \forall [\![\overline{K_3}]\!](\alpha_3 \to \rho_3) \text{ using } \sigma' \tag{2}$$

If $D_0 \in \left\{ \overline{instantiate(\underline{d_0}, \sigma_0)} \right\}$, $arrow(D_0) = \forall [\![ ]\!](\sigma_0\alpha_2 \to \sigma_0\rho_2)$ for some $\sigma_0 \in \{\overline{\sigma_0}\}$ and

$$\Delta \vdash \alpha <: \sigma_0\alpha_2.$$

For any $\sigma \in \{\overline{\sigma}\}$,

$$\Delta \vdash \alpha <: \sigma\alpha_1,$$

thus

$$\Delta \vdash \alpha <: (\sigma \circ \sigma_0)(\alpha_1 \sqcap \alpha_2),$$

therefore $(\sigma \circ \sigma_0)(\alpha_1 \sqcap \alpha_2)$ is not `Bottom`. Since σ_r represents the minimum requirement that prevents $\sigma_r(\alpha_1 \sqcap \alpha_2)$ from being `Bottom`, there exists σ'' such that $\sigma \circ \sigma_0 = \sigma'' \circ \sigma_r$. Note that it follows from (1) that

$$\Delta \cup \{\overline{K_3}\} \vdash \sigma_r(\alpha_1 \sqcap \alpha_2) \equiv \alpha_3 \text{ and } \Delta \cup \{\overline{K_3}\} \vdash \sigma_r\rho_2 \equiv \rho_3,$$

and it follows from (2) that

$$\Delta \cup \{\overline{K_3}\} \vdash \alpha_3 <: \sigma'\alpha_1 \text{ and } \Delta \cup \{\overline{K_3}\} \vdash \sigma'\rho_1 <: \rho_3.$$

By combining these relations, we have

$$\Delta \cup \{\overline{K_3}\} \vdash \sigma_r(\alpha_1 \sqcap \alpha_2) <: \sigma'\alpha_1 \text{ and } \Delta \cup \{\overline{K_3}\} \vdash \sigma'\rho_1 <: \sigma_r\rho_2$$

and by lemma C.2,

$$\Delta \vdash (\sigma \circ \sigma_0)(\alpha_1 \sqcap \alpha_2) <: (\sigma'' \circ \sigma')\alpha_1 \text{ and } \Delta \vdash (\sigma'' \circ \sigma')\rho_1 <: (\sigma \circ \sigma_0)\rho_2$$

Hence,

$$\Delta \vdash \alpha <: (\sigma'' \circ \sigma')\alpha_1 \text{ and } \Delta \vdash (\sigma'' \circ \sigma')\rho_1 <: \sigma_0\rho_2$$

since $\sigma\rho_2 = \rho_2$. Therefore, we have $\sigma'' \circ \sigma' \in \{\overline{\sigma}\}$ and there exists $D = instantiate(d, \sigma'' \circ \sigma')$ with $arrow(D) = \forall [\![ ]\!](_ \to (\sigma'' \circ \sigma')\rho_1)$ such that $\Delta \vdash (\sigma'' \circ \sigma')\rho_1 <: \sigma_0\rho_2$. □

Proof of Lemma 5.8

Proof. Since $d_0 \in \{\overline{d_0'}\} = \mathit{visible}(\Delta, (C, m))$, $d_0 \in \mathit{methods}(\Delta, C')$ for some $C' \in \mathit{ancestors}(\Delta, C)$. Since $\Psi \vdash G <: \sigma C$, $\sigma C \in \mathit{ancestors}(\Psi, G)$, and by lemma C.2, $\sigma C' \in \mathit{ancestors}(\Psi, G)$. Hence,

$$\sigma d_0 \in \{\overline{d'}\} = \mathit{visible}(\Psi, (G, m)).$$

Let $\mathit{arrow}(d_0) = \forall[\![\, \overline{K_2} \,]\!](\alpha_2 \to \rho_2)$. From $(d_0, \{\overline{\sigma_0}\}) \in \mathit{applicable}(\Delta, \{\overline{d_0'}\}, \alpha)$, it follows that

$$\overline{\underline{\Delta} \vdash \exists[\![\,]\!]\underline{\alpha} \sqsubseteq \exists\underline{[\![\, \overline{K_2} \,]\!]}\,\underline{\alpha_2} \text{ using } \sigma_0}$$

which implies

$$\overline{\underline{\Delta} \vdash \underline{\alpha} <: \sigma_0 \underline{\alpha_2}}$$

By lemma C.2,

$$\overline{\underline{\Psi} \vdash \underline{\sigma\alpha} <: \underline{\sigma}\sigma_0\underline{\alpha_2}}$$

thus

$$\overline{\underline{\Psi} \vdash \exists[\![\,]\!]\underline{k} \sqsubseteq \exists\underline{[\![\, \overline{K_2} \,]\!]}\,\underline{\sigma\alpha_2} \text{ using } \sigma_0}.$$

Therefore, $(\sigma d_0, \{\overline{\sigma_0'}\}) \in \mathit{applicable}(\Psi, \{\overline{d'}\}, k)$ with $\{\overline{\sigma_0}\} \subseteq \{\overline{\sigma_0'}\}$. If

$$D_0 \in \left\{\overline{\mathit{instantiate}(\underline{d_0}, \sigma_0)}\right\} \text{ with } \mathit{arrow}(D_0) = \forall[\![\,]\!](_ \to \rho_0),$$

then

$$\sigma D_0 \in \left\{\overline{\mathit{instantiate}(\underline{\sigma d_0}, \sigma_0')}\right\} \text{ with } \mathit{arrow}(\sigma D_0) = \forall[\![\,]\!](_ \to \sigma\rho_0),$$

and by lemma 5.7,

$$\exists D \in \left\{\overline{\mathit{instantiate}(\underline{d}, \sigma)}\right\} \text{ with } \mathit{arrow}(D) = \forall[\![\,]\!](_ \to h) \text{ such that } \Psi \vdash h <: \sigma\rho_0.$$

□

Proof of Lemma 5.9

Proof. Let $\mathit{arrow}(d) = \forall[\![\, \overline{\{\overline{\chi}\} <: P <: \{\overline{\eta}\}} \,]\!](\alpha \to \rho)$. If $\mathit{arrow}(D) = \forall[\![\,]\!](\sigma\alpha \to \sigma\rho)$ for some $\sigma \in \{\overline{\sigma}\}$ with $\Psi \vdash \sigma\rho <: g$, since $D \in \left\{\overline{\mathit{instantiate}(\underline{d}, \sigma)}\right\}$,

$$\Psi \vdash k <: \sigma\alpha \text{ and } \Psi \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \left\langle \overline{\bigsqcup\{\overline{\chi}\}} \right\rangle \text{ and } \left\langle \overline{\bigsqcap\{\overline{\eta}\}} \right\rangle.$$

Therefore, by theorem 5.6,

$$\Psi \vdash k \text{ and } g \text{ dispatch } d \text{ using } \sigma'$$

for some σ'. Since d is the most specific method declaration among $\{\overline{d'}\}$ that is applicable to k and has an instance whose return type is a subtype of g,

$$\mathit{firstDispatchable}(\Psi, \mathit{topologicalSort}(\Psi, \{\overline{d'}\}), k, g) = D_1$$

with $\mathit{arrow}(D_1) = \forall[\![\,]\!](\sigma'\alpha \to \sigma'\rho)$. By theorem 5.5,

$$\Psi \vdash \sigma' \text{ on } \langle \overline{P} \rangle \text{ obeys } \left\langle \overline{\bigsqcup\{\overline{\chi}\}} \right\rangle \text{ and } \left\langle \overline{\bigsqcap\{\overline{\eta}\}} \right\rangle \text{ and } \Psi \vdash k <: \sigma'\alpha \text{ and } \Psi \vdash \sigma'\rho <: g,$$

thus $D_1 \in \left\{\overline{\mathit{instantiate}(\underline{d}, \sigma)}\right\}$. □

Proof of Theorem 5.10

Proof. Let $\{\overline{d'}\} = \mathit{visible}(\Psi, (O[\![\overline{g''}]\!], m))$, and $(d, \{\overline{\sigma}\}) \in \mathit{mostSpecific}(\Psi, \mathit{applicable}(\Psi, \{\overline{d'}\}, (O[\![\overline{g''}]\!], \overline{\mathit{ilk}(v')}$ If $\Psi; \bullet \vdash O[\![\overline{g''}]\!](\overline{v}).m(\overline{v'}) : (_, g)$, by rule [T-Method],

$$\overline{\underline{\Psi}; \bullet \vdash v' : (_, \alpha)} \text{ and } \mathit{msav}(\Psi, (O[\![\overline{g''}]\!], m), (O[\![\overline{g''}]\!], \overline{\alpha})) = D_0$$

where $\mathit{arrow}(D_0) = \forall[\![\,]\!](_ \to g)$. By rule [MSAV],

$$(d_0, \{\overline{\sigma_0}\}) \in \mathit{mostSpecific}(\Psi, \mathit{applicable}(\Psi, \{\overline{d'}\}, (O[\![\overline{g''}]\!], \overline{\alpha}))) \text{ and } D_0 \in \left\{\overline{\mathit{instantiate}(\underline{d_0}, \sigma_0)}\right\}.$$

Since $\Psi \vdash (O[\![\overline{g''}]\!], \overline{\mathit{ilk}(v')}) <: (O[\![\overline{g''}]\!], \overline{\alpha})$ by theorem 5.11, by lemma 5.8,

$$\exists D_1 \in \left\{\overline{\mathit{instantiate}(\underline{d}, \sigma)}\right\} \text{ with } \mathit{arrow}(D_1) = \forall[\![\,]\!](_ \to h) \text{ such that } \Psi \vdash h <: g$$

Therefore, by lemma 5.9,

$$D = (_, m[\![\,]\!](\overline{x' : _}) : _ = \epsilon) = \mathit{firstDispatchable}(\Psi, \langle \overline{d} \rangle, (O[\![\overline{g''}]\!], \overline{\mathit{ilk}(v')}), g)$$
$$\text{with } \mathit{arrow}(D) = \forall[\![\,]\!](g_1 \to g_2) \text{ and } \Psi \vdash (\overline{\mathit{ilk}(v')}) <: g_1 \text{ and } \Psi \vdash g_2 <: g$$

When

$$\Psi; \bullet \vdash [O[\![\overline{g''}]\!](\overline{v})/\mathtt{self}][\overline{v/x}][\overline{v'/x'}]\epsilon : (_, g'),$$

since substitution preserves type, $\Psi \vdash g' <: g_2$, therefore $\Psi \vdash g' <: g$. □

Proof of Theorem 5.11

Proof Sketch. Type soundness follows from subject reduction, that is if $\Psi; \bullet \vdash e : (\epsilon, g)$ and $\Psi \vdash \epsilon \longrightarrow \epsilon'$ then $\Psi; \bullet \vdash \epsilon' : (_, g'')$ with $\Psi \vdash g'' <: g$, and progress, that is if $\Psi; \bullet \vdash e : (\epsilon, g)$, then either ϵ is a value or $\exists \epsilon'$ such that $\Psi \vdash \epsilon \longrightarrow \epsilon'$. The proof of both subject reduction and progress is a case analysis on reduction $\Psi \vdash \epsilon \longrightarrow \epsilon'$. Since the closure application case is conventional, we focus on method invocation case, which follows from theorem 5.10. □